Sign up for the platform:

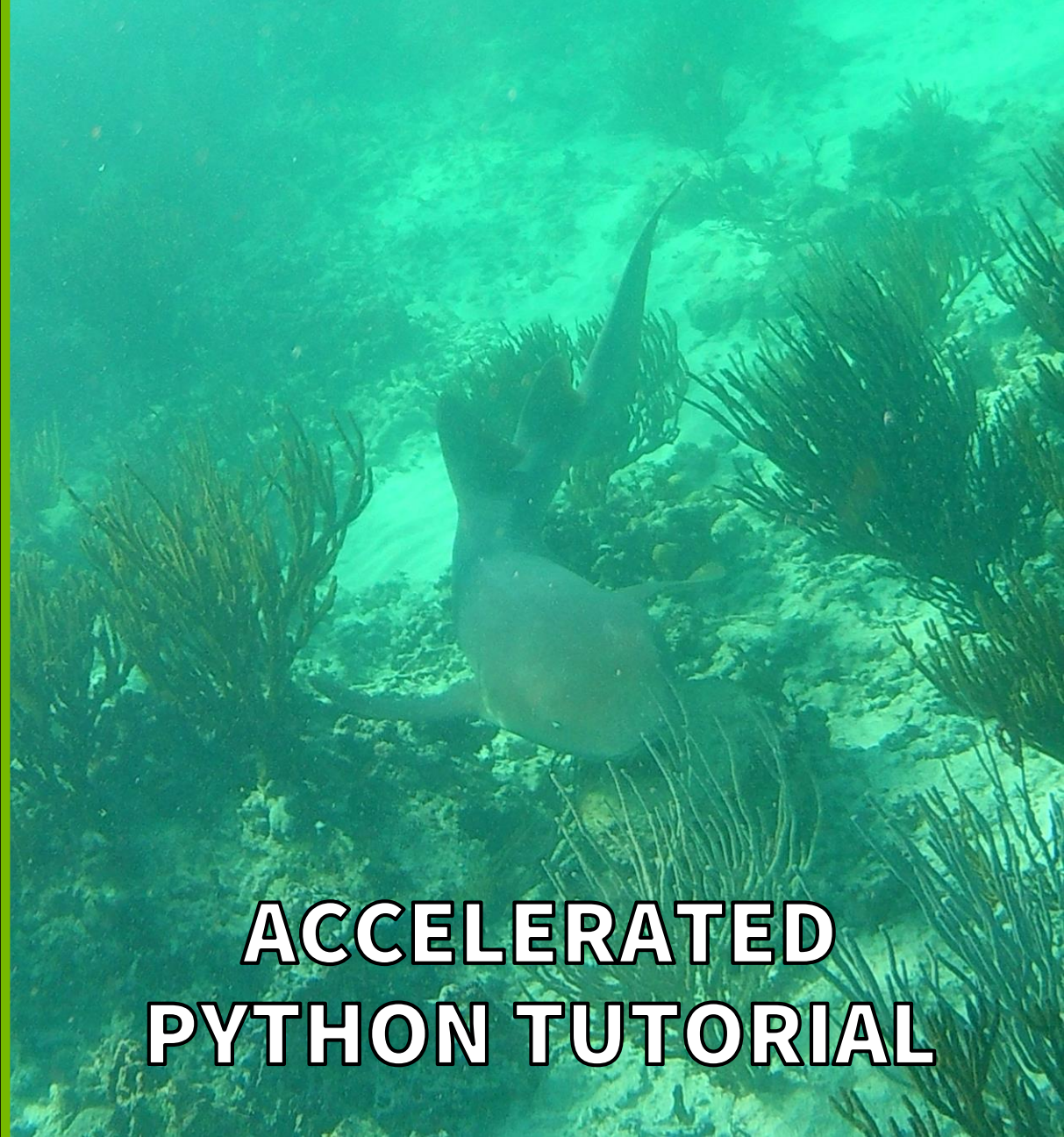
nvda.ws/48iJpkf

Open this doc where we can share info & ask questions:

nvda.ws/43u04ig

WIFI: SSIDSSIDSSIDSSID

PW: PASSWORDPASSWORD





Bryce Adelstein Lelbach
Principal Engineer @ NVIDIA
Vanguard Programming

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Bryce Adelstein Lelbach
Principal Engineer @ NVIDIA
Vanguard Programming



Andy Terrel
Product Manager @ NVIDIA
CUDA Python

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



Bryce Adelstein Lelbach
Principal Engineer @ NVIDIA
Vanguard Programming



Andy Terrel
Product Manager @ NVIDIA
CUDA Python



Stan Seibert
Senior Director @ Anaconda
OSS Compilers

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Bryce Adelstein Lelbach
Principal Engineer @ NVIDIA
Vanguard Programming



Andy Terrel
Product Manager @ NVIDIA
CUDA Python



Stan Seibert
Senior Director @ Anaconda
OSS Compilers



Matthew Cawood
Research Engineer @ TACC
HPC Perf and Archs Group

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



Bryce Adelstein Lelbach
Principal Engineer @ NVIDIA
Vanguard Programming



Andy Terrel
Product Manager @ NVIDIA
CUDA Python



Stan Seibert
Senior Director @ Anaconda
OSS Compilers



Matthew Cawood
Research Engineer @ TACC
HPC Perf and Archs Group



Sam Foreman
Computational Scientist @ ALCF
AI and HPC

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



Bryce Adelstein Lelbach
Principal Engineer @ NVIDIA
Vanguard Programming



Andy Terrel
Product Manager @ NVIDIA
CUDA Python



Stan Seibert
Senior Director @ Anaconda
OSS Compilers



Matthew Cawood
Research Engineer @ TACC
HPC Perf and Archs



Sam Foreman
Computational Scientist @ ALCF
AI and HPC



Daniel Margala
Software Engineer @ NERSC
Prog Environments and Models

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

All content is open source and available on GitHub:

github.com/NVIDIA/accelerated-computing-hub

Directory: tutorials/accelerated-python

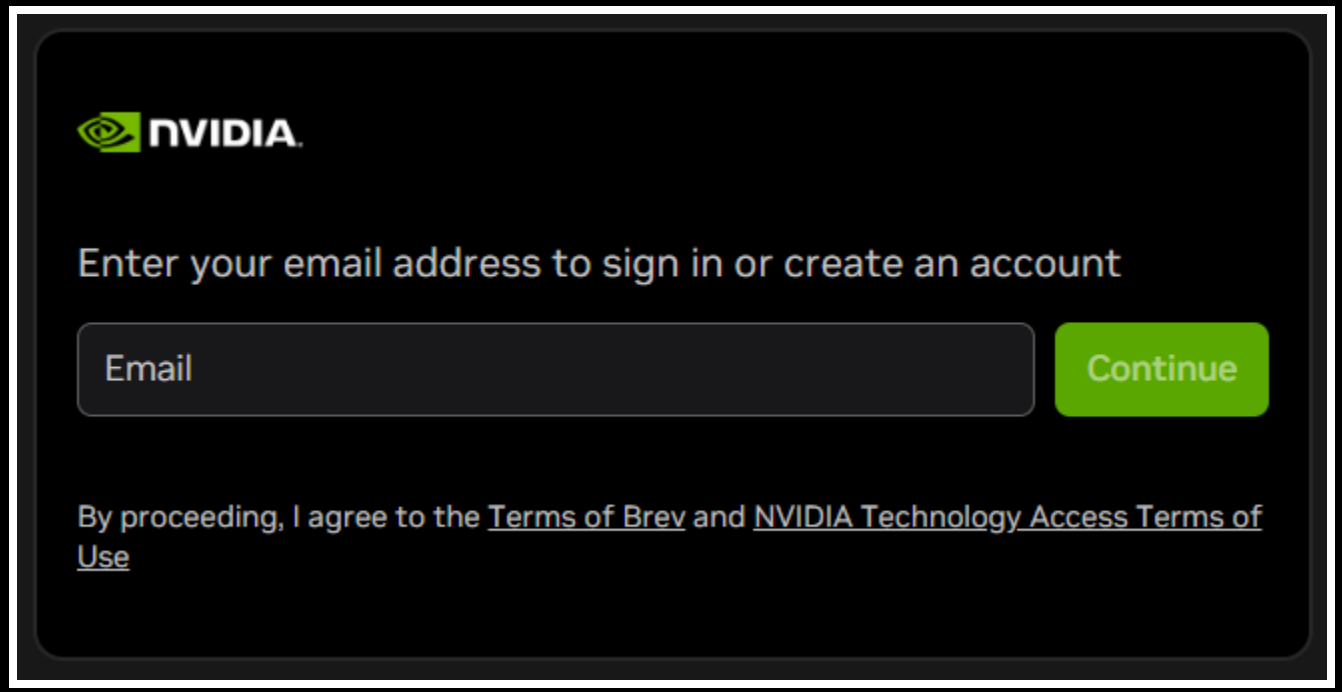
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

This course is web-based; no local installation needed.

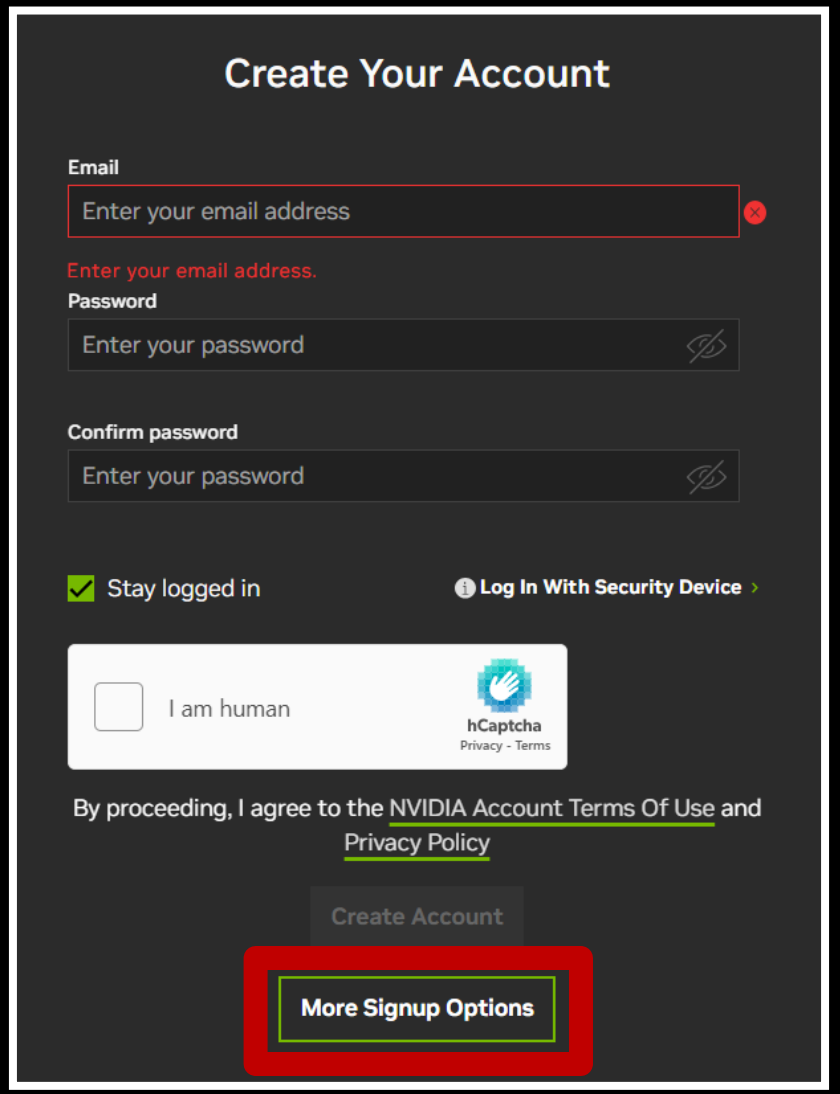
Recommended browser: Chrome (28+).

Javascript and WebRTC must be enabled.

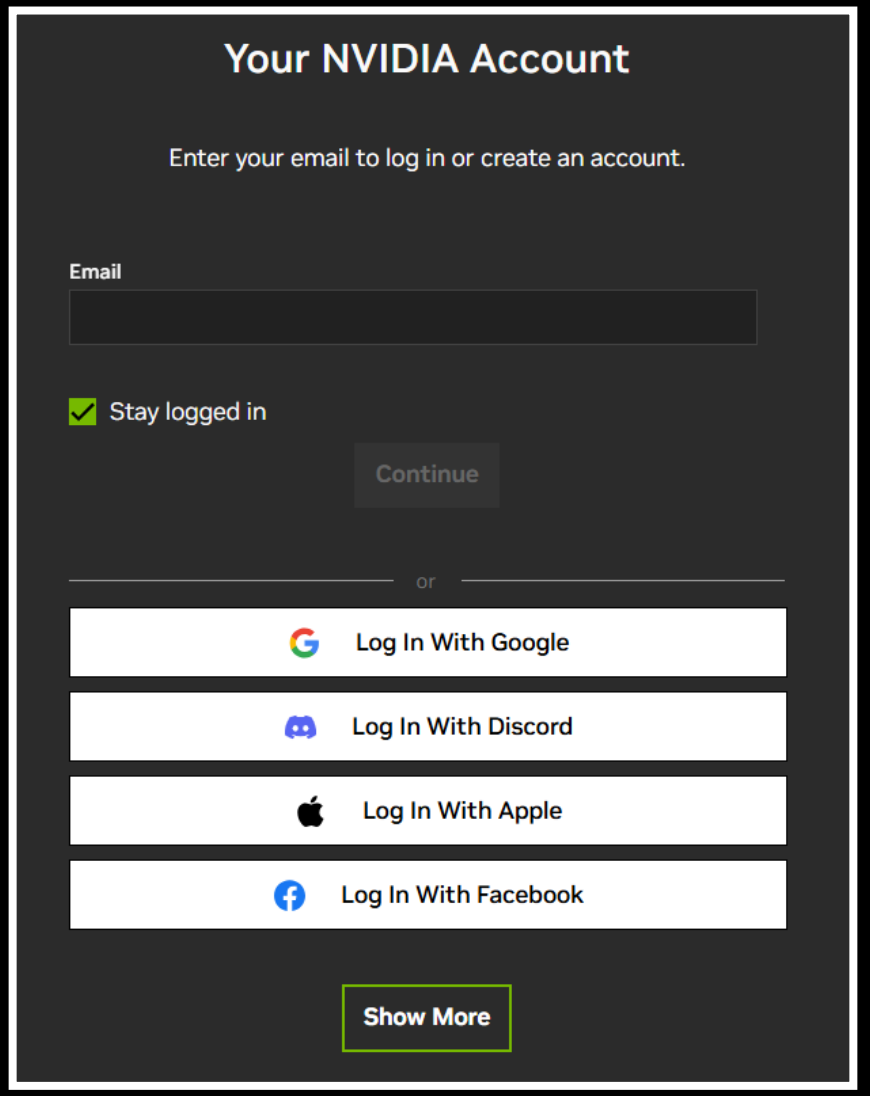
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



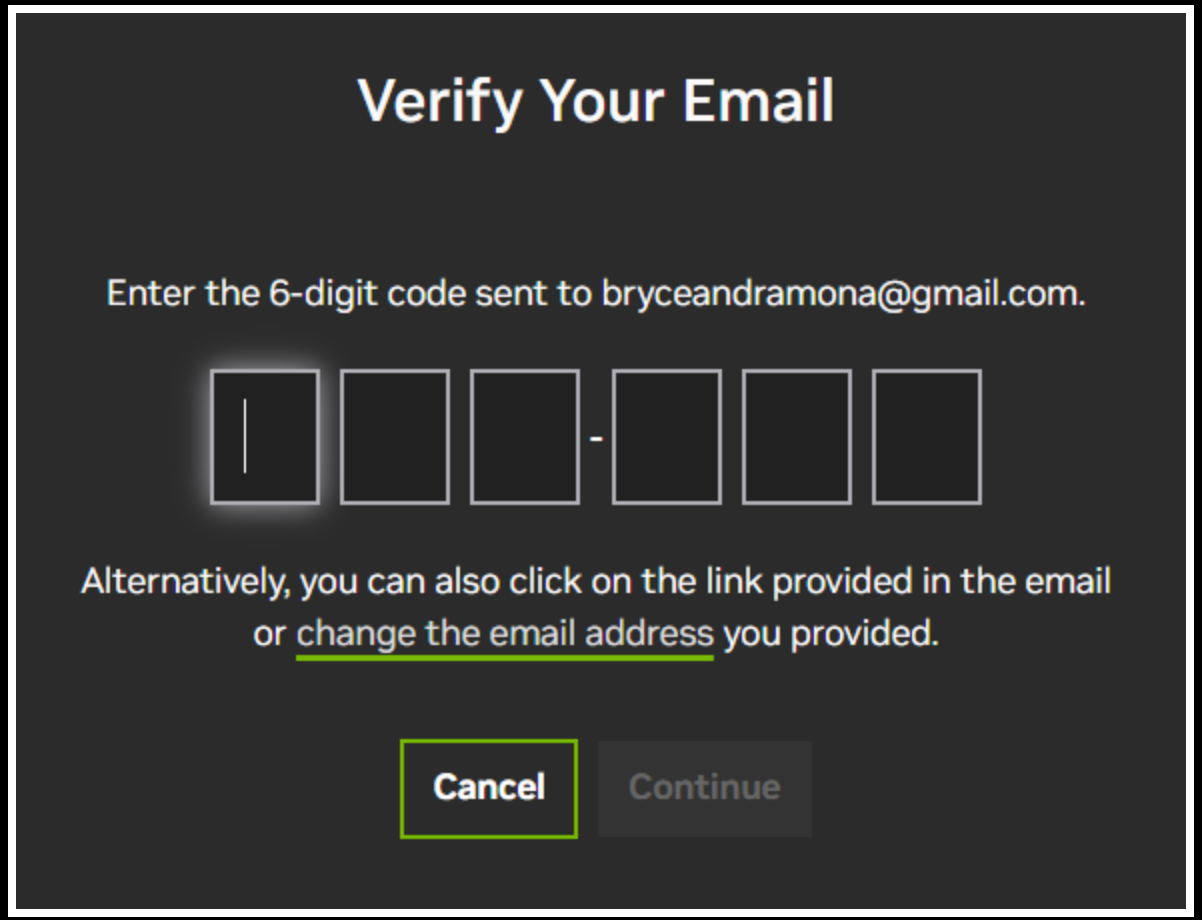
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



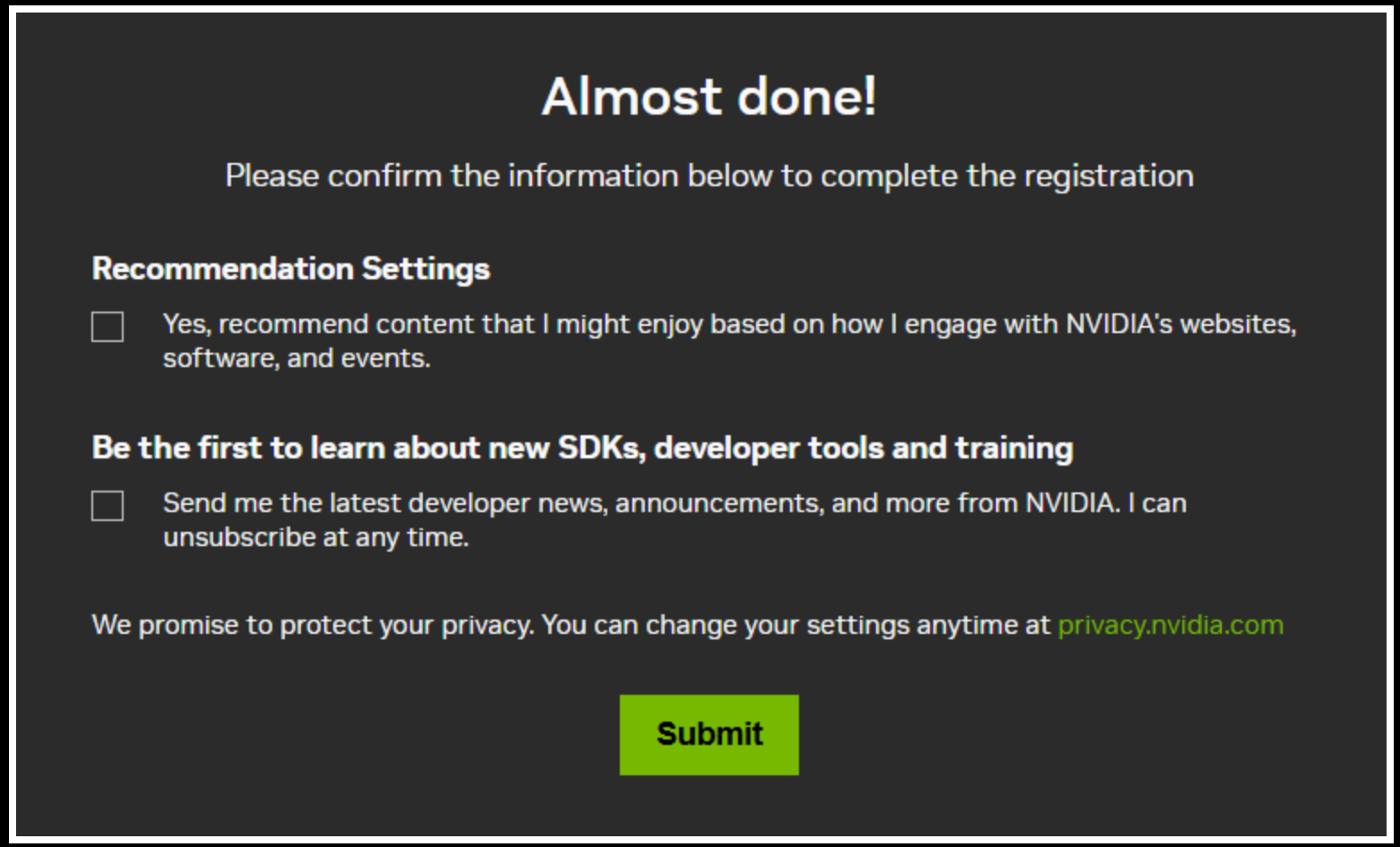
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



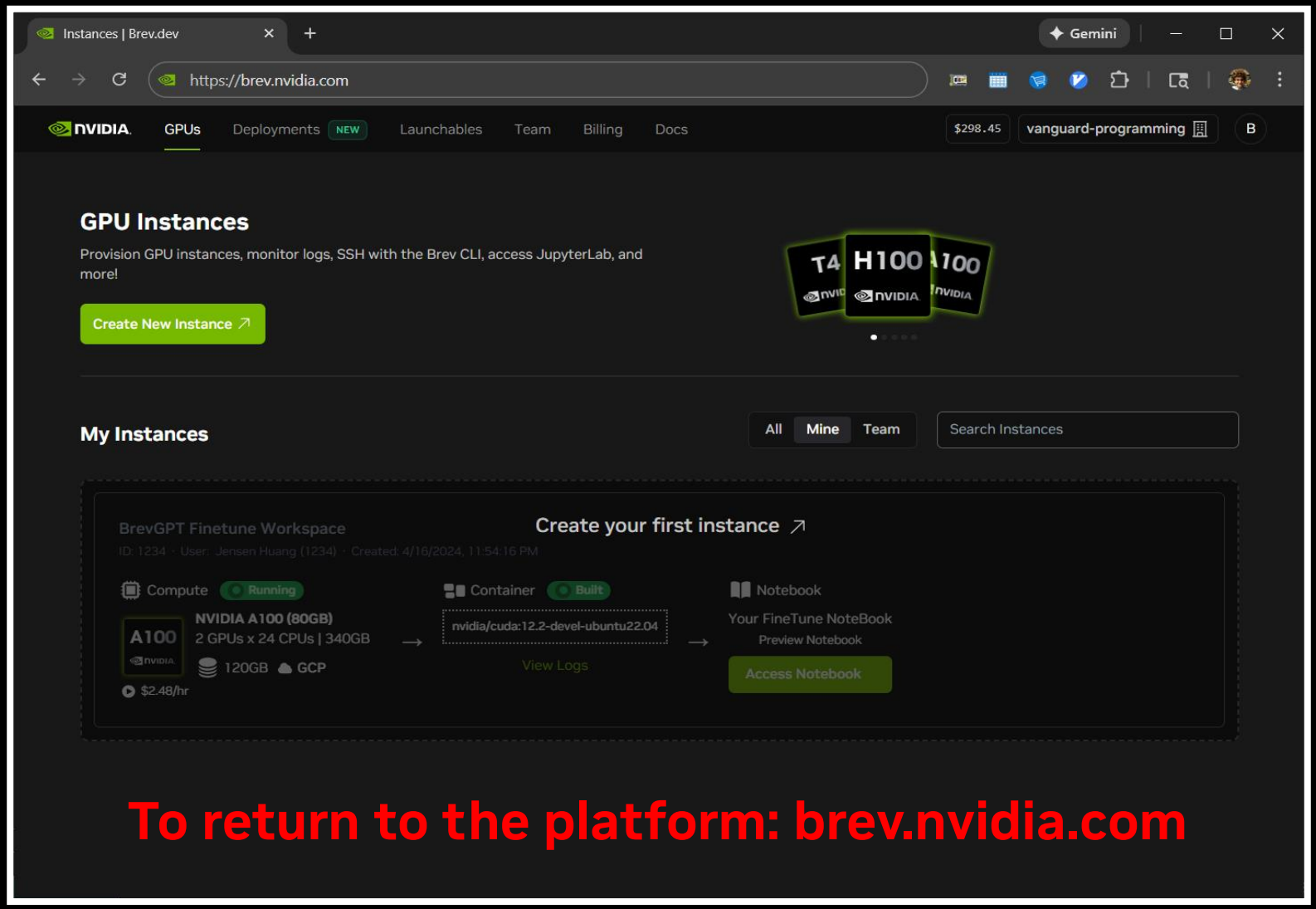
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Please enter your first & last name here!



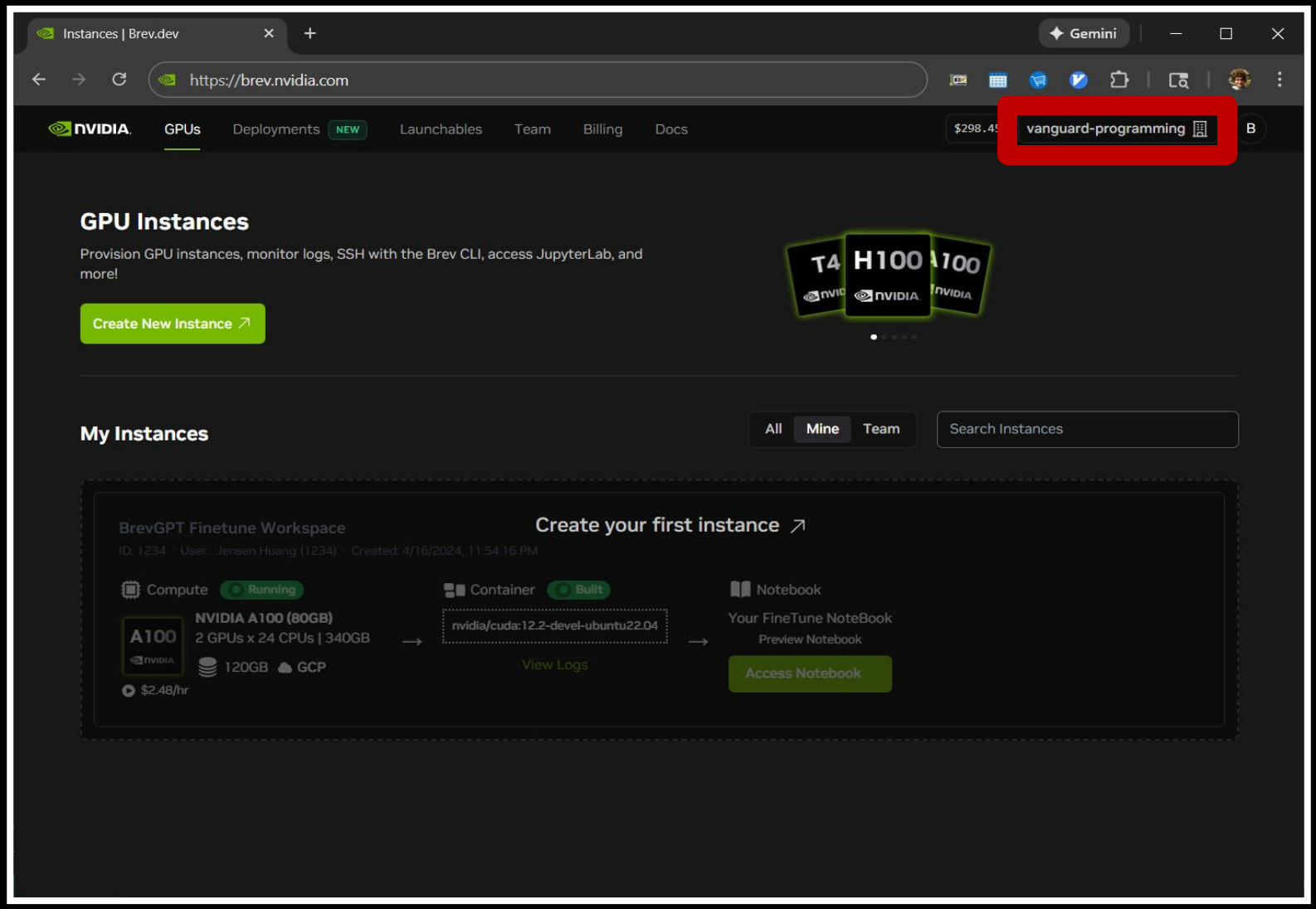
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



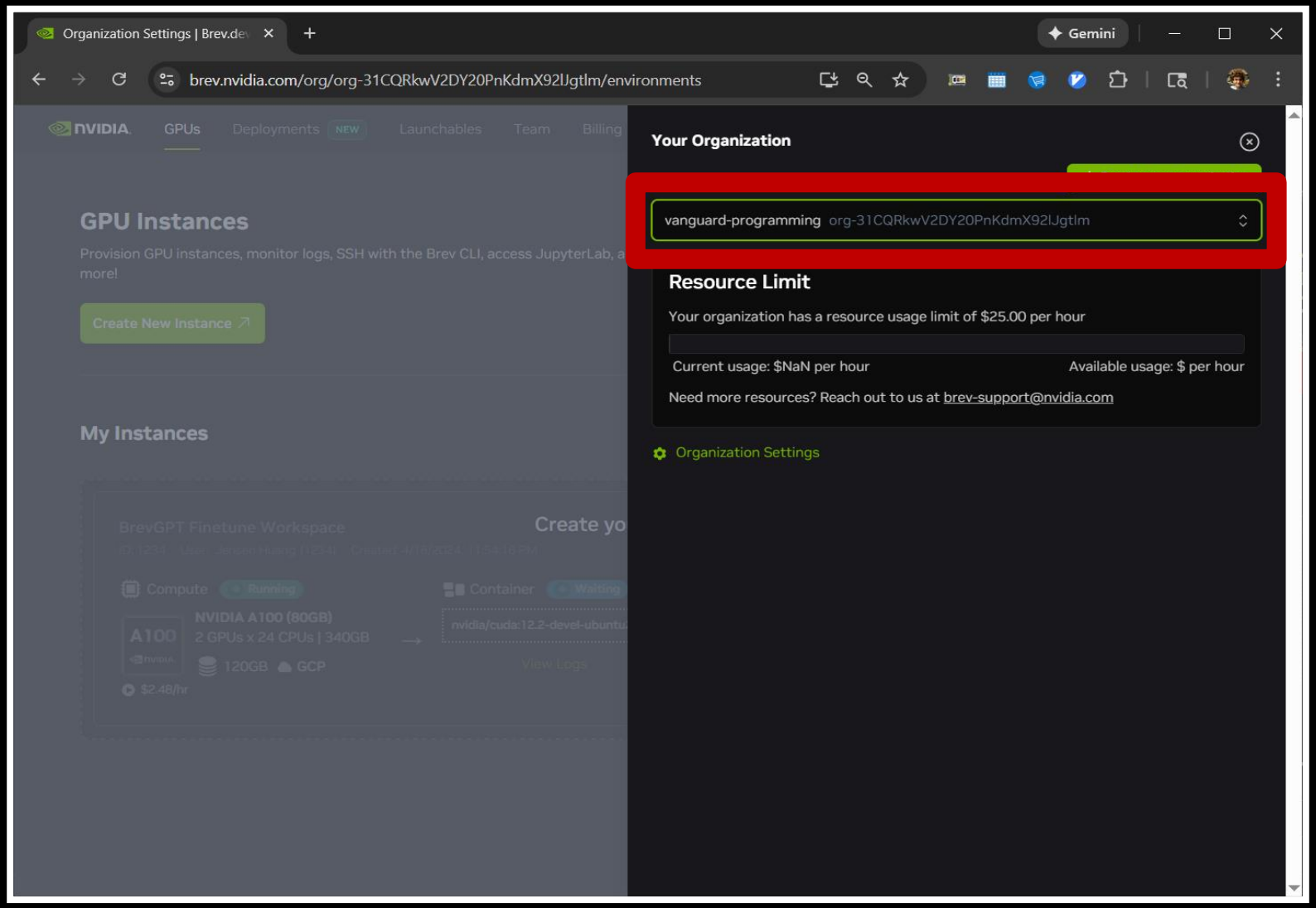
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



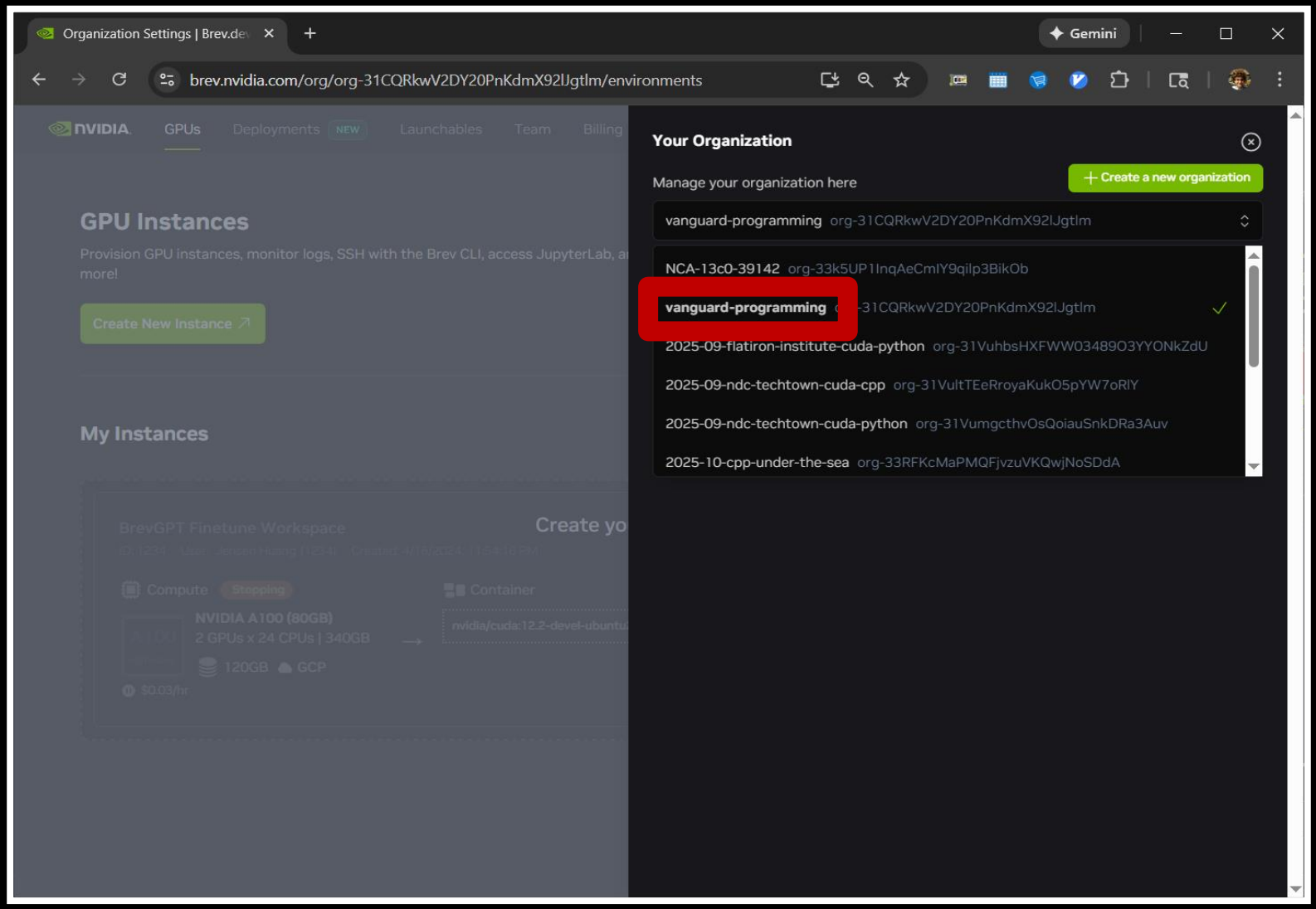
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



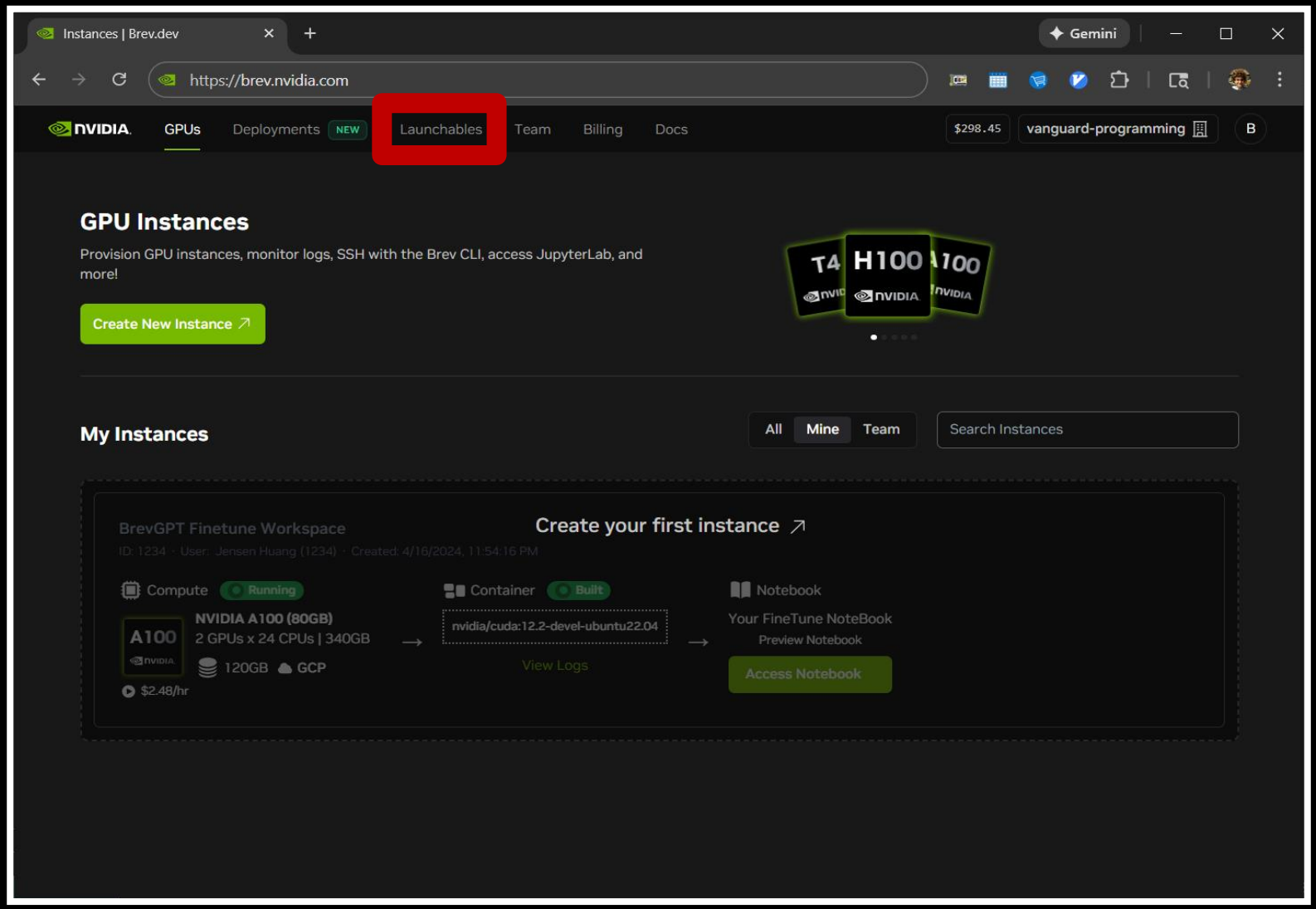
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



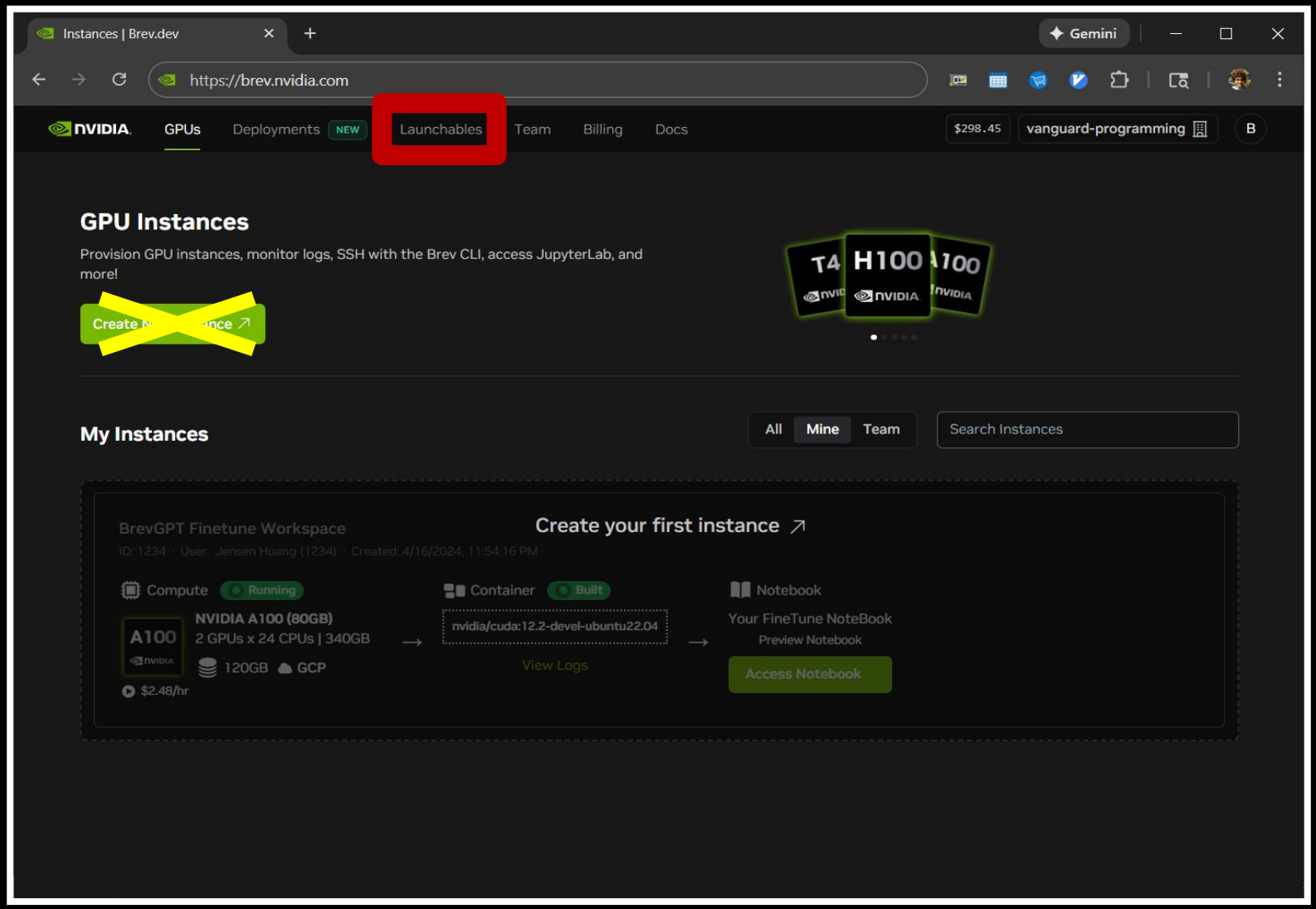
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



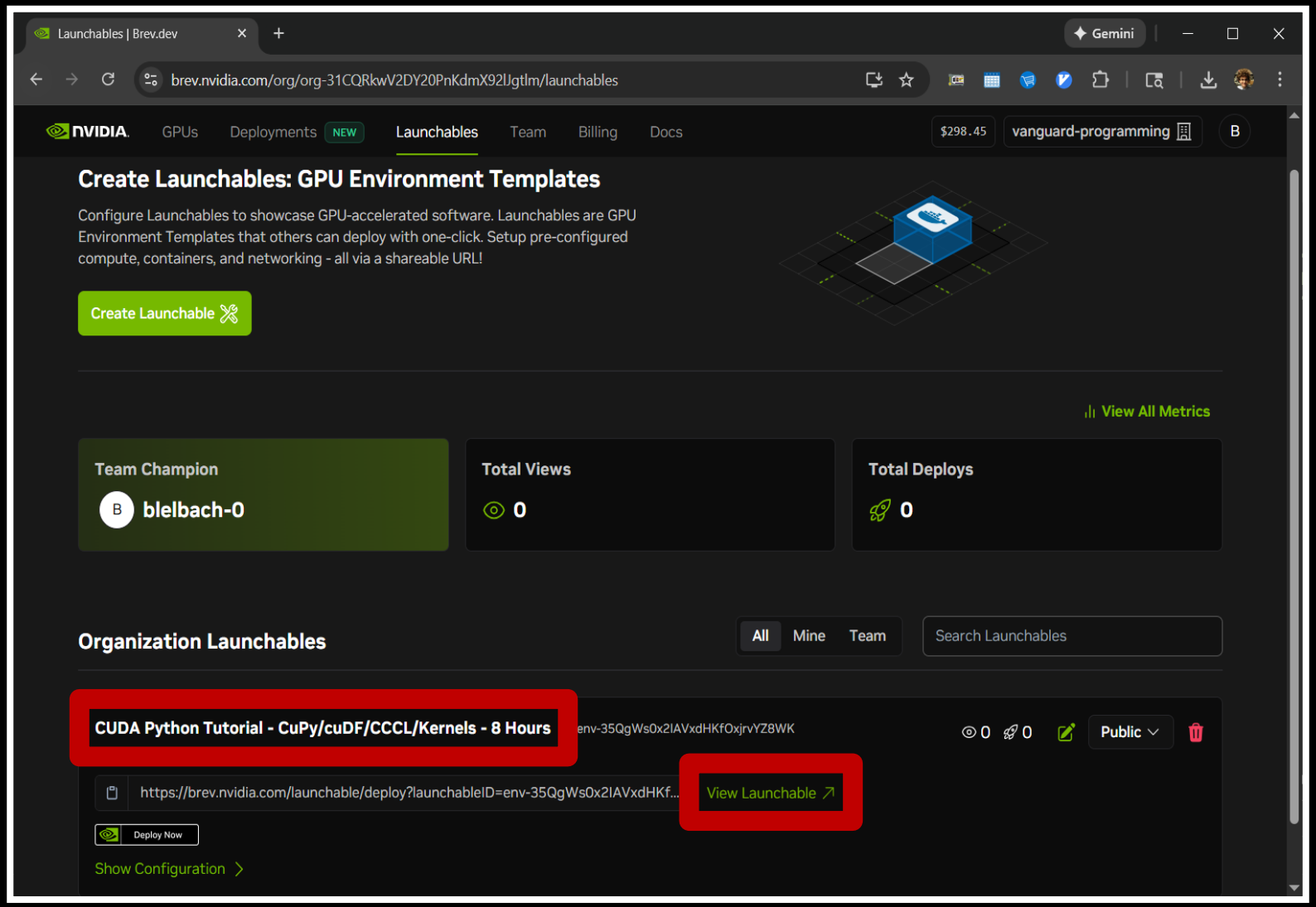
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD

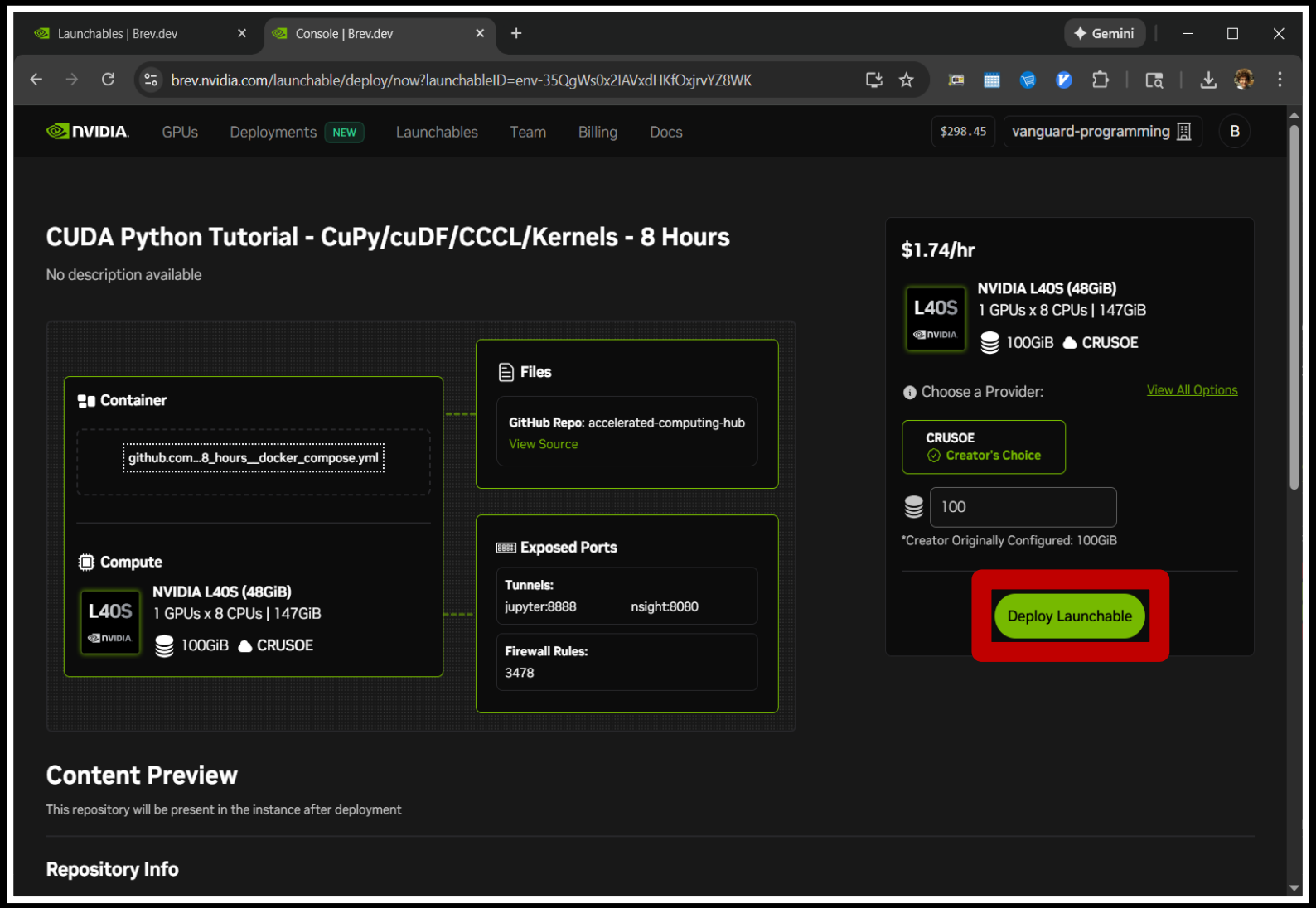


Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD

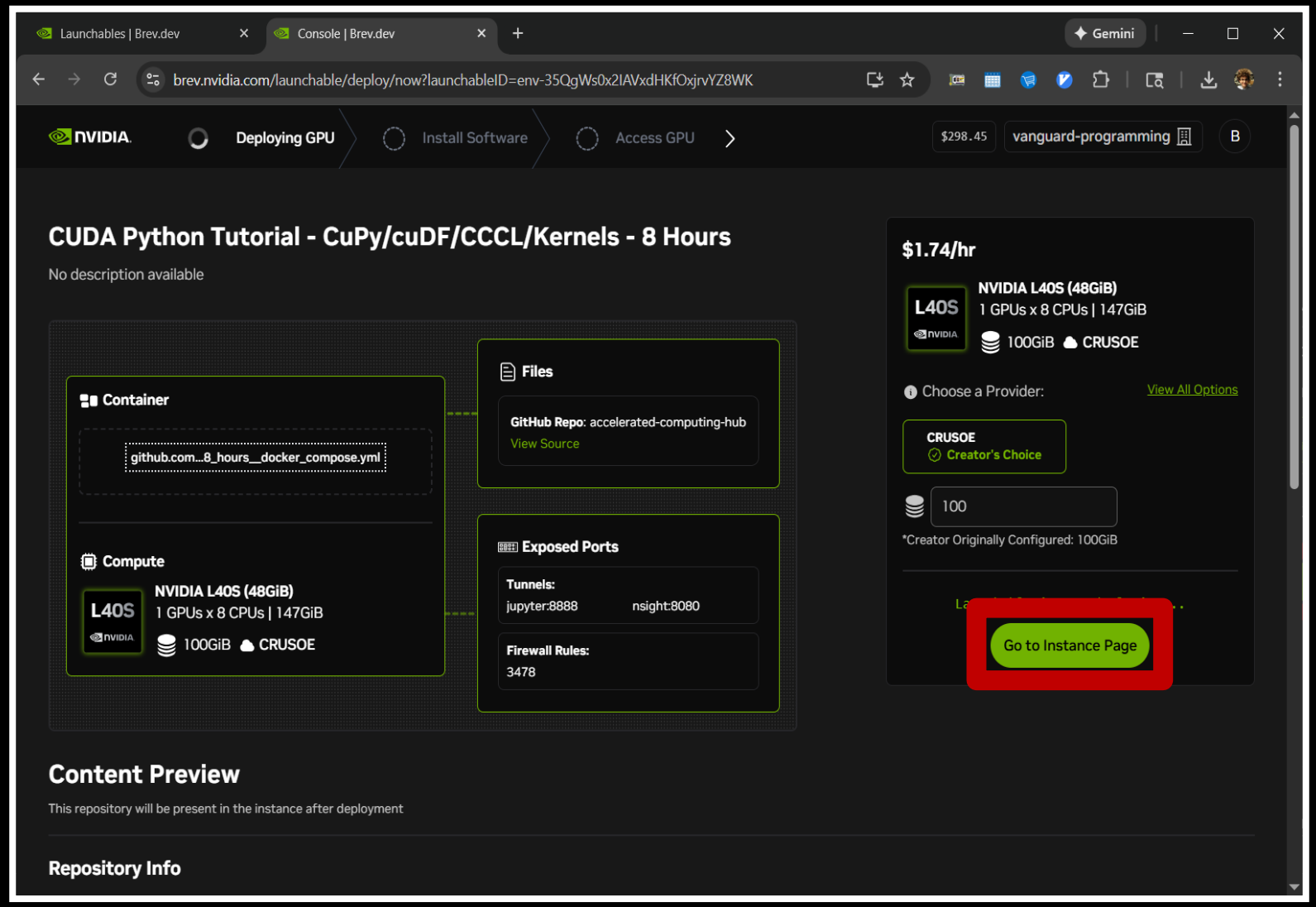


Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



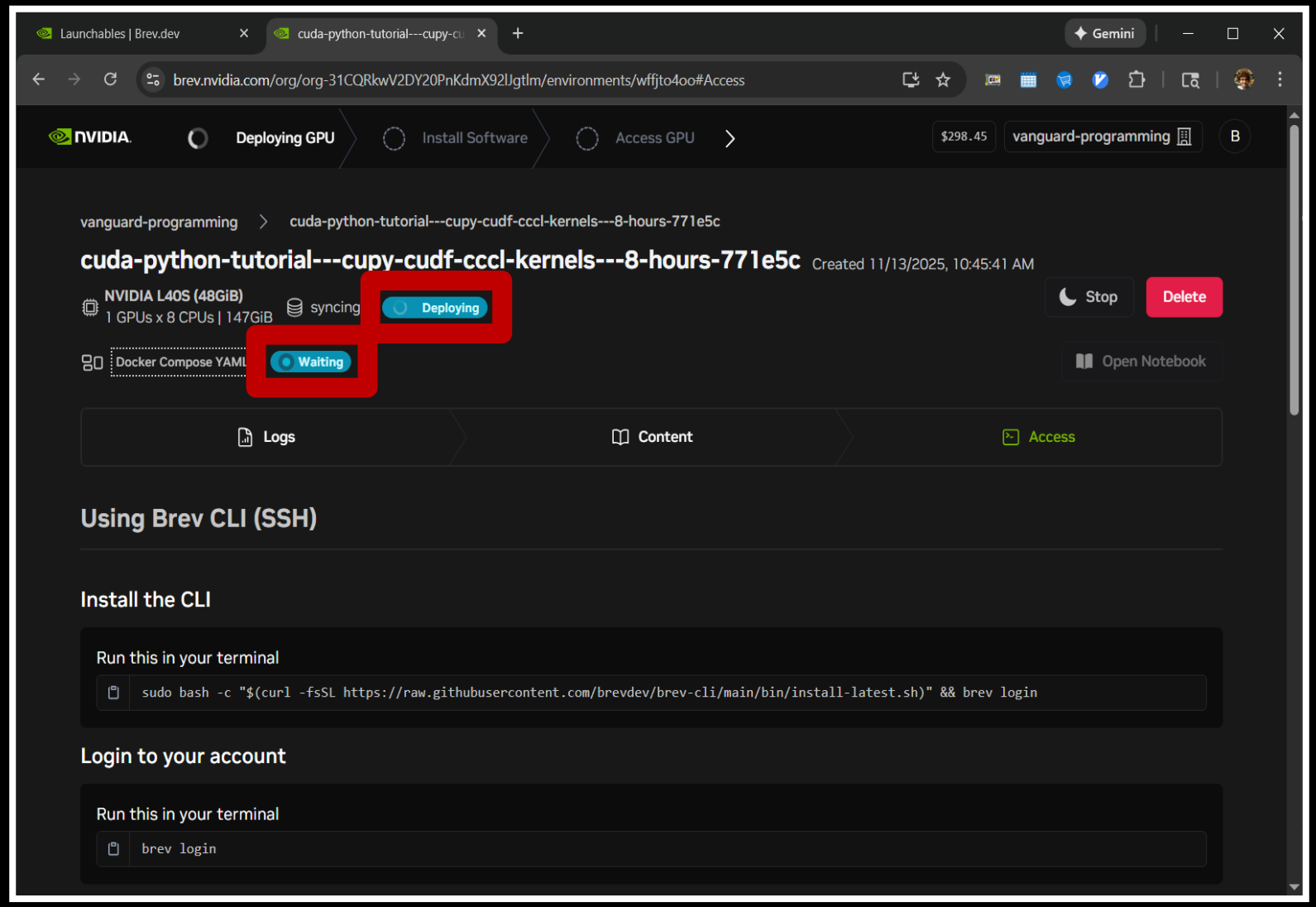
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



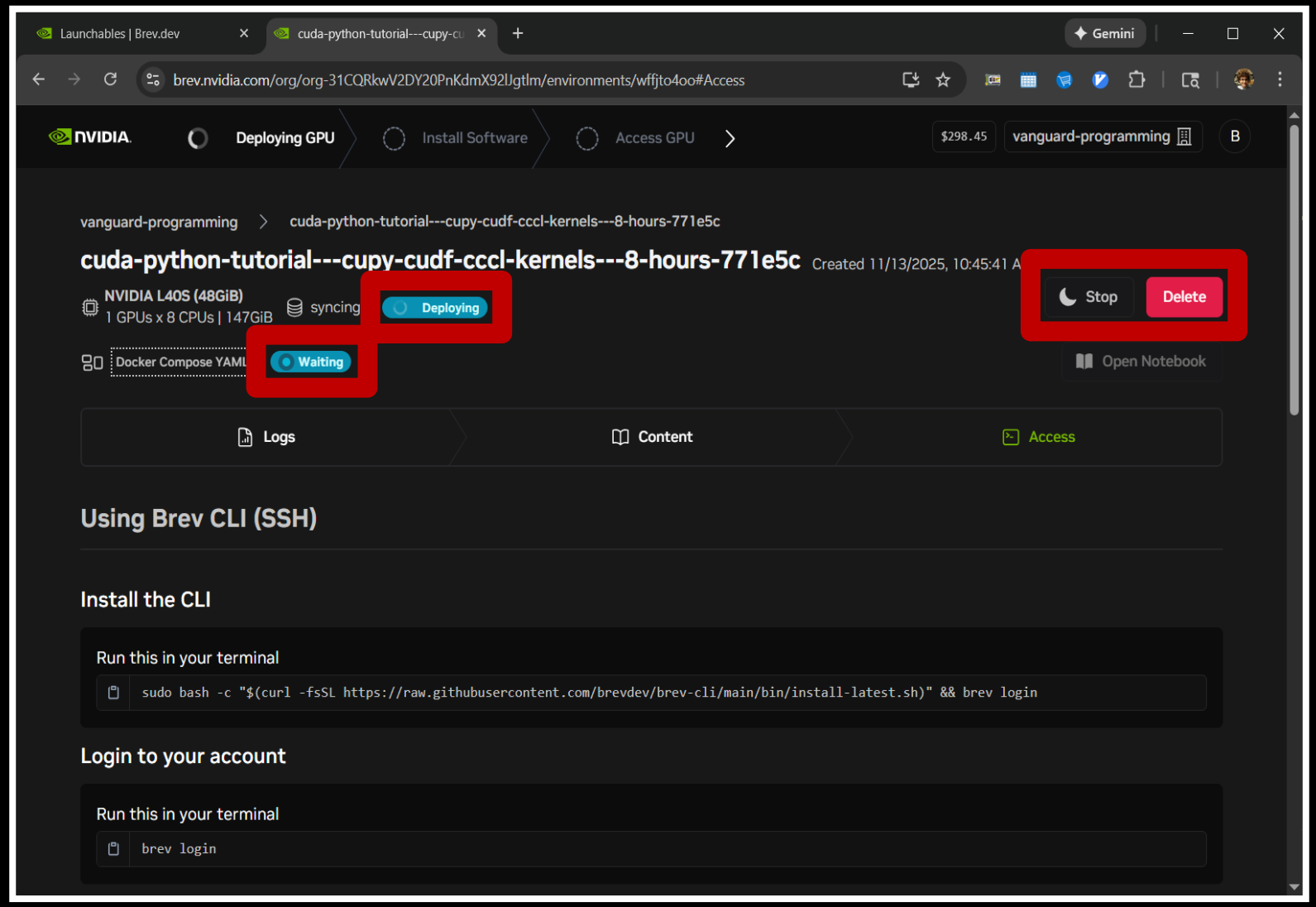
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



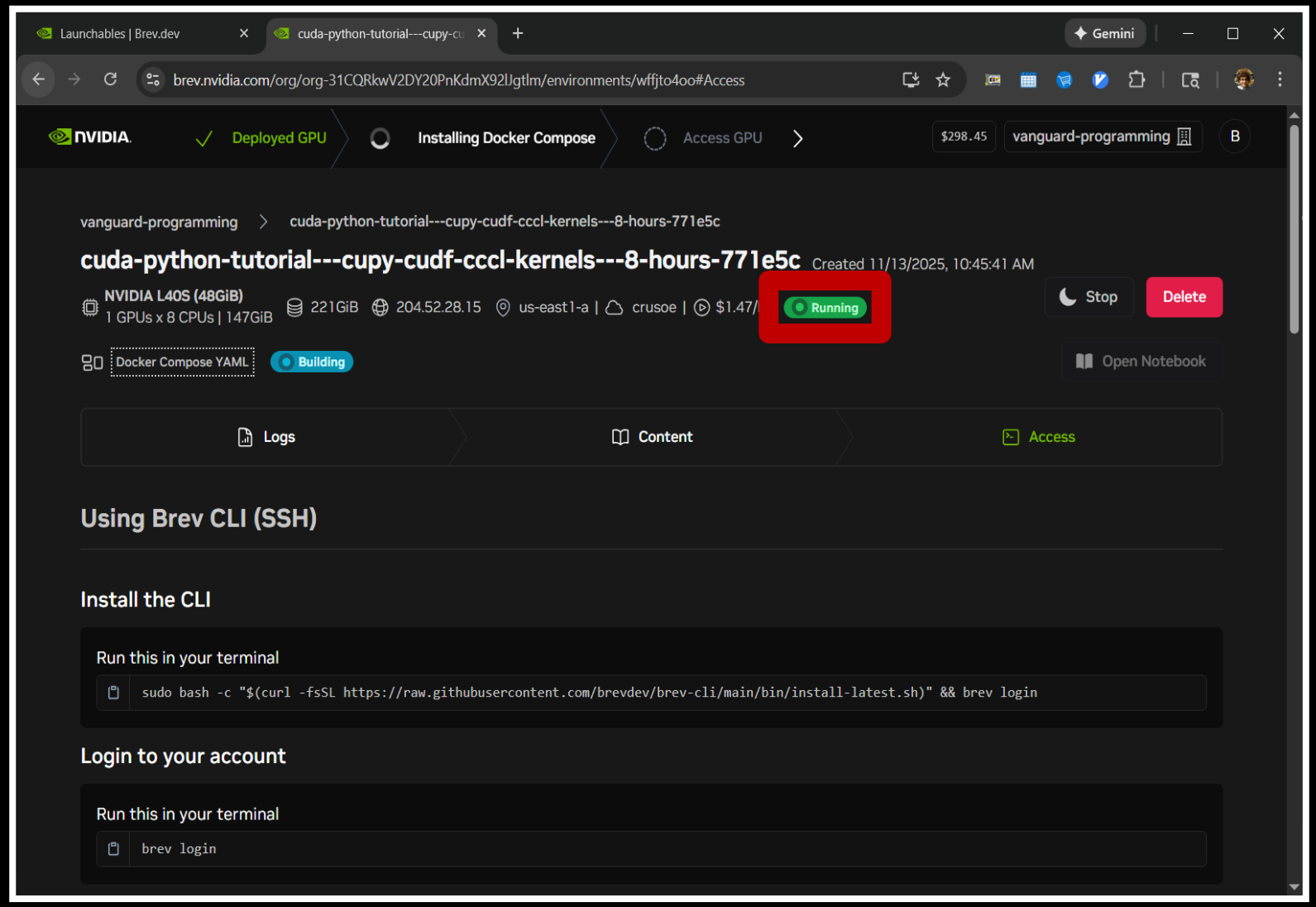
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



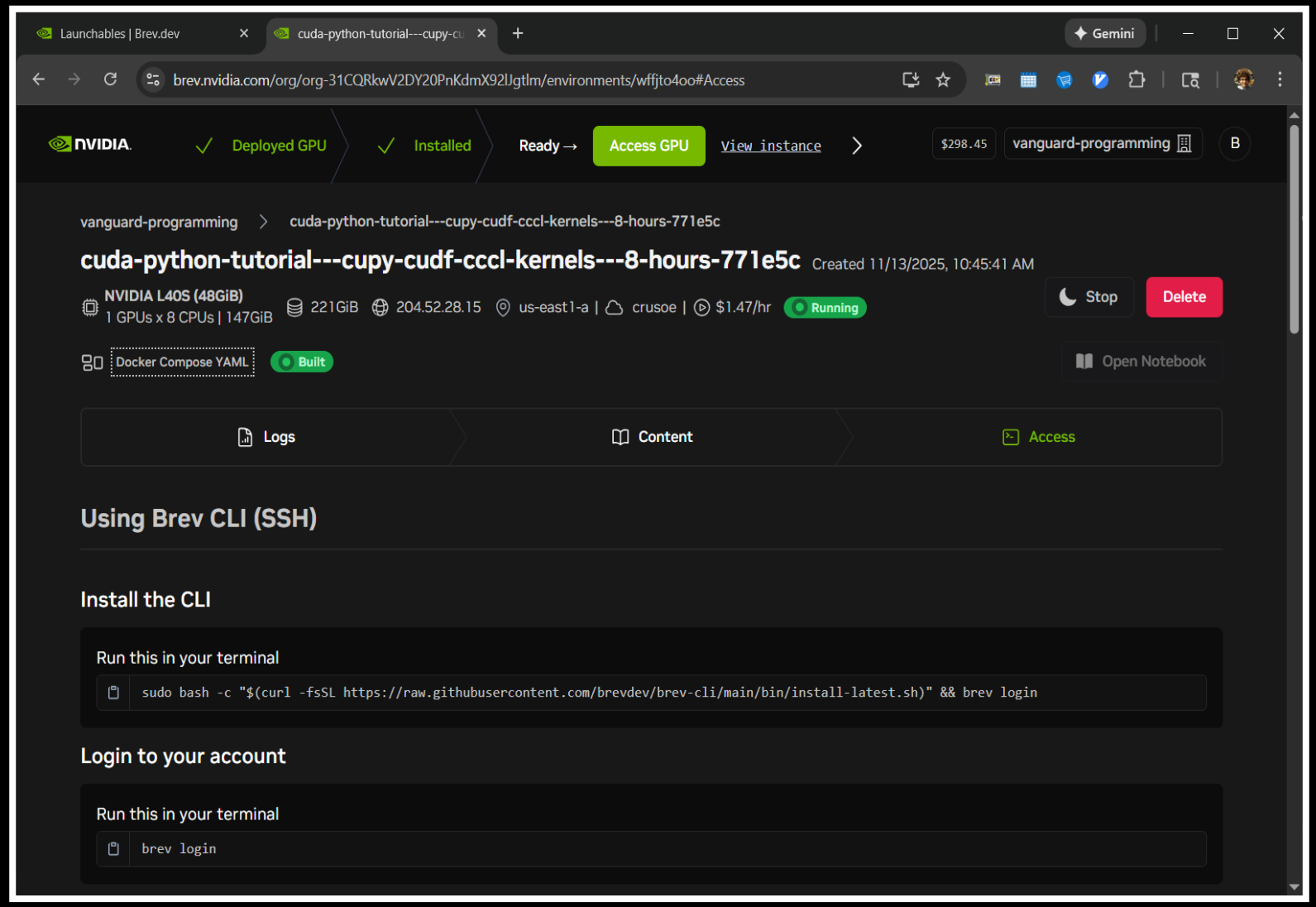
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD



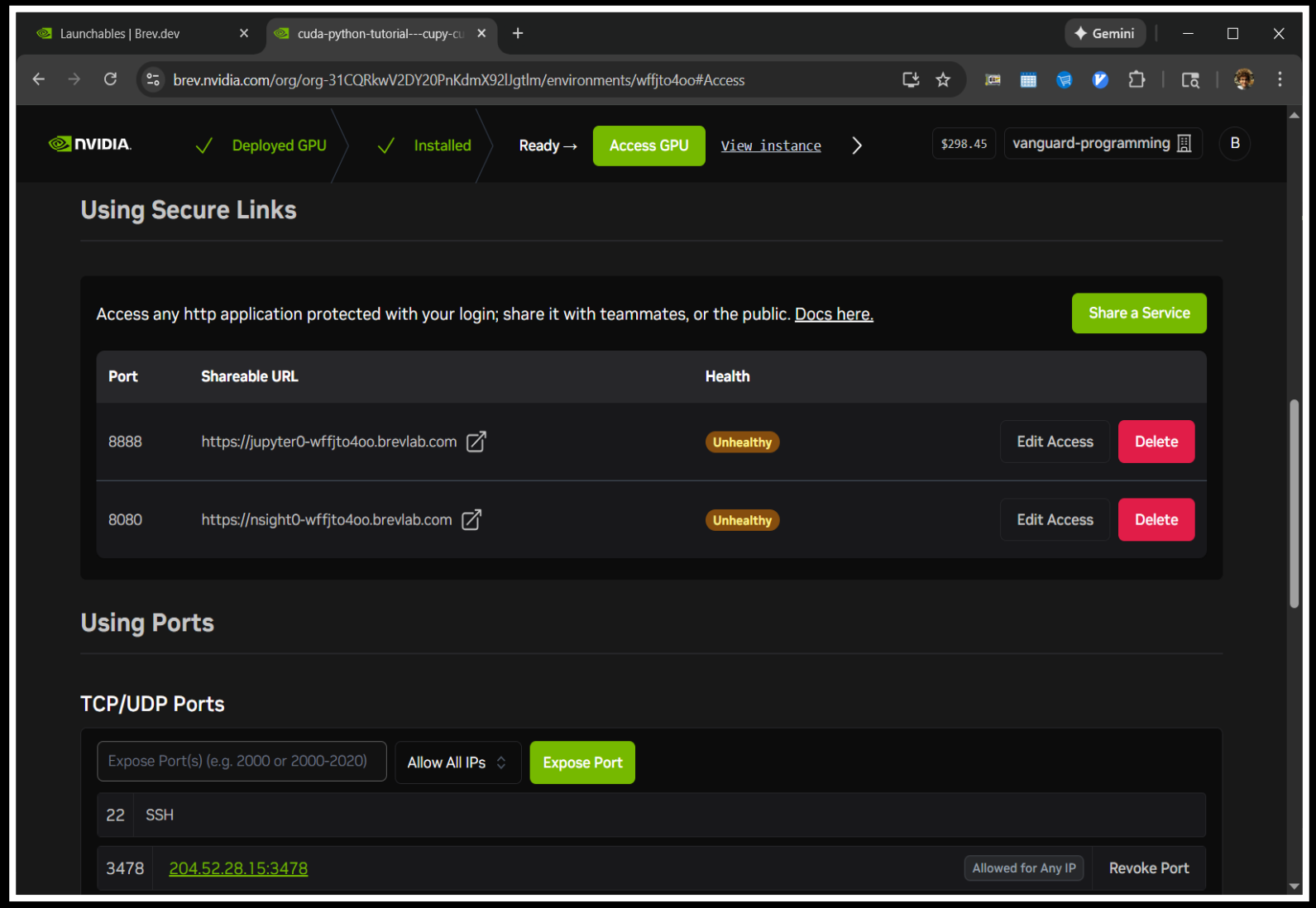
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD

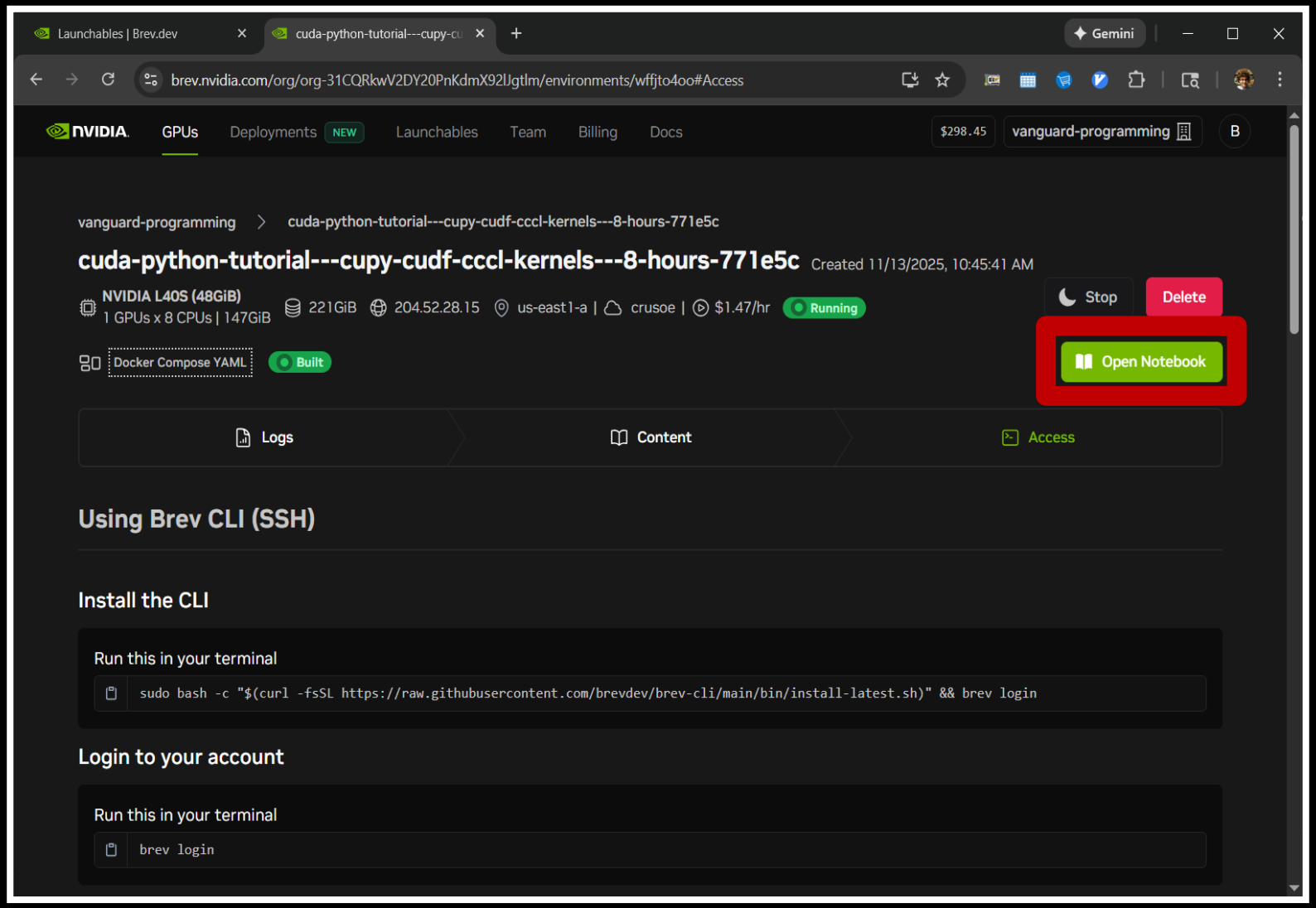


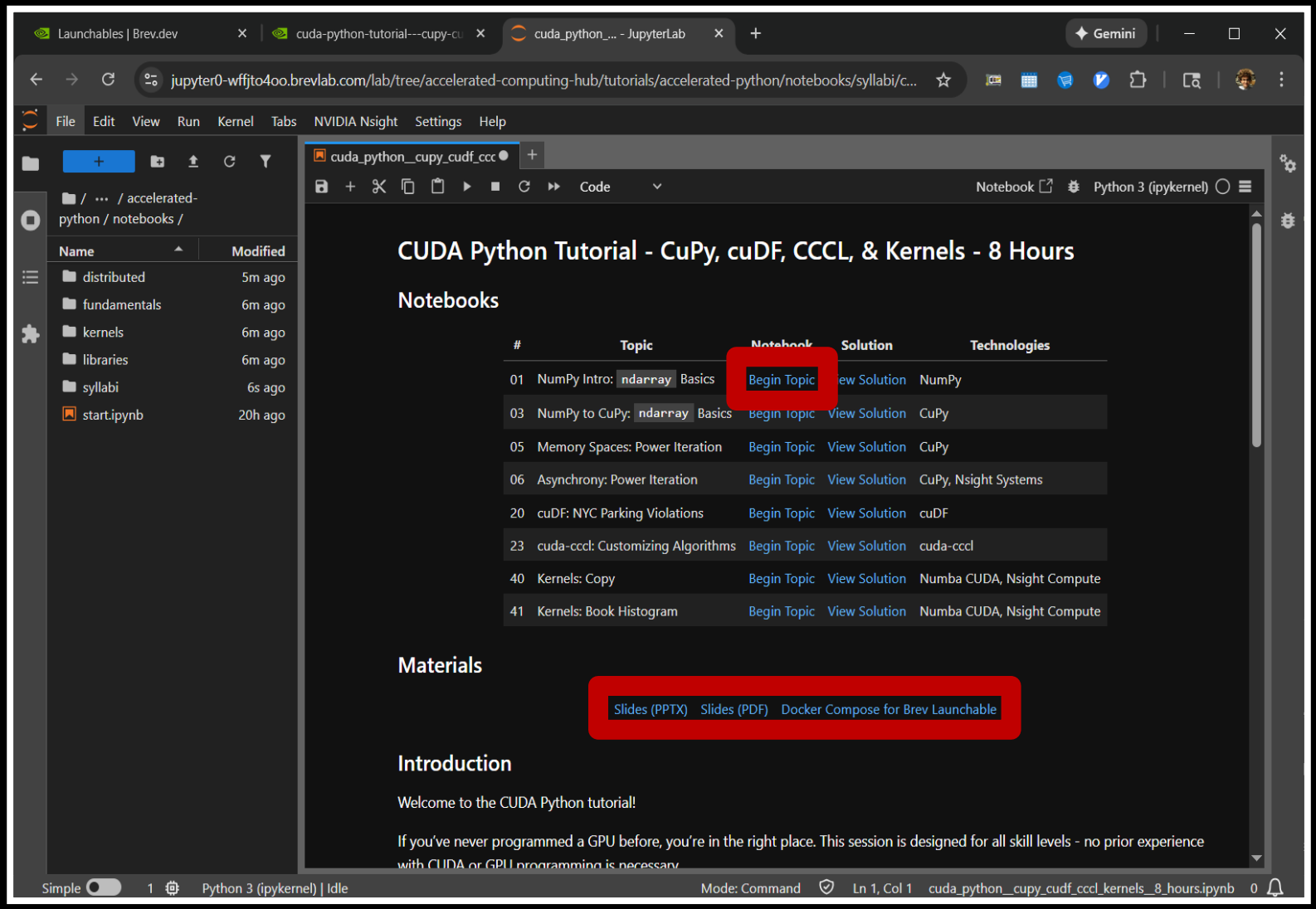
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Brev: nvda.ws/48iJpkf Q&A: nvda.ws/43u04ig WIFI: SSIDSSIDSSID PW: PASSWORD

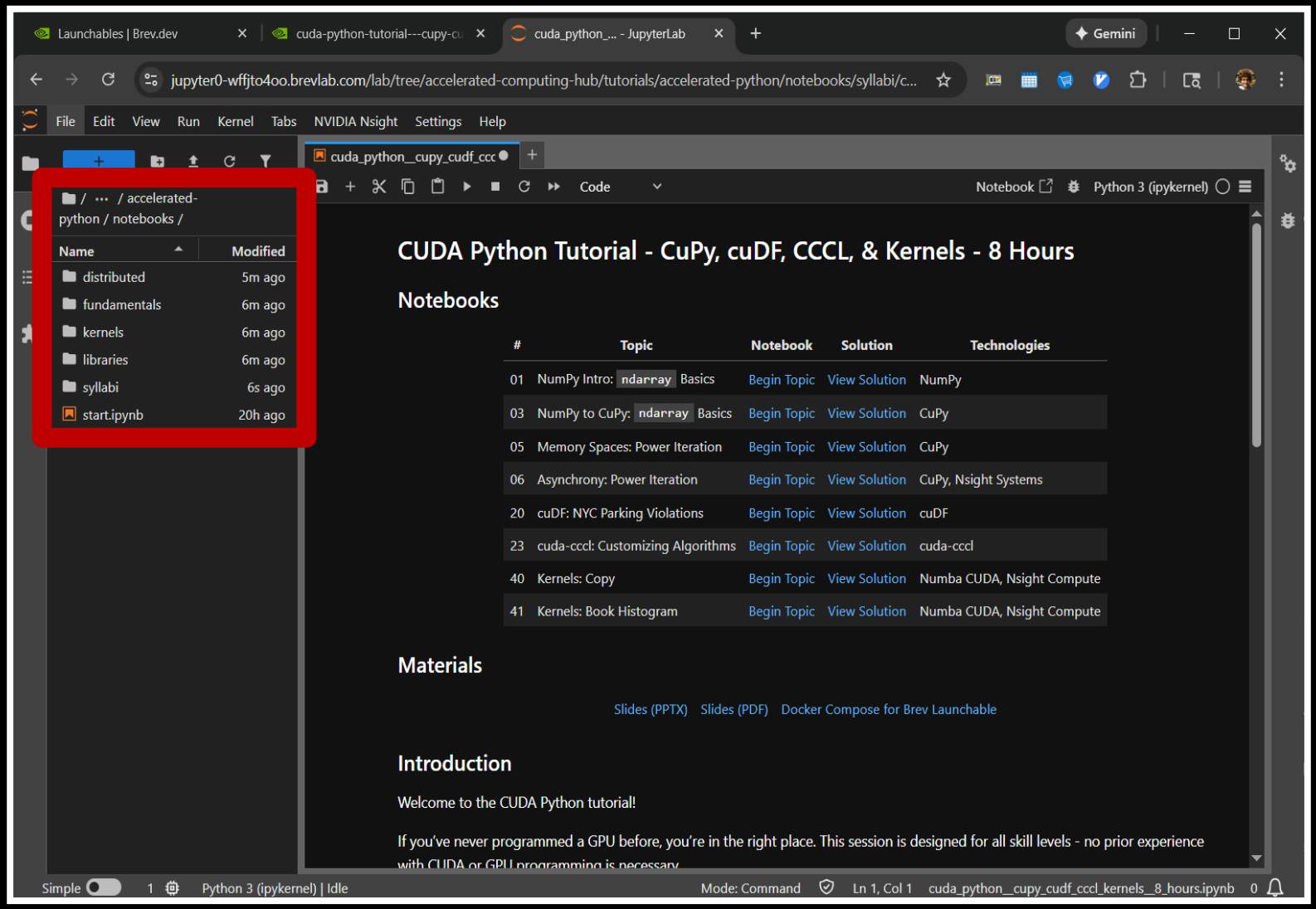


Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig


Launchables | Brev.dev
cuda-python-tutorial---cupy-cu
Gemini
brev.nvidia.com/org/org-31CQRkwV2DY20PnKdmX92lJgtlm/environments/wffjto4oo#Access
NVIDIA
GPUs
Deployments
NEW
Launchables
Team
Billing
Docs
$298.45
vanguard-programming
B
vanguard-programming
cuda-python-tutorial---cupy-cudf-cccl-kernels---8-hours-771e5c
cuda-python-tutorial---cupy-cudf-cccl-kernels---8-hours-771e5c
Created 11/13/2025, 10:45:41 AM
NVIDIA L40S (48GiB)
1 GPUs x 8 CPUs | 147GiB
221GiB
204.52.28.15
us-east1-a | crusoe | $1.47/hr
Running
Stop
Delete
Docker Compose YAML
Built
Open Notebook
Logs
Content
Access
Using Brev CLI (SSH)
Install the CLI
Run this in your terminal
sudo bash -c "$(curl -fsSL https://raw.githubusercontent.com/brevdev/brev-cli/main/bin/install-latest.sh)" && brev login
Login to your account
Run this in your terminal
brev login



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



NumPy: Array Programming

```
import numpy as np

# Riemann sum of ∫₀¹x²dx
x = np.linspace(0, 1, 100)
y = x**2
dx = x[1] - x[0]
I = np.sum(y) * dx
```

The de facto standard for array data in Python.

Every HPC and ML library references NumPy in some way!

NumPy specifies:

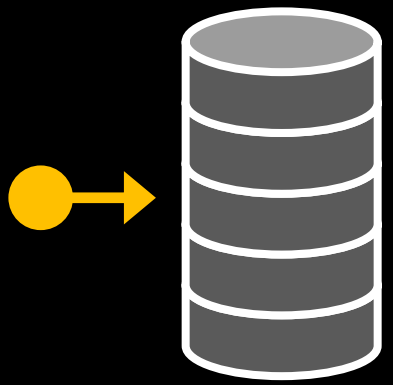
- `ndarray`: Multidimensional array type.
- Array indexing and slicing.
- A common set of array functions.
- How to define your own array functions.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

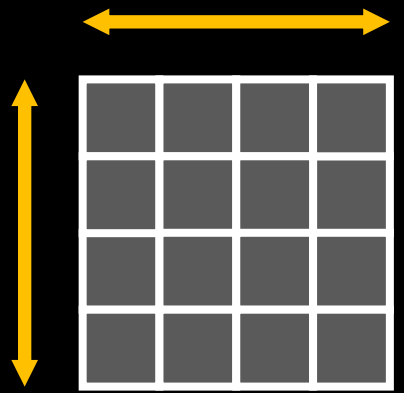
ndarray



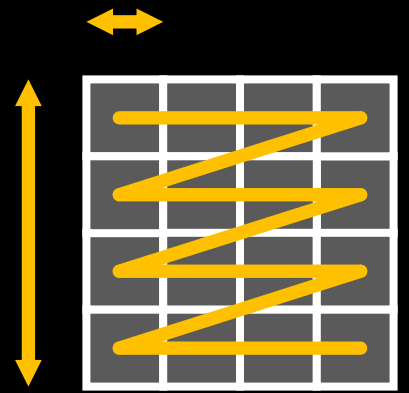
dtype: the data type of all elements of this array.
Ex: `int32`



data: a pointer to the block of memory holding all the elements in this array.

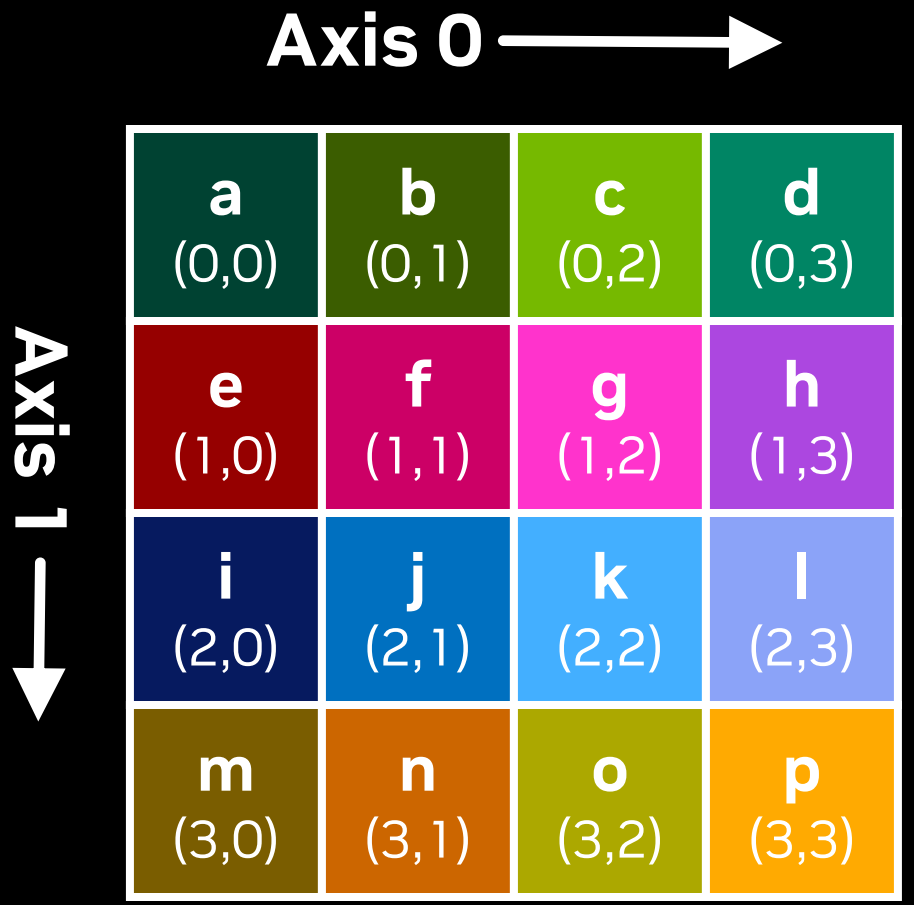


shape: the number of dimensions and the size of each dimension.
Ex: `(4, 3)`



strides: the number of bytes between elements along each dimension.
Ex: `(12, 4)`

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

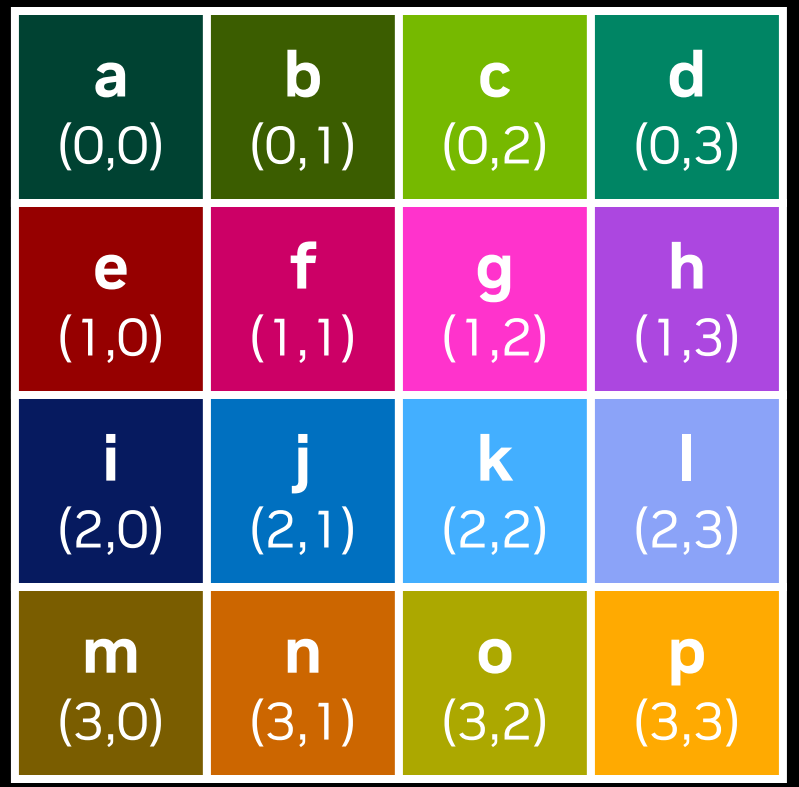


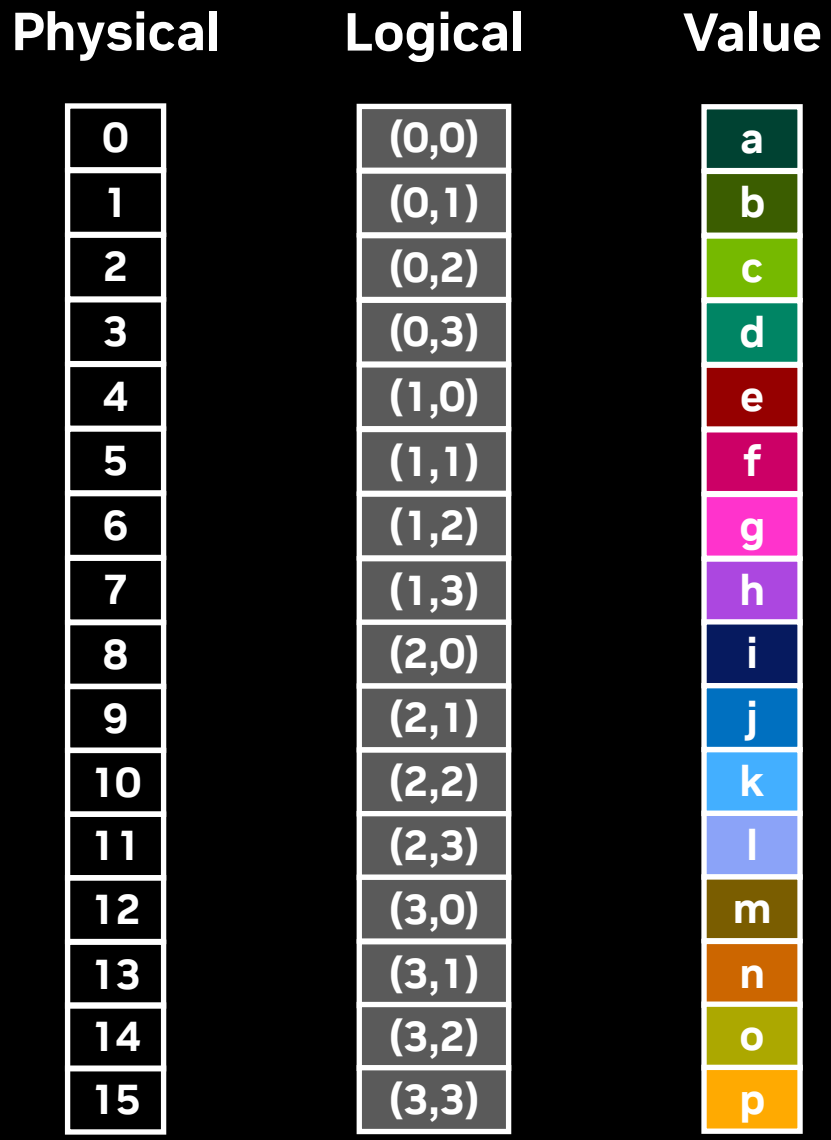
Each axis of the array can have whatever meaning makes sense for your situation:

- 1D - temperature data: [time step]
- 2D - greyscale image: [row, col]
- 3D - color image: [row, col, rgb]
- 4D - video: [time, row, col, rgb]
- 4D - image stack: [img#, row, col, rgb]

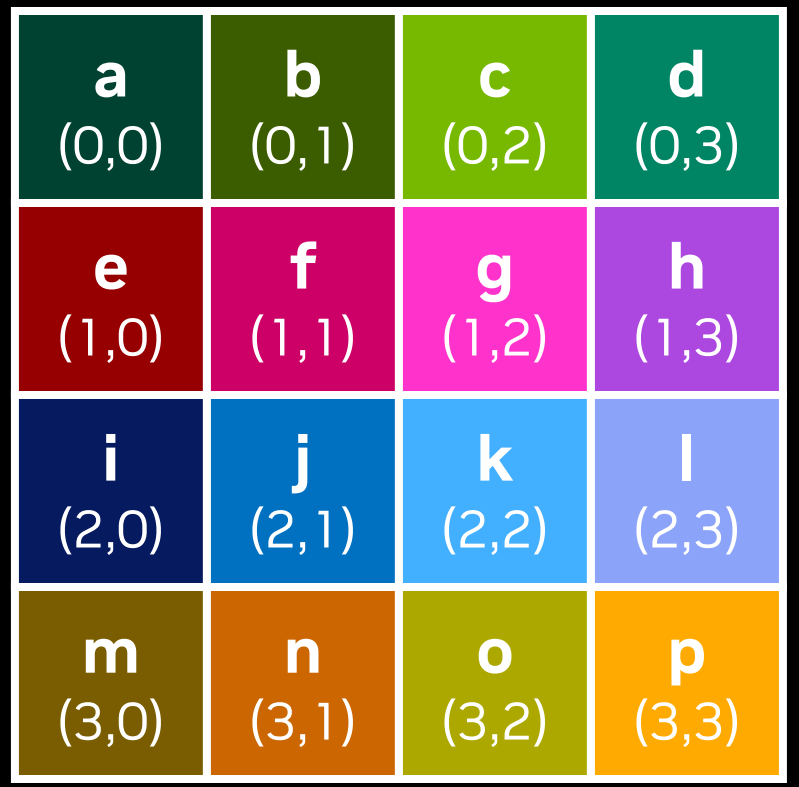
Axis ordering and memory layout are independent!

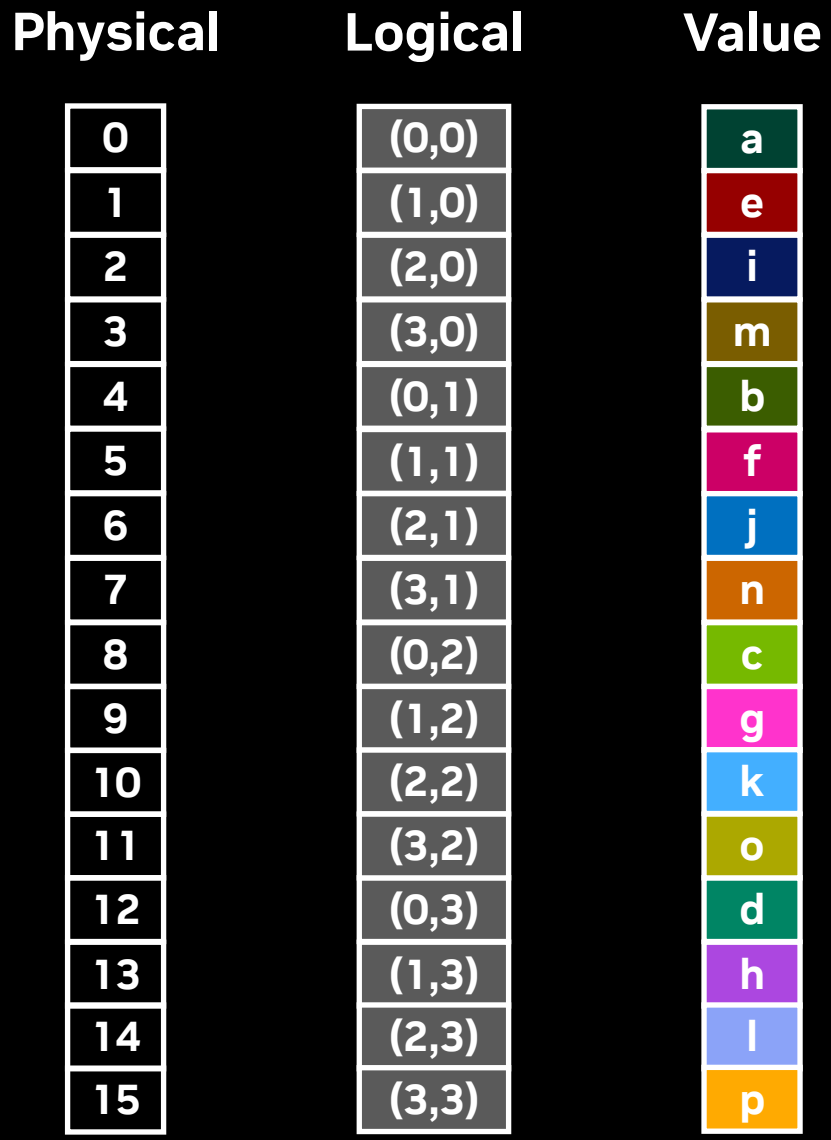
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig





Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig





Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Storage

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

```
X = np.ndarray((4,4))
```

a	b	c	d
e	f	g	h
i	j	k	l
m	n	o	p

```
np.transpose(X)
X.transpose()
X.T
```

a	e	i	m
b	f	j	n
c	g	k	o
d	h	l	p

Views of the original data, *not a copy*

```
np.reshape(X, (2,8))
X.reshape((2,8))
```

a	b
c	d
e	f
g	h
i	j
k	l
m	n
o	p

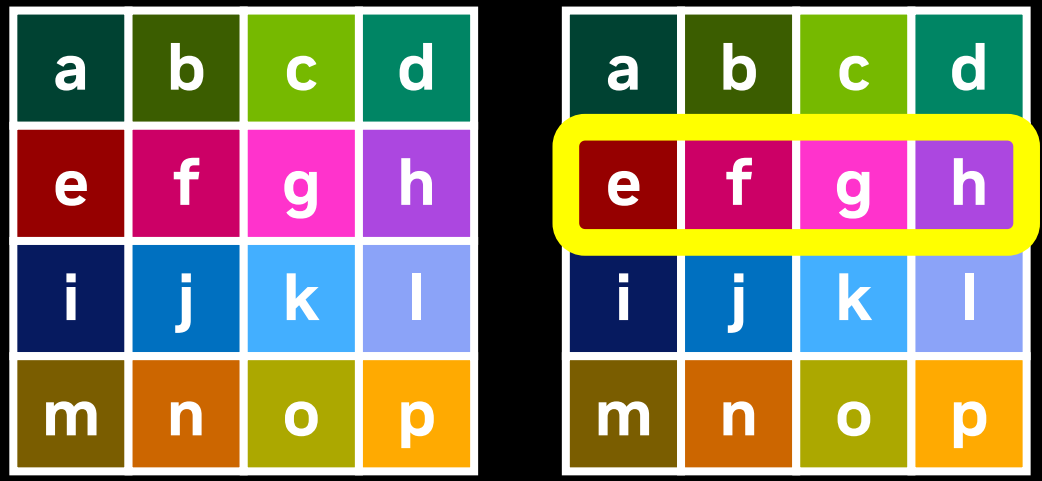
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



`X[1, 2]`

0D (Scalar)

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



`X[1, 2]`

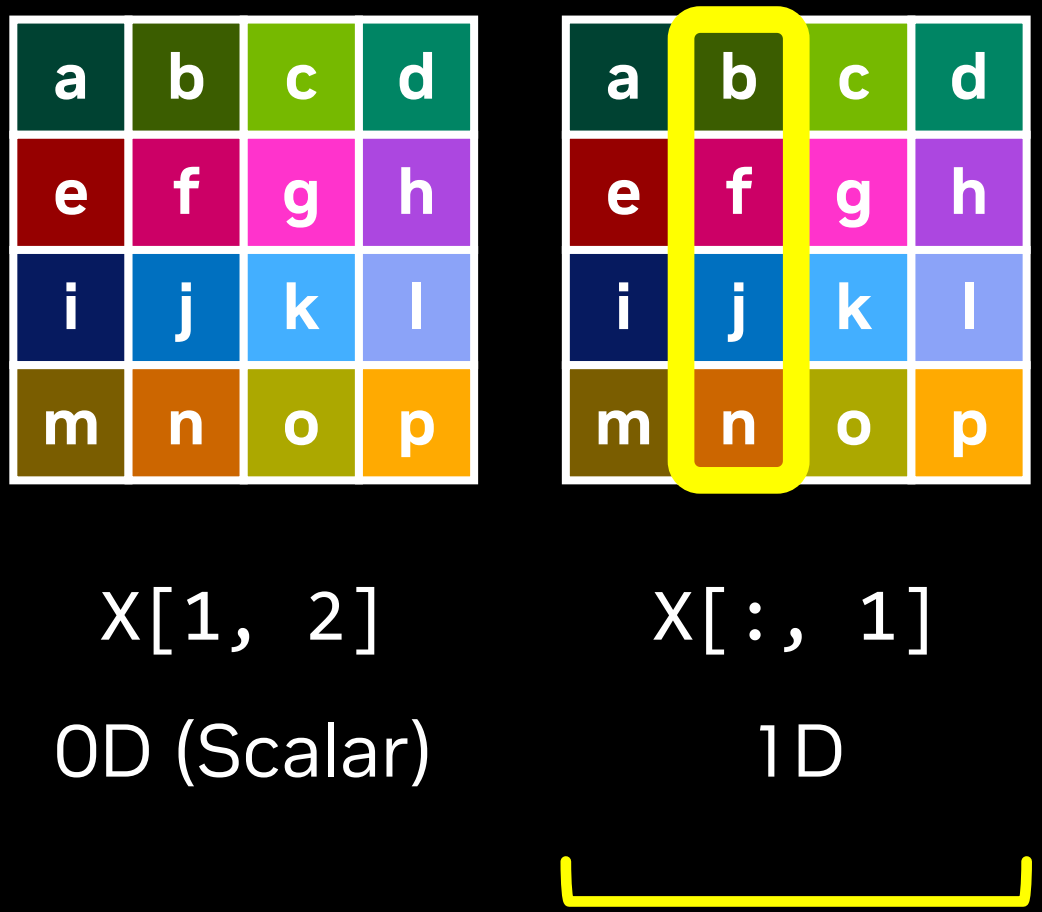
0D (Scalar)

`X[1]`

1D

Views of the original data, *not a copy*

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



`X[1, 2]`

0D (Scalar)

`X[:, 1]`

1D

Views of the original data, *not a copy*

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



`X[1, 2]`

0D (Scalar)

`X[:, 1]`

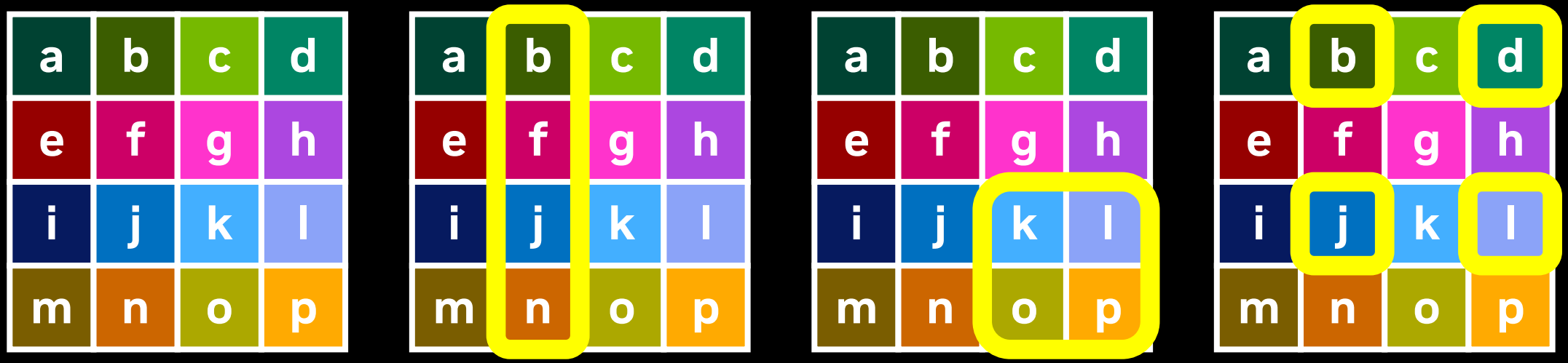
1D

`X[2:, 2:]`

2D

Views of the original data, *not a copy*

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



`X[1, 2]`

0D (Scalar)



`X[:, 1]`

1D



`X[2:, 2:]`

2D

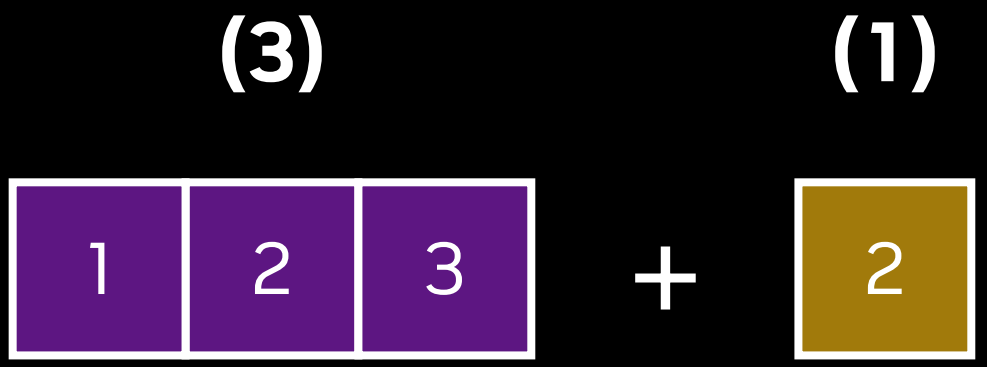


`A[::2, 1::2]`

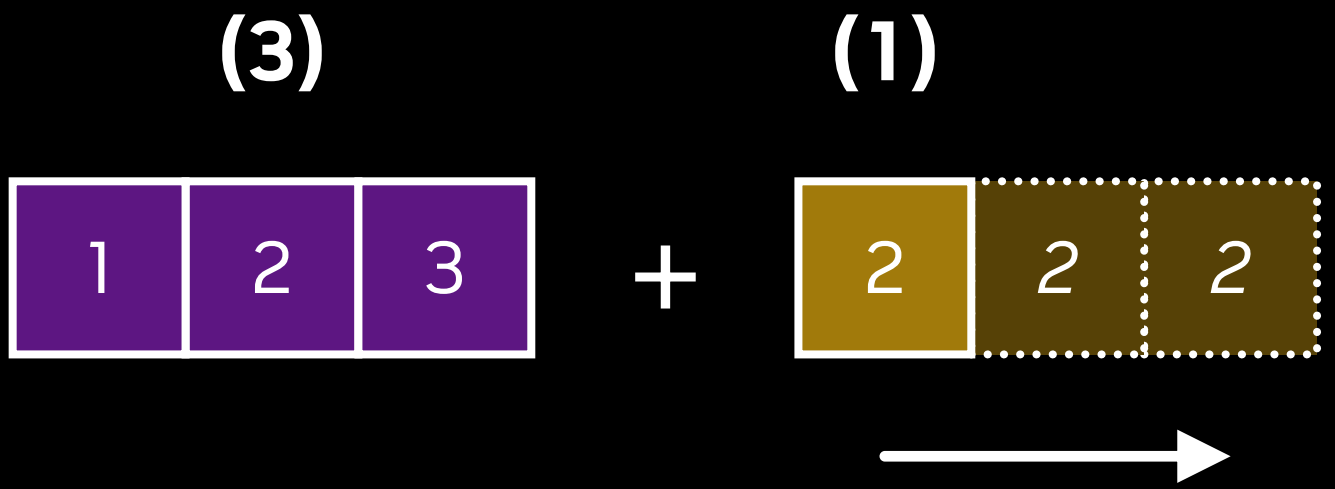
2D

`start:stop:step`

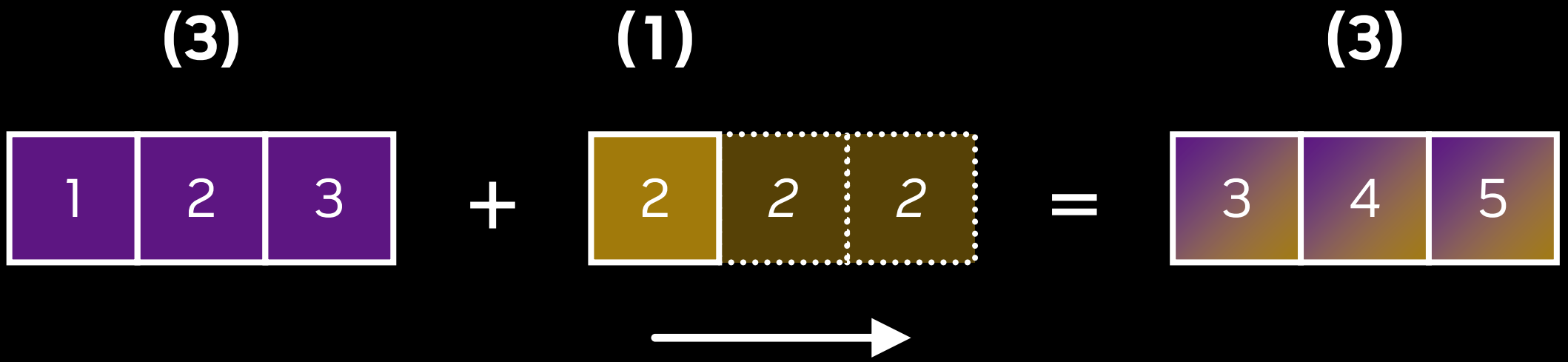
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

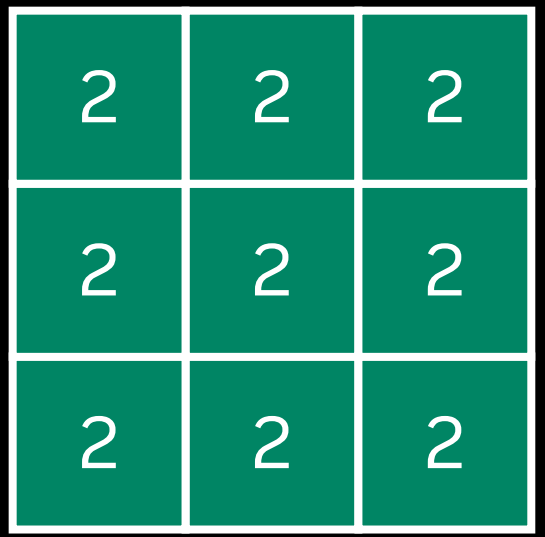


Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



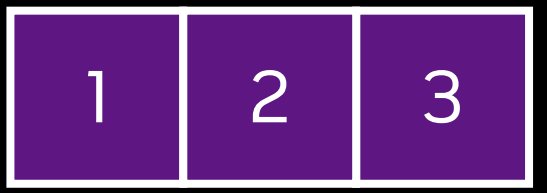
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

(3, 3)

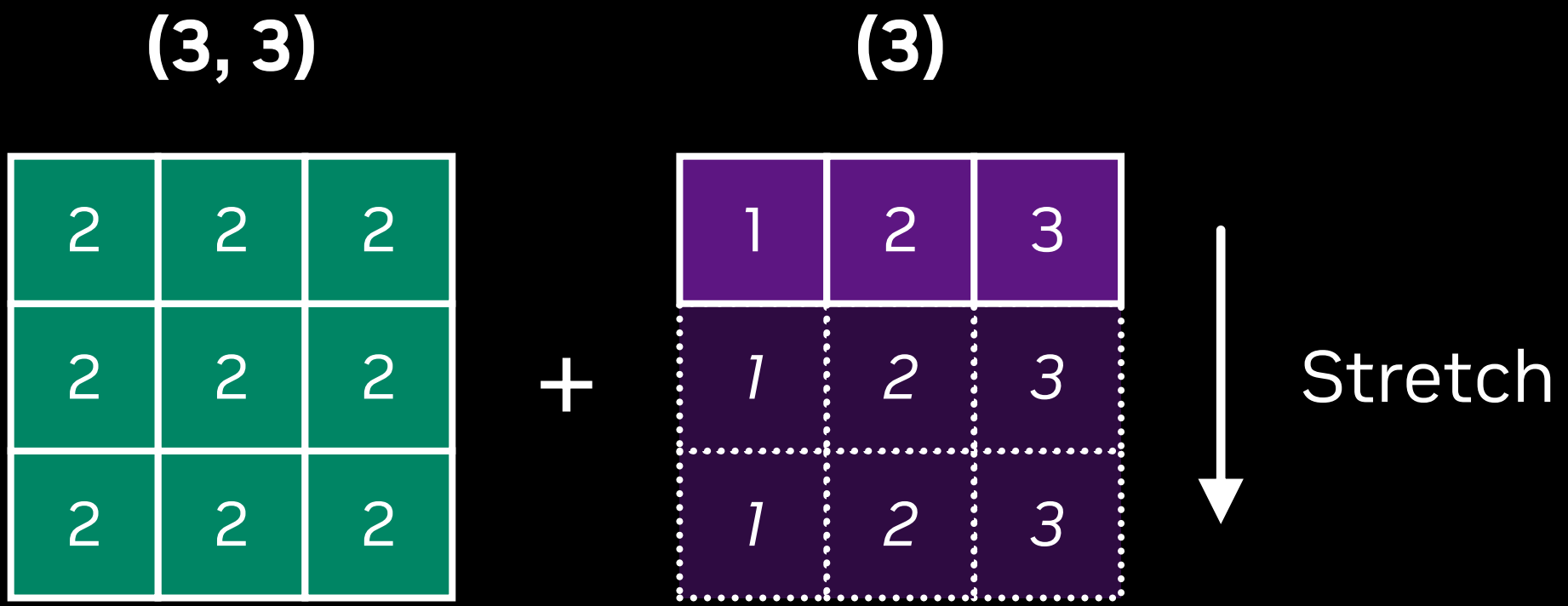


+

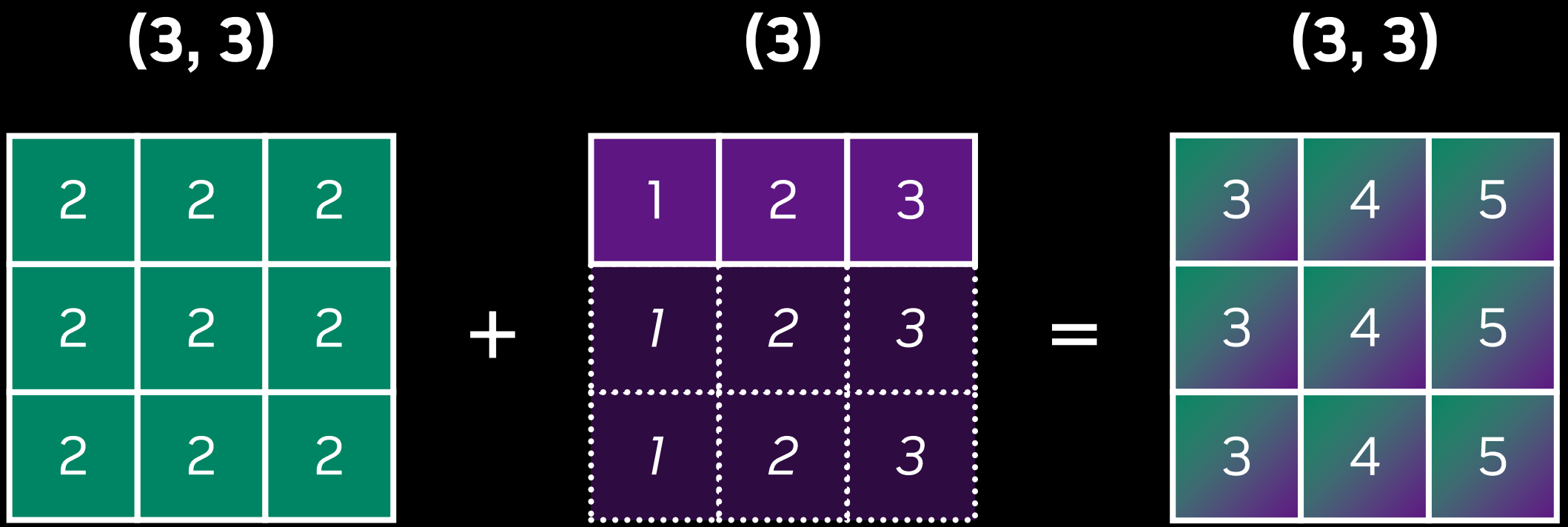
(3)



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

(3, 2)

-1	1
1	-1
-1	1

+

(3)

1	2	3

=

`ValueError:` operands could not be broadcast together with shapes (3,2) (3,)

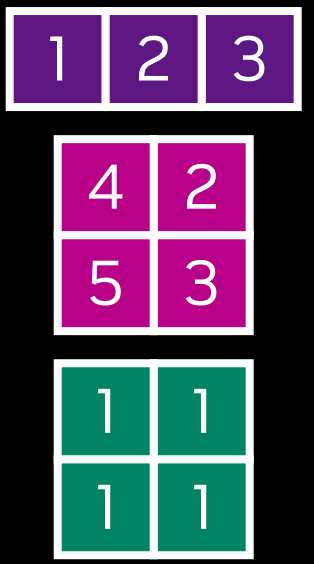
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Array Functions apply operations to the contents of arrays

```
A = np.arange(1, 3)

B = np.array([[4, 2],
              [5, 3]])

C = np.ones((2, 2),
      dtype=np.int32)
```



Some create new arrays. Specify the type with the dtype parameter.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

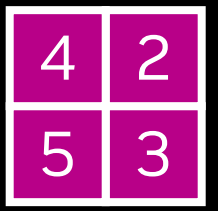
Array Functions apply operations to the contents of arrays

```
A = np.arange(1, 3)

B = np.array([[4, 2],
              [5, 3]])

C = np.ones((2, 2),
      dtype=np.int32)
```



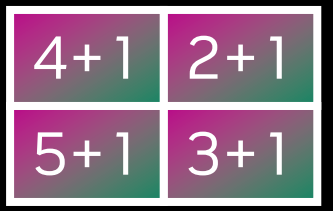




Some create new arrays. Specify the type with the dtype parameter.

```
D = np.add(B, C)

D = B + C
```



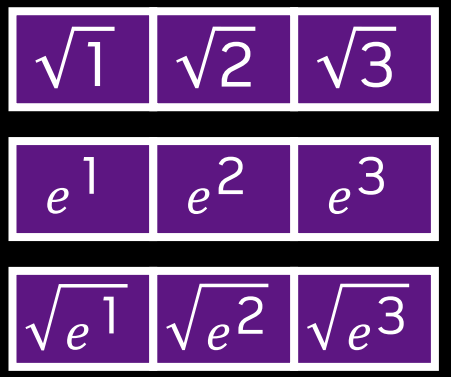
Arithmetic can be done with operators.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Array Functions apply operations to the contents of arrays

```
F = np.sqrt(A)
G = np.exp(A)

np.sqrt(A, out=A)
np.exp(A, out=A)
```



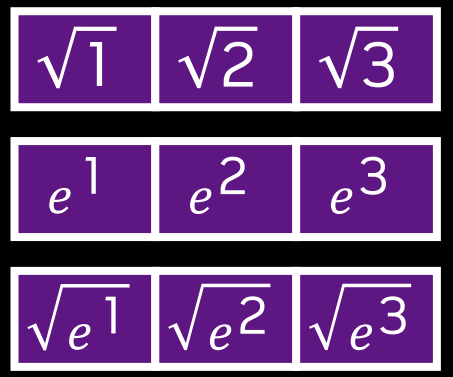
Most are elementwise and return a new array. Use the out parameter to avoid an allocation and/or operate inplace.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Array Functions apply operations to the contents of arrays

```
F = np.sqrt(A)
G = np.exp(A)

np.sqrt(A, out=A)
np.exp(A, out=A)
```

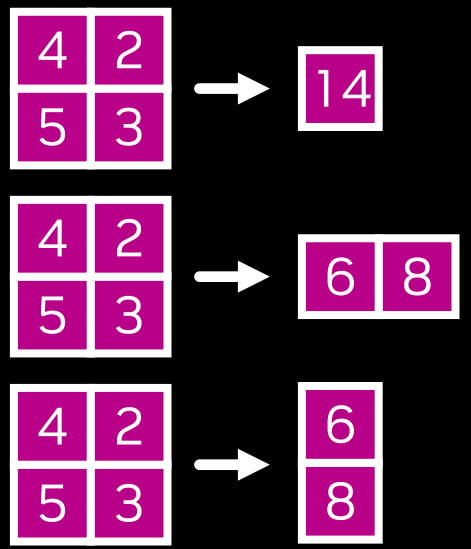


Most are elementwise and return a new array. Use the out parameter to avoid an allocation and/or operate inplace.

```
X = np.sum(A))

Y = np.sum(A, axis=1))

Z = np.sum(A, axis=1,
           keepdims=True))
```



Some can be applied to a certain axis and may reduce rank. Use the keepdims parameter to preserve the rank.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Array Functions are faster than for loops

```
A = np.random.random((1024, 1024))

def loop(A):
  for i in range(A.shape[0]):
    for j in range(A.shape[1]):
      A[i,j] = A[i,j] + A[i,j]**2

def array_ops(A):
  return A + A**2

%timeit loop(A)
%timeit array_ops(A)
```

Array functions are implemented with optimized native code in a language like C.

689 ms ± 8.76 ms per loop
(mean ± std. dev. of 7 runs, 1 loop each)
2.63 ms ± 98.7 µs per loop
(mean ± std. dev. of 7 runs, 100 loops each)

Google Colab T4 VM, Python 3.11

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Exercise 01

NumPy Intro

ndarray Basics

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Array Creation
(`np.empty, np.zeros, np.arange, …`)

Linear Algebra
(`np.dot, np.linalg.eig, …`)

Indexing / Slicing
(`np.take, np.put, np.choose, …`)

Input / Output
(`np.loadtxt, np.save, …`)

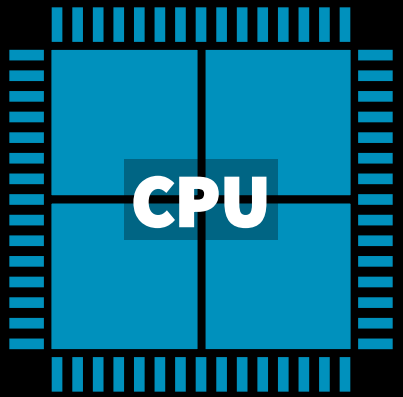
Random Numbers & Sampling
(`np.random.rand, np.random.normal,…`)

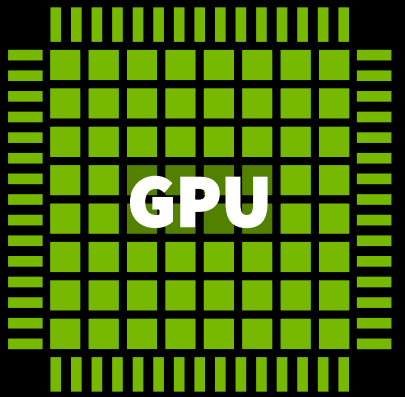
Shape Manipulation
(`np.transpose, np.hstack, …`)

Math / Logic Functions
(`np.exp, np.sin, np.isnan, …`)

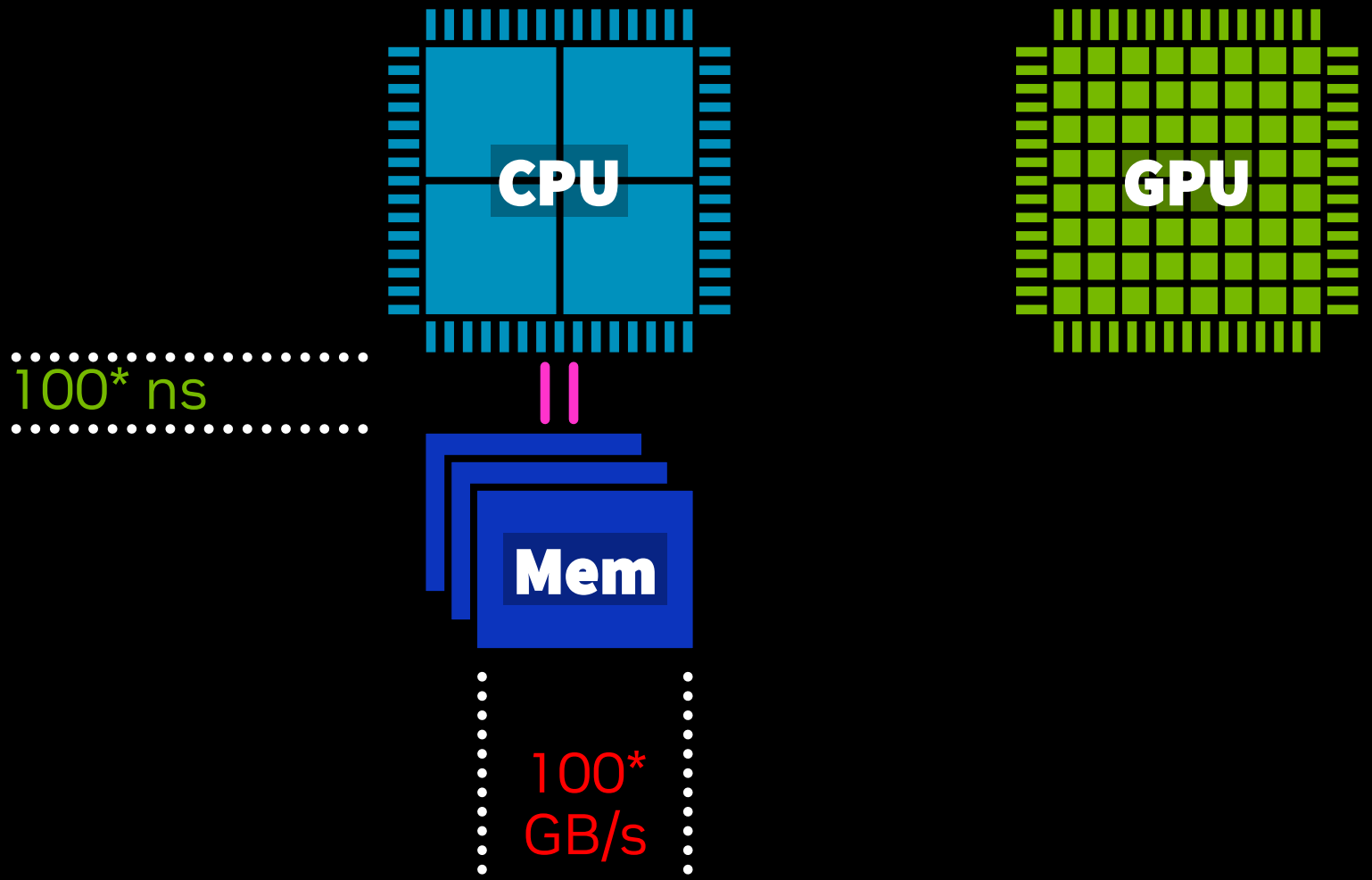
…and many more categories
(and SciPy has *even more*)

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



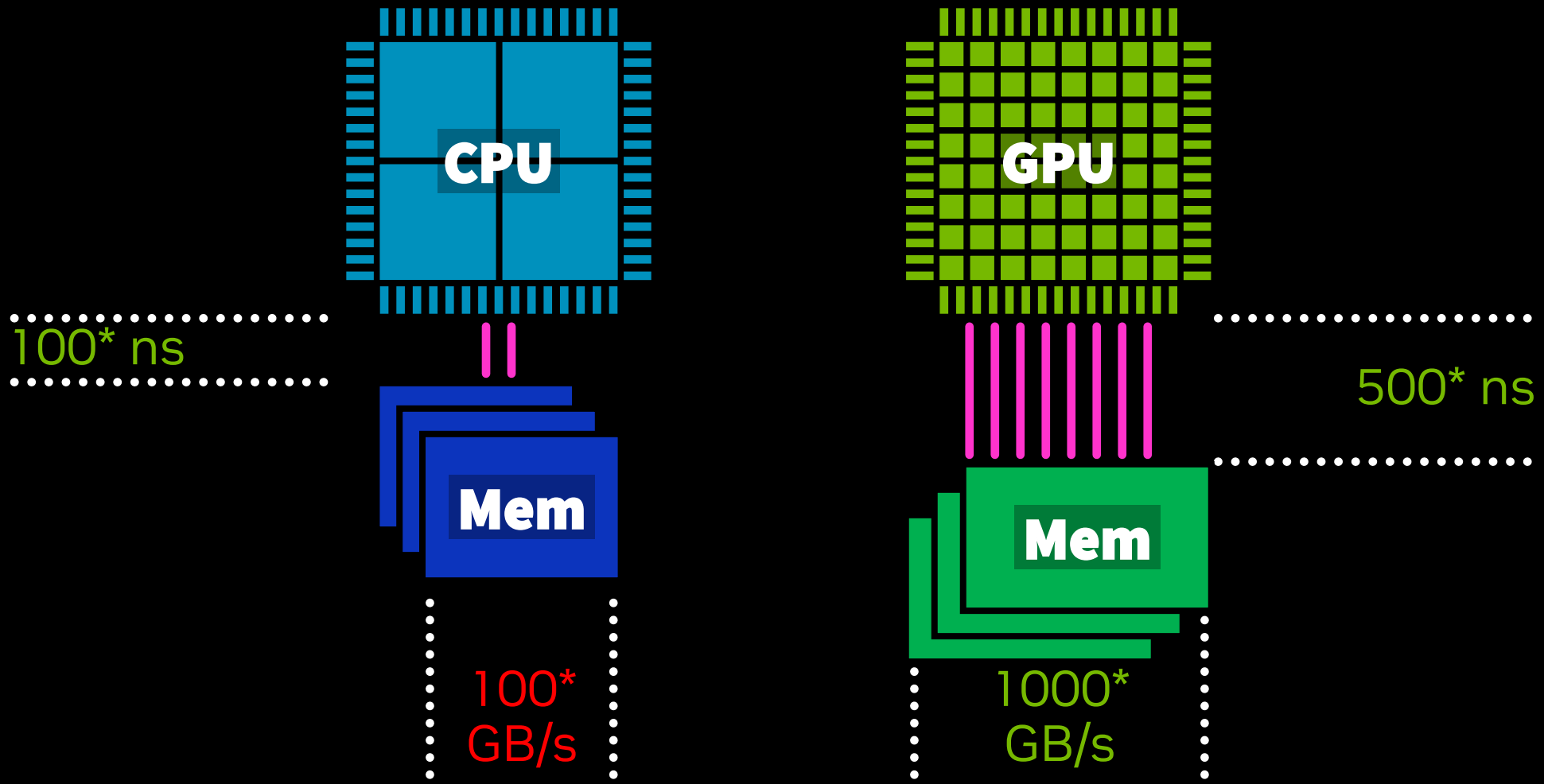


Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



Numbers are made up and
for expository purposes only.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



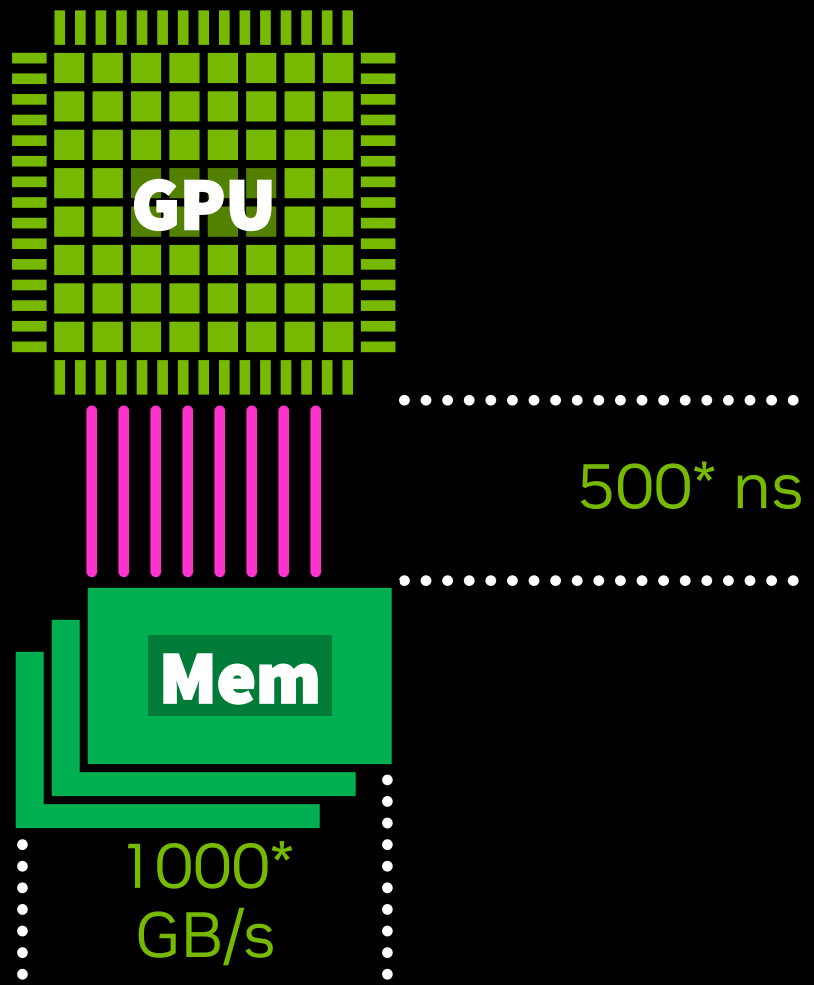
Numbers are made up and
for expository purposes only.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

You need a sufficient workload to benefit from GPUs.

O(N) compute => GBs of data.

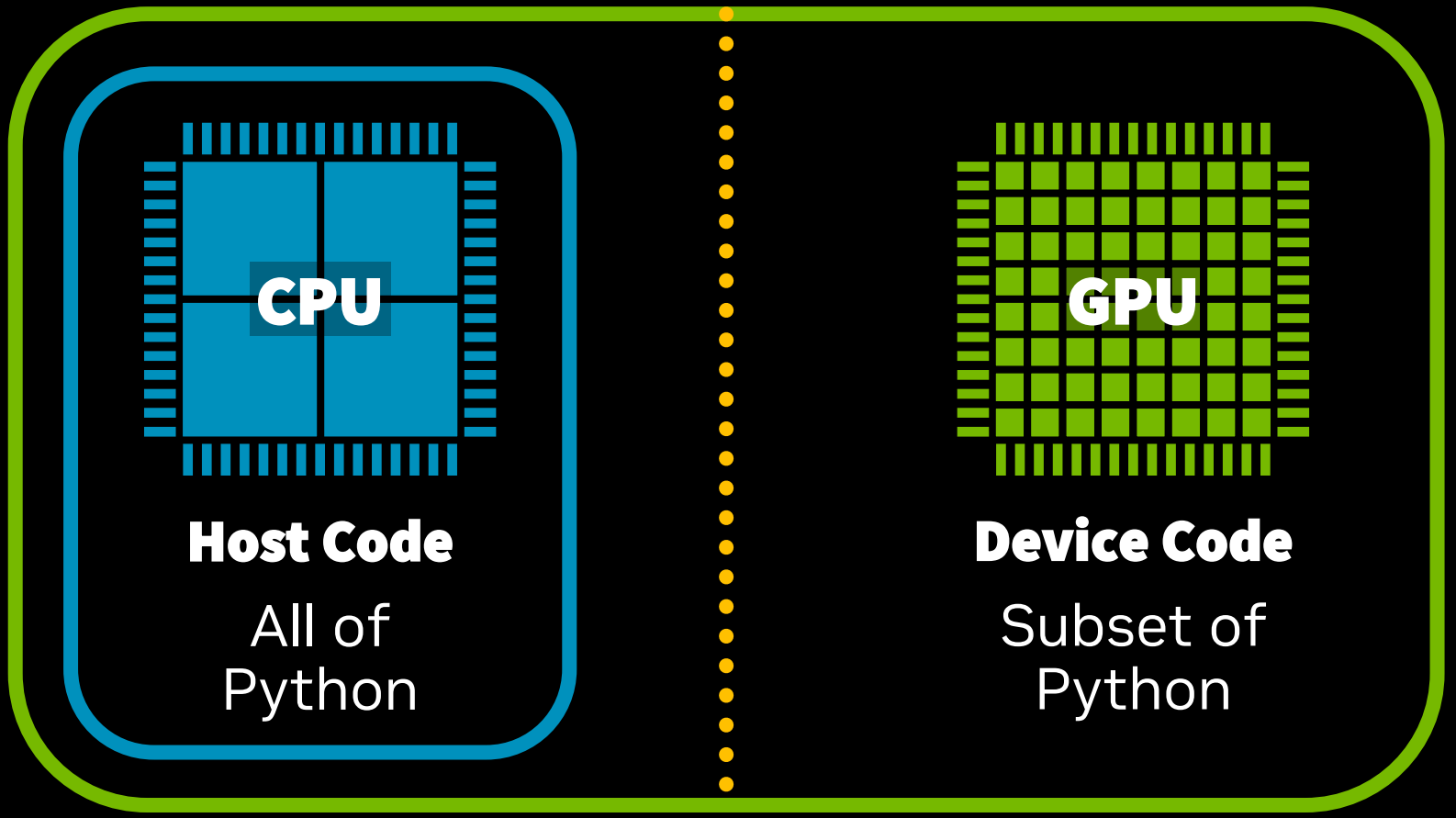
More than O(N) compute => less data.



Numbers are made up and for expository purposes only.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

CUDA Python is an extension of Python for writing programs that simultaneously run on CPUs (hosts) & GPUs (devices).



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



NumPy: Array Programming

```python
import numpy as np

# Riemann sum of ∫₀¹x²dx
x = np.linspace(0, 1, 100)
y = x**2
dx = x[1] - x[0]
I = np.sum(y) * dx
```

The de facto standard for array data in Python.

Every HPC and ML library references NumPy in some way!

NumPy specifies:

- `ndarray`: Multidimensional array type.
- Array indexing and slicing.
- A common set of array functions.
- How to define your own array functions.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



CuPy: NumPy API for GPUs

- A NumPy / SciPy library that runs *on the GPU.*
- 80/90% compatible; we'll learn some of the limitations today.
- Handles allocating memory on the GPU & running GPU kernels for you.
- Open source, NVIDIA CUDA & AMD ROCm backends.

```
import numpy as np

current_state = (np.random.uniform(size=1000*1000) > 0.8).reshape((1000,1000))

def life_step_array_func(X):
    nbrs_count = sum(np.roll(np.roll(X, i, 0), j, 1)
                     for i in (-1, 0, 1) for j in (-1, 0, 1)
                     if (i != 0 or j != 0))
    return (nbrs_count == 3) | (X & (nbrs_count == 2))

%timeit life_step_array_func(current_state)
```

```
11.1 ms ± 11.1 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

```
import cupy as cp

current_state = (cp.random.uniform(size=1000*1000) > 0.8).reshape((1000,1000))

def life_step_array_func_cupy(X):
    nbrs_count = sum(cp.roll(cp.roll(X, i, 0), j, 1)
                     for i in (-1, 0, 1) for j in (-1, 0, 1)
                     if (i != 0 or j != 0))
    return (nbrs_count == 3) | (X & (nbrs_count == 2))

%timeit life_step_array_func_cupy(current_state)
```

```
955 µs ± 1.95 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

Nearly identical code, **10x faster** (Quadro RTX 8000)

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Exercise 03

NumPy to CuPy

ndarray Basics

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

CuPy Isn't *Always* Just Like NumPy

- Not all NumPy / SciPy functions exist (but most do).
- Structured object, and string dtypes are not supported.
- Out of bounds access wraps around back to the beginning of the array.
- Memory transfers are necessary:
  - Arrays from NumPy and libraries like OpenCV need to be sent to the GPU to use with CuPy.
  - CuPy arrays need to be brought back to the CPU to use with other libraries, like matplotlib.
- Different promotion rules.
  - NumPy is safe even if slow, CuPy prefers speed and avoids implicit slowness.
- Not bit-for-bit equivalent.
  - Some algorithms like pseudo-random number generation give different results.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Avoid Serial For Loops!

```
a_cpu = np.zeros(1000, dtype=np.float32)
```

```
%%timeit
for i in range(a_cpu.shape[0]):
    a_cpu[i] = i
```

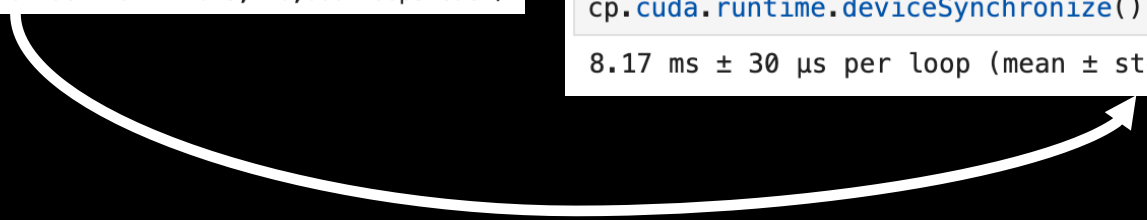
```
82.4 µs ± 220 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

```
a_gpu = cp.zeros(1000, dtype=cp.float32)
```

```
%%timeit
for i in range(a_gpu.shape[0]):
    a_gpu[i] = i

## need to make sure GPU code is done for benchmarking purposes
cp.cuda.runtime.deviceSynchronize()
```

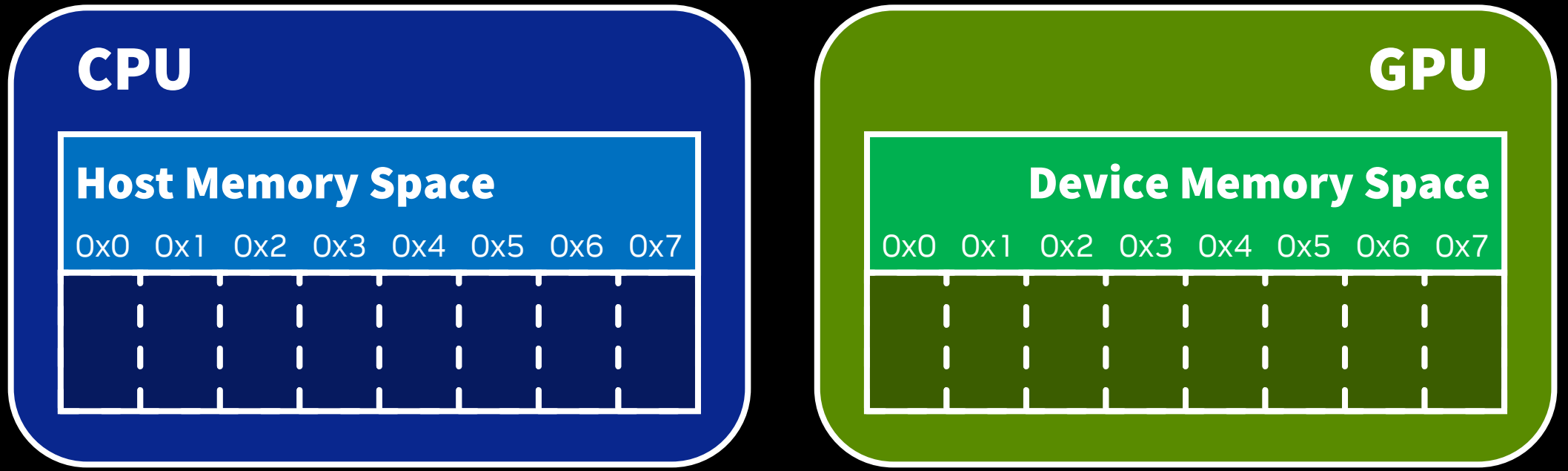
```
8.17 ms ± 30 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```



Elementwise operations on arrays are **100x worse** on the GPU than the CPU.

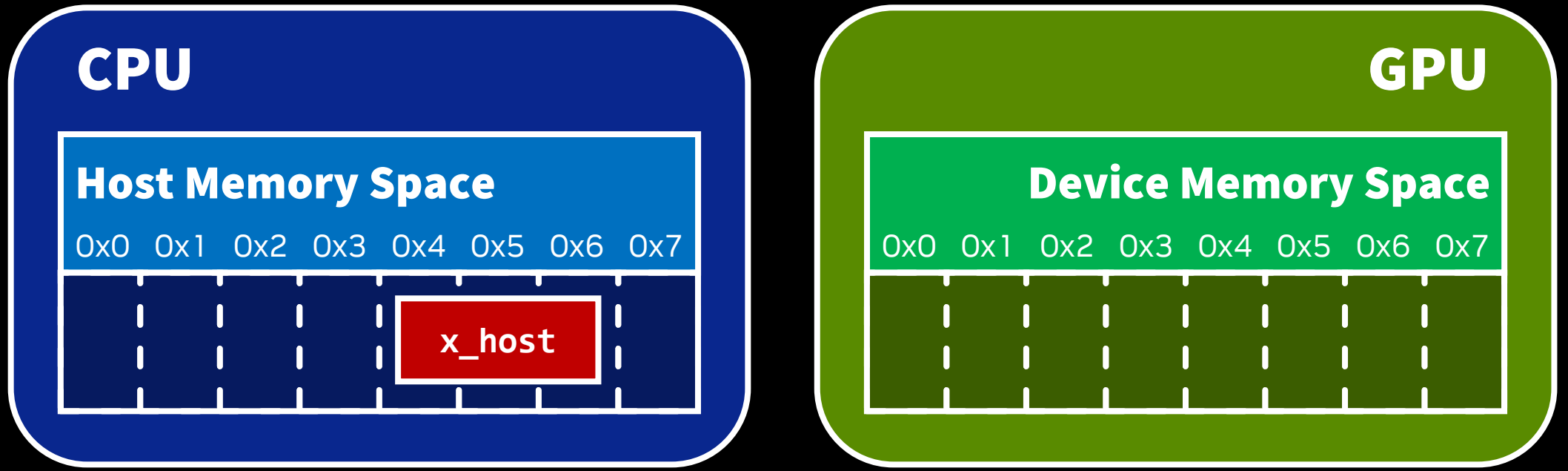
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Heterogeneous systems have multiple kinds of memory.



CPUs & GPUs may only have access to a subset of memory.

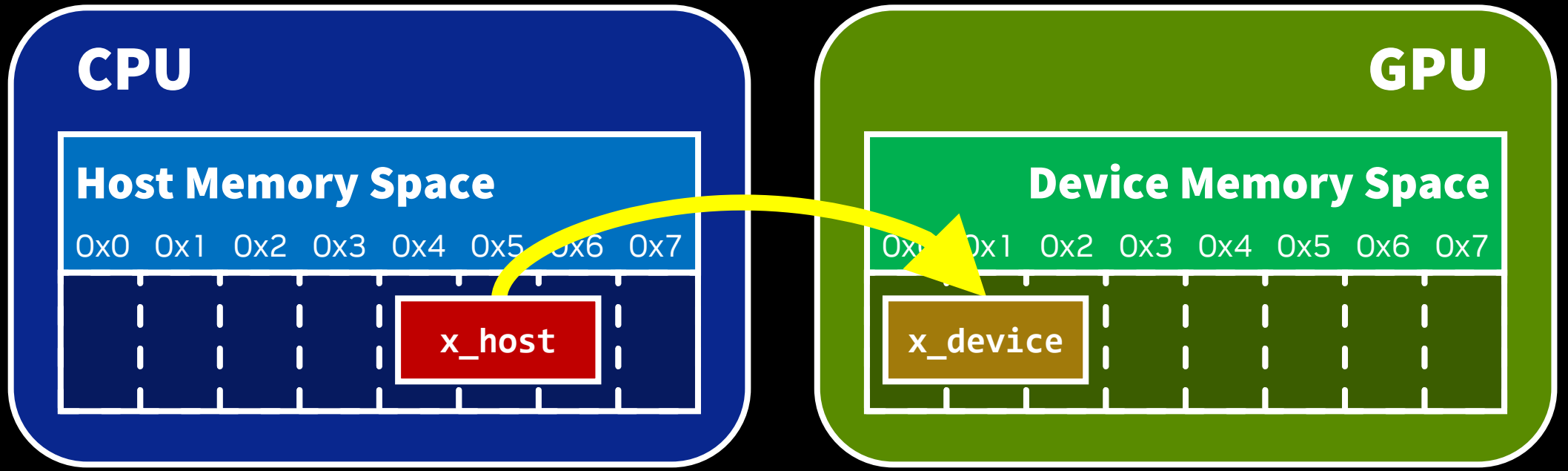
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig



How do we transfer data between host and device?

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

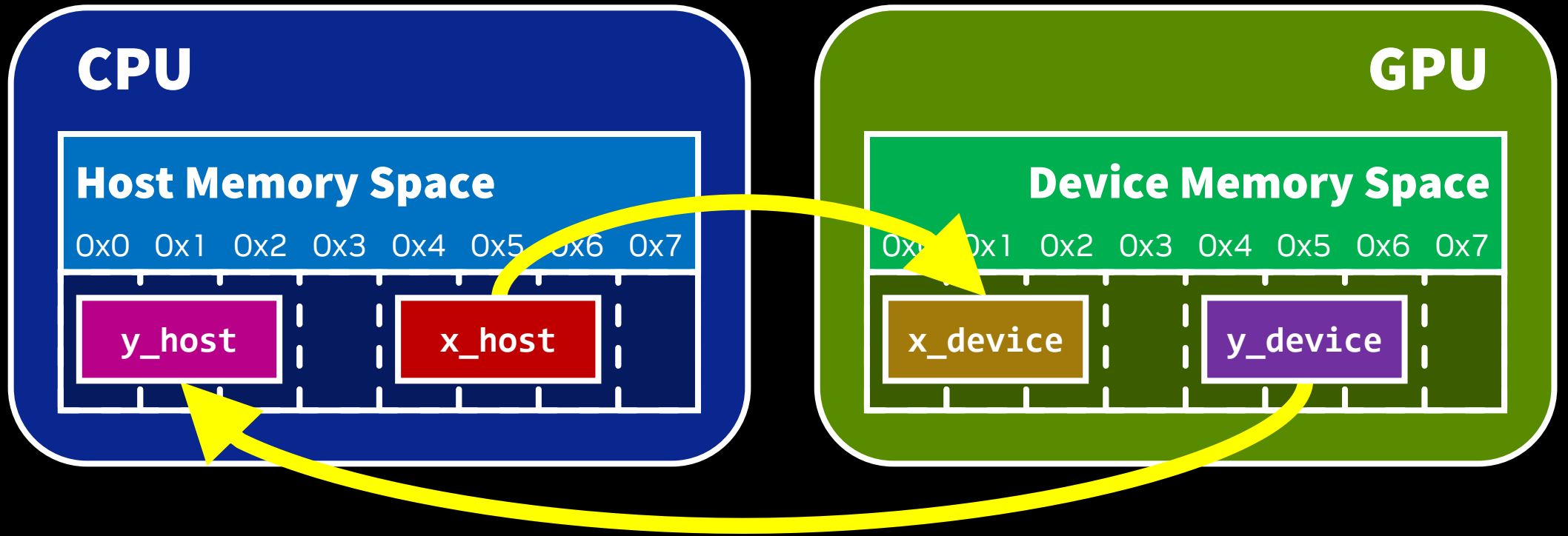
```
x_device = cp.asarray(x_host)
```



How do we transfer data between host and device?

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
x_device = cp.asarray(x_host)
```




```
y_host = cp.asnumpy(y_device)
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

- Printing or representing (`print(x)`, `str(x)`, f-string interpolation).

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

- Printing or representing (`print(x)`, `str(x)`, f-string interpolation).
- Converting to Python scalars or lists (`int(x)`, `float(x)`, `bool(x)`, `x.item()`).

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

- Printing or representing (`print(x)`, `str(x)`, f-string interpolation).
- Converting to Python scalars or lists (`int(x)`, `float(x)`, `bool(x)`, `x.item()`).
- CuPy function that falls back to NumPy/SciPy because no GPU version exists (`save(x)`, `load(x)`).

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

- Printing or representing (`print(x)`, `str(x)`, f-string interpolation).
- Converting to Python scalars or lists (`int(x)`, `float(x)`, `bool(x)`, `x.item()`).
- CuPy function that falls back to NumPy/SciPy because no GPU version exists (`save(x)`, `load(x)`).

```
N2 = np.zeros((2, 2))
C2 = cp.zeros((2, 2))
C0 = cp.sum(C1)
B = 3.14
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

- Printing or representing (`print(x)`, `str(x)`, f-string interpolation).
- Converting to Python scalars or lists (`int(x)`, `float(x)`, `bool(x)`, `x.item()`).
- CuPy function that falls back to NumPy/SciPy because no GPU version exists (`save(x)`, `load(x)`).

```
N2 = np.zeros((2, 2))
C2 = cp.zeros((2, 2))
C0 = cp.sum(C1)
B = 3.14

# For arrays of rank 1 and greater:
C2 + C2 # CuPy + CuPy doesn't transfer/sync.
C2 + N2 # CuPy + NumPy is an ERROR.
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

- Printing or representing (`print(x)`, `str(x)`, f-string interpolation).
- Converting to Python scalars or lists (`int(x)`, `float(x)`, `bool(x)`, `x.item()`).
- CuPy function that falls back to NumPy/SciPy because no GPU version exists (`save(x)`, `load(x)`).

```
N2 = np.zeros((2, 2))
C2 = cp.zeros((2, 2))
C0 = cp.sum(C1)
B = 3.14

# For arrays of rank 1 and greater:
C2 + C2 # CuPy + CuPy doesn't transfer/sync.
C2 + N2 # CuPy + NumPy is an ERROR.

# For 0D arrays (e.g. scalars):
C0 + N2 # CuPy + NumPy is an ERROR.
C0 + B  # CuPy + builtins transfers w/o sync.
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

- Printing or representing (`print(x)`, `str(x)`, f-string interpolation).
- Converting to Python scalars or lists (`int(x)`, `float(x)`, `bool(x)`, `x.item()`).
- CuPy function that falls back to NumPy/SciPy because no GPU version exists (`save(x)`, `load(x)`).

```
N2 = np.zeros((2, 2))
C2 = cp.zeros((2, 2))
C0 = cp.sum(C1)
B = 3.14

# For arrays of rank 1 and greater:
C2 + C2 # CuPy + CuPy doesn't transfer/sync.
C2 + N2 # CuPy + NumPy is an ERROR.

# For 0D arrays (e.g. scalars):
C0 + N2 # CuPy + NumPy is an ERROR.
C0 + B  # CuPy + builtins transfers w/o sync.

print(f"{C2}") # Printing will transfer & sync.
if C0: pass    # Conditions will transfer & sync.
float(C0)      # Conversion will transfer & sync.
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Beware Implicit Memory Transfers

CuPy silently transfers & ***synchronizes*** when:

- Printing or representing (`print(x)`, `str(x)`, f-string interpolation).
- Converting to Python scalars or lists (`int(x)`, `float(x)`, `bool(x)`, `x.item()`).
- CuPy function that falls back to NumPy/SciPy because no GPU version exists (`save(x)`, `load(x)`).

```
N2 = np.zeros((2, 2))
C2 = cp.zeros((2, 2))
C0 = cp.sum(C1)
B = 3.14

# For arrays of rank 1 and greater:
C2 + C2 # CuPy + CuPy doesn't transfer/sync.
C2 + N2 # CuPy + NumPy is an ERROR.

# For 0D arrays (e.g. scalars):
C0 + N2 # CuPy + NumPy is an ERROR.
C0 + B  # CuPy + builtins transfers w/o sync.

print(f"{C2}") # Printing will transfer & sync.
if C0: pass    # Conditions will transfer & sync.
float(C0)      # Conversion will transfer & sync.

C2[0, 0] # Indexing doesn't transfer.
C2[0, :] # Slicing doesn't transfer.
C2[0, 0] = 7 # Scalar assigns transfer w/o sync.
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Exercise 05

Memory Spaces

Power Iteration

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
A = cp.asarray(…)

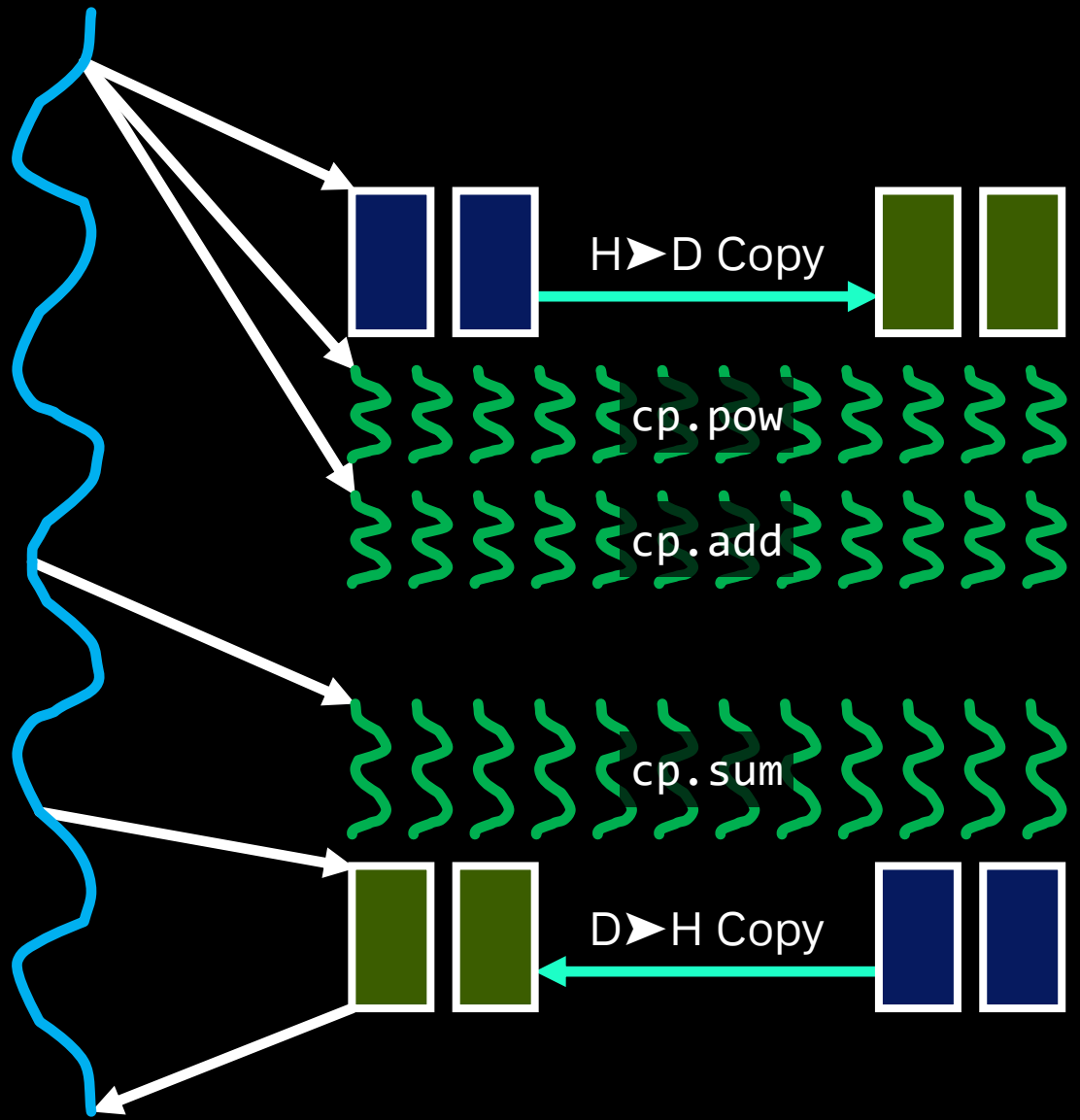
A = A + A**2

do_cpu_stuff()

x = A.sum(axis=1)

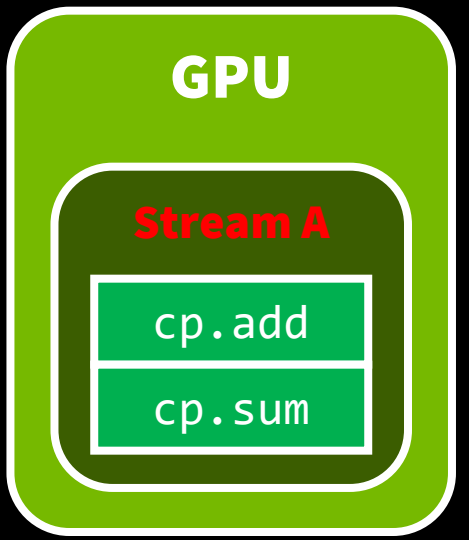
do_cpu_other_stuff()

x_host = cp.asnumpy(x)
```



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

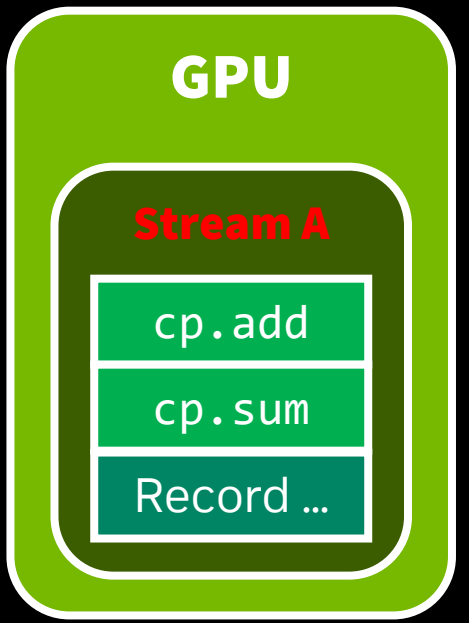
```
A = cp.cuda.Stream(non_blocking=True)
with A:
  cp.add(…)
  cp.sum(…)
```



A stream is a chain of device work that is executed in order.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
A = cp.cuda.Stream(non_blocking=True)
with A:
  cp.add(…)
  cp.sum(…)
  A.synchronize() # Host waits.
```

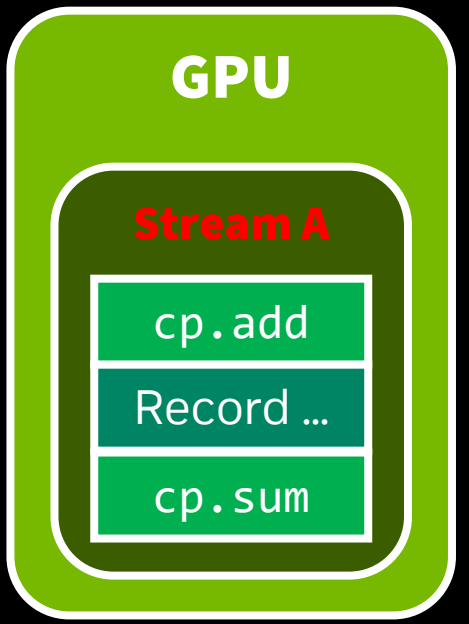


A stream is a chain of device work that is executed in order.

They can record events that host code can sync with.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
A = cp.cuda.Stream(non_blocking=True)
with A:
  cp.add(…)
  A.synchronize() # Host waits.
  cp.sum(…)
```



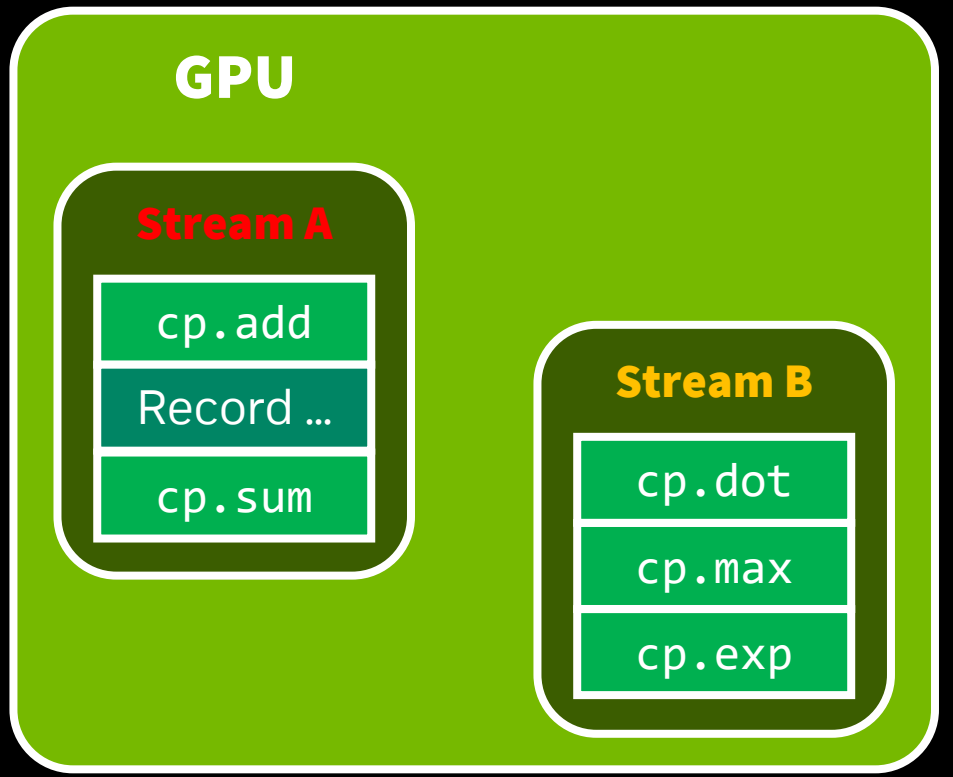
A stream is a chain of device work that is executed in order.

They can record events that host code can sync with.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
A = cp.cuda.Stream(non_blocking=True)
with A:
  cp.add(…)
  A.synchronize() # Host waits.
  cp.sum(…)

B = cp.cuda.Stream(non_blocking=True)
with B:
  cp.dot(…)
  cp.max(…)
  cp.exp(…)
```



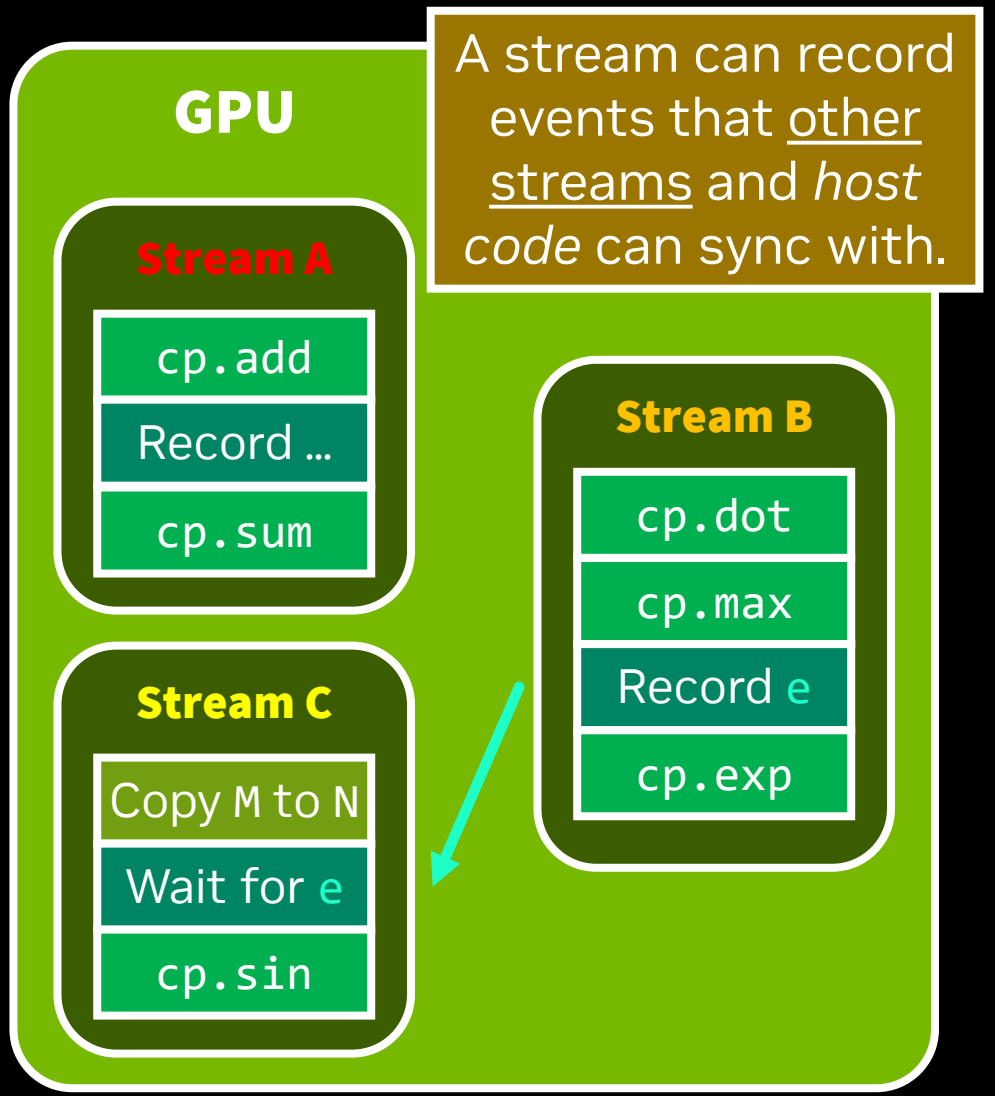
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
A = cp.cuda.Stream(non_blocking=True)
with A:
  cp.add(…)
  A.synchronize() # Host waits.
  cp.sum(…)

B = cp.cuda.Stream(non_blocking=True)
with B:
  cp.dot(…)
  cp.max(…)
  e = B.record()
  cp.exp(…)

C = cp.cuda.Stream(non_blocking=True)
with C:
  N = cp.asnumpy(M)
  C.wait_event(e) # Device waits.
  cp.sin(…)

e.synchronize() # Host waits.
```



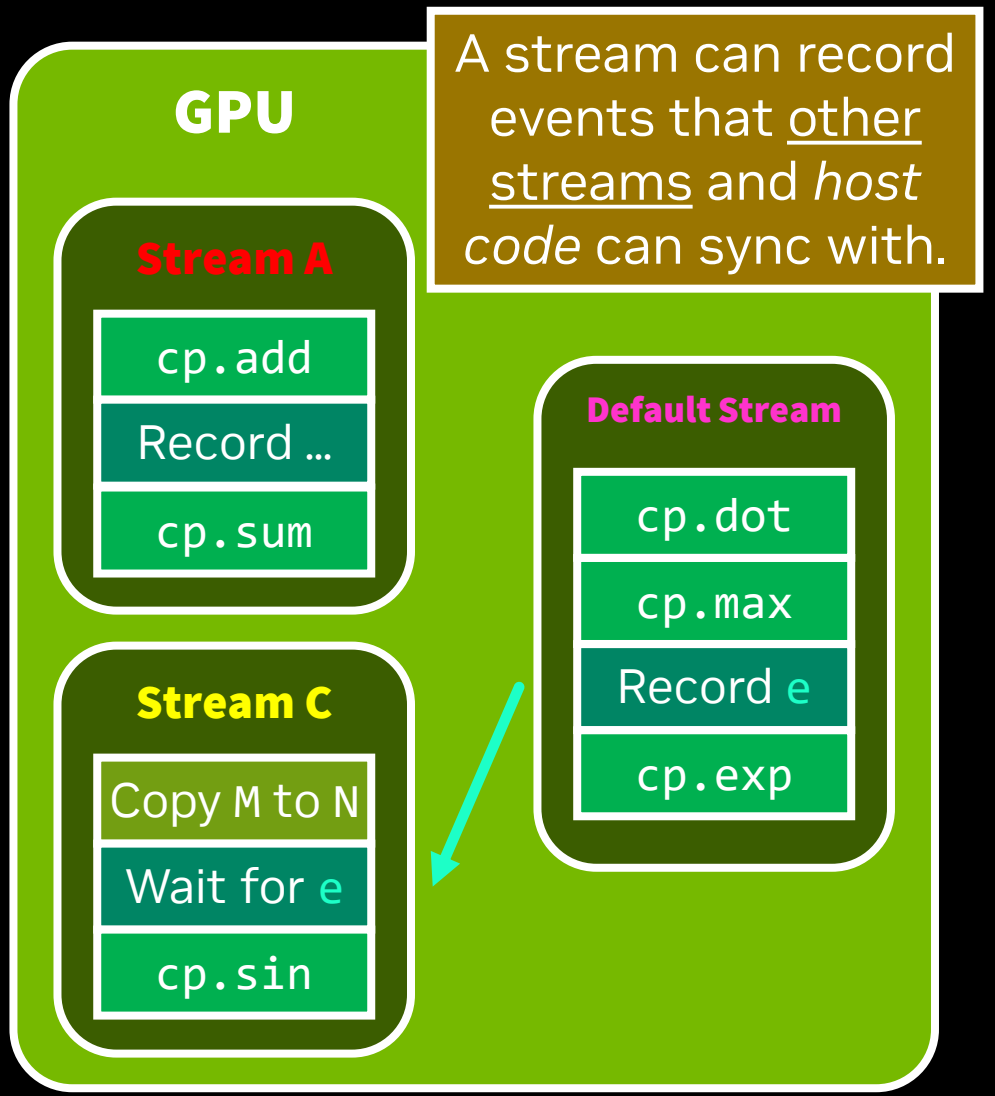
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
A = cp.cuda.Stream(non_blocking=True)
with A:
  cp.add(…)
  A.synchronize() # Host waits.
  cp.sum(…)

cp.dot(…)
cp.max(…)
e = cp.cuda.get_current_stream().record()
cp.exp(…)

C = cp.cuda.Stream(non_blocking=True)
with C:
  N = cp.asnumpy(M)
  C.wait_event(e) # Device waits.
  cp.sin(…)

e.synchronize() # Host waits.
```



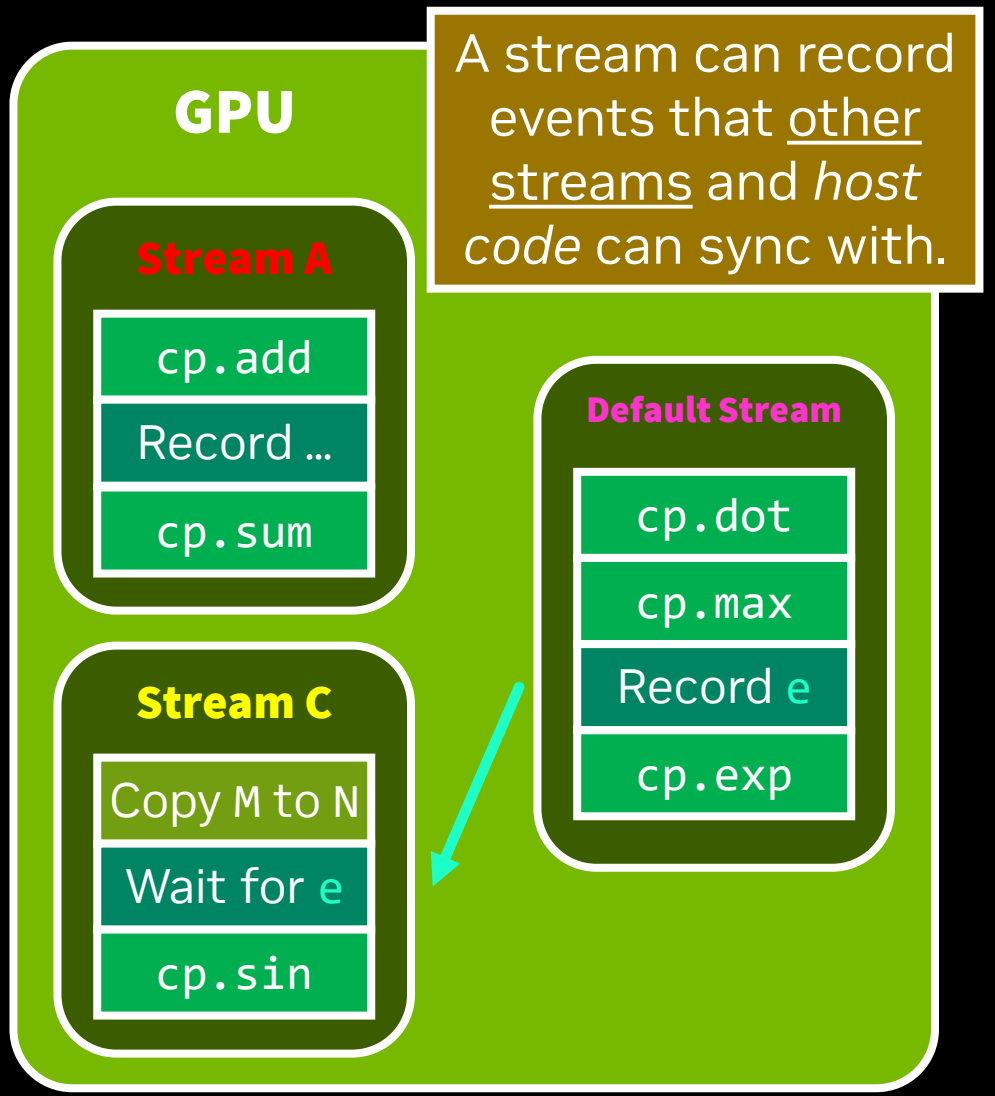
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
A = cp.cuda.Stream(non_blocking=True)
with A:
  cp.add(…)
  A.synchronize() # Host waits.
  cp.sum(…)

cp.dot(…)
cp.max(…)
e = cp.cuda.get_current_stream().record()
cp.exp(…)

C = cp.cuda.Stream(non_blocking=True)
with C:
  N = cp.asnumpy(M)
  C.wait_event(e) # Device waits.
  cp.sin(…)

e.synchronize() # Host waits.
```



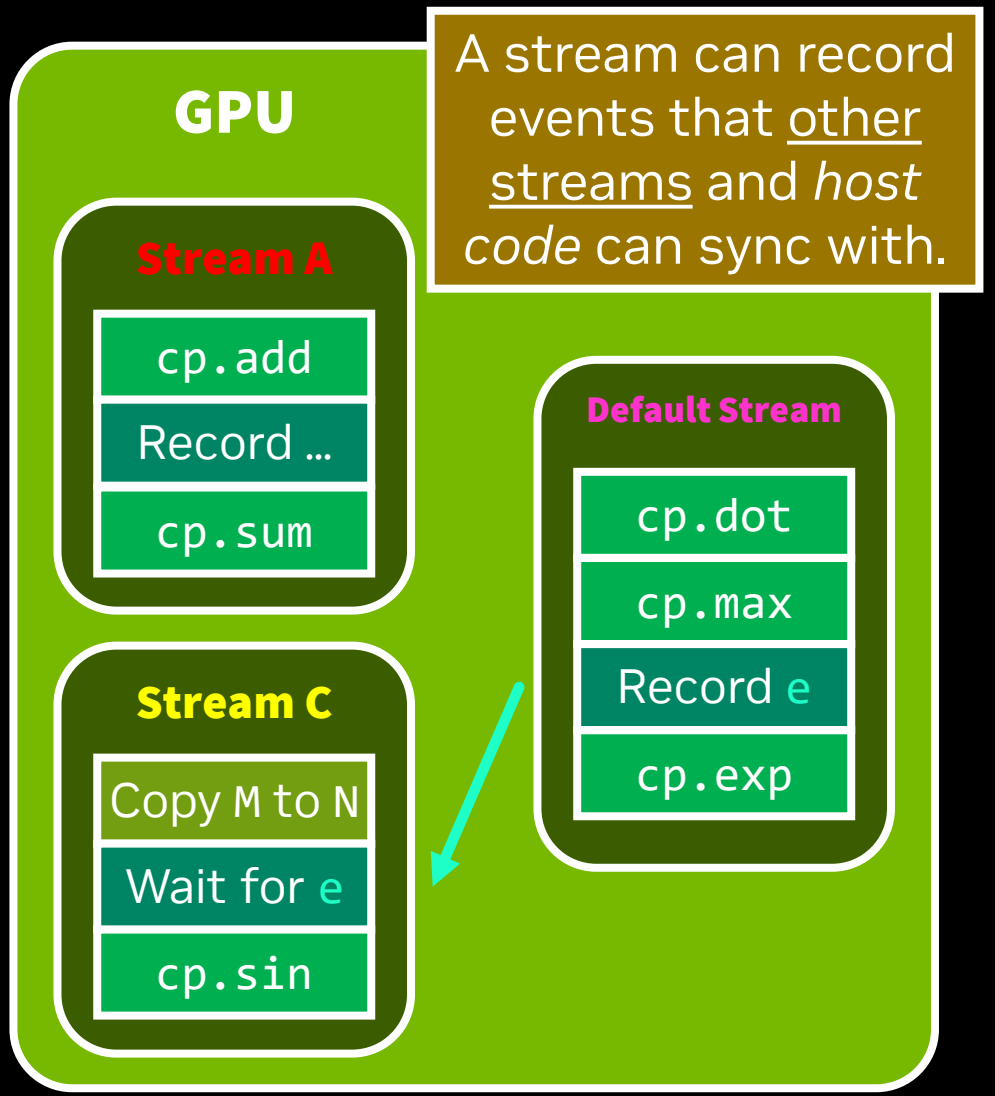
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

```
A = cp.cuda.Stream(non_blocking=True)
with A:
  cp.add(…)
  A.synchronize() # Host waits.
  cp.sum(…)

cp.dot(…)
cp.max(…)
e = cp.cuda.get_current_stream().record()
cp.exp(…)

C = cp.cuda.Stream(non_blocking=True)
with C:
  N = cp.asnumpy(M, blocking=False)
  C.wait_event(e) # Device waits.
  cp.sin(…)

e.synchronize() # Host waits.
```



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

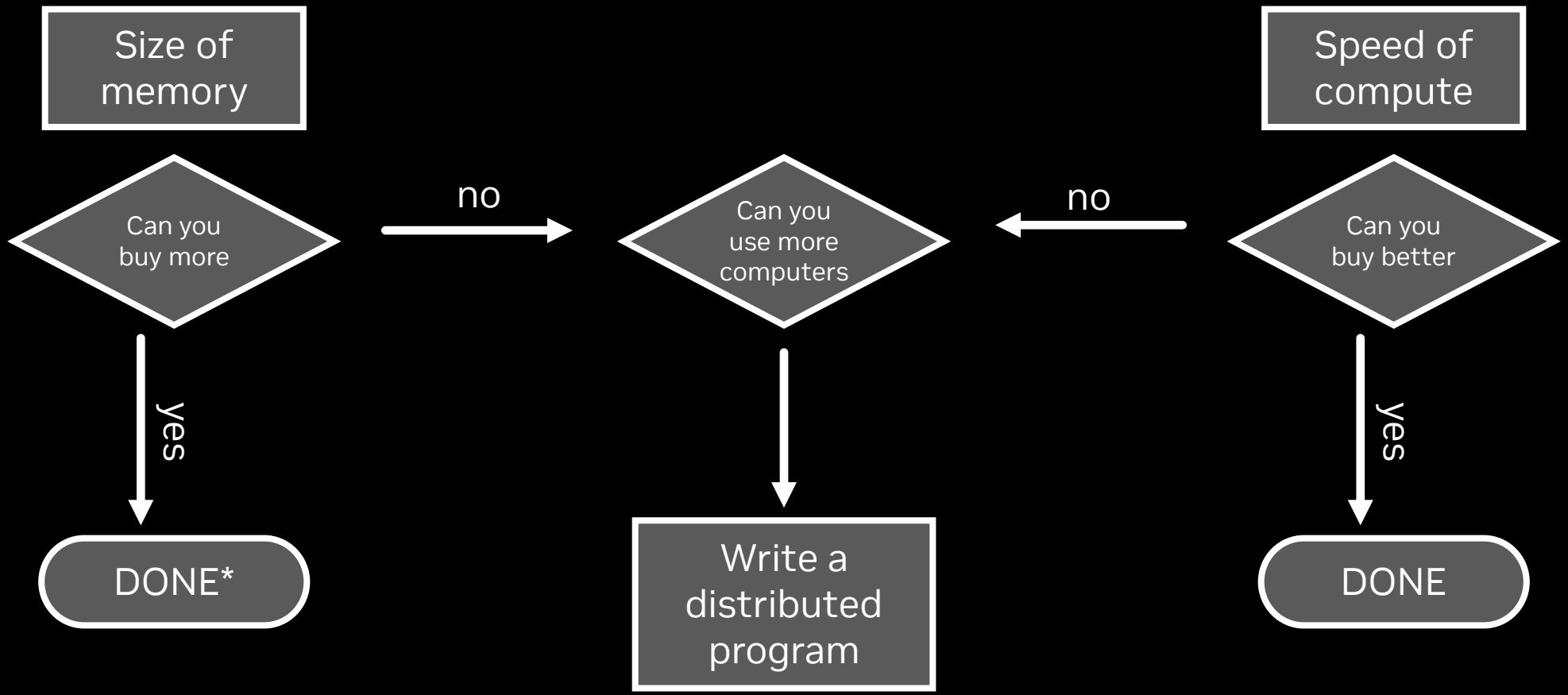
Exercise 06

Asynchrony

Power Iteration

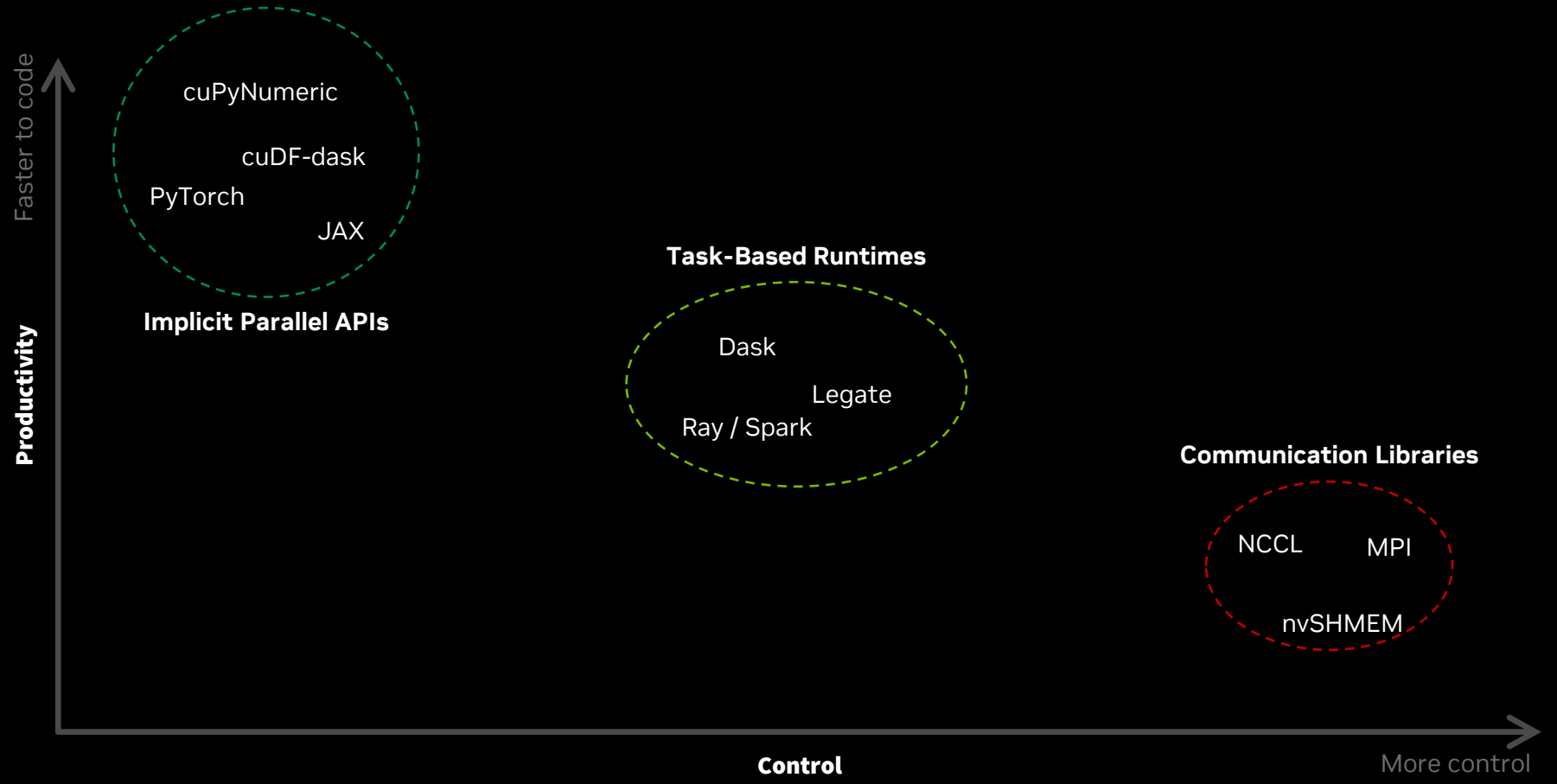
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

What are some limits to scale?



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Distributed Programming Landscape



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

MPI – The HPC mainstay

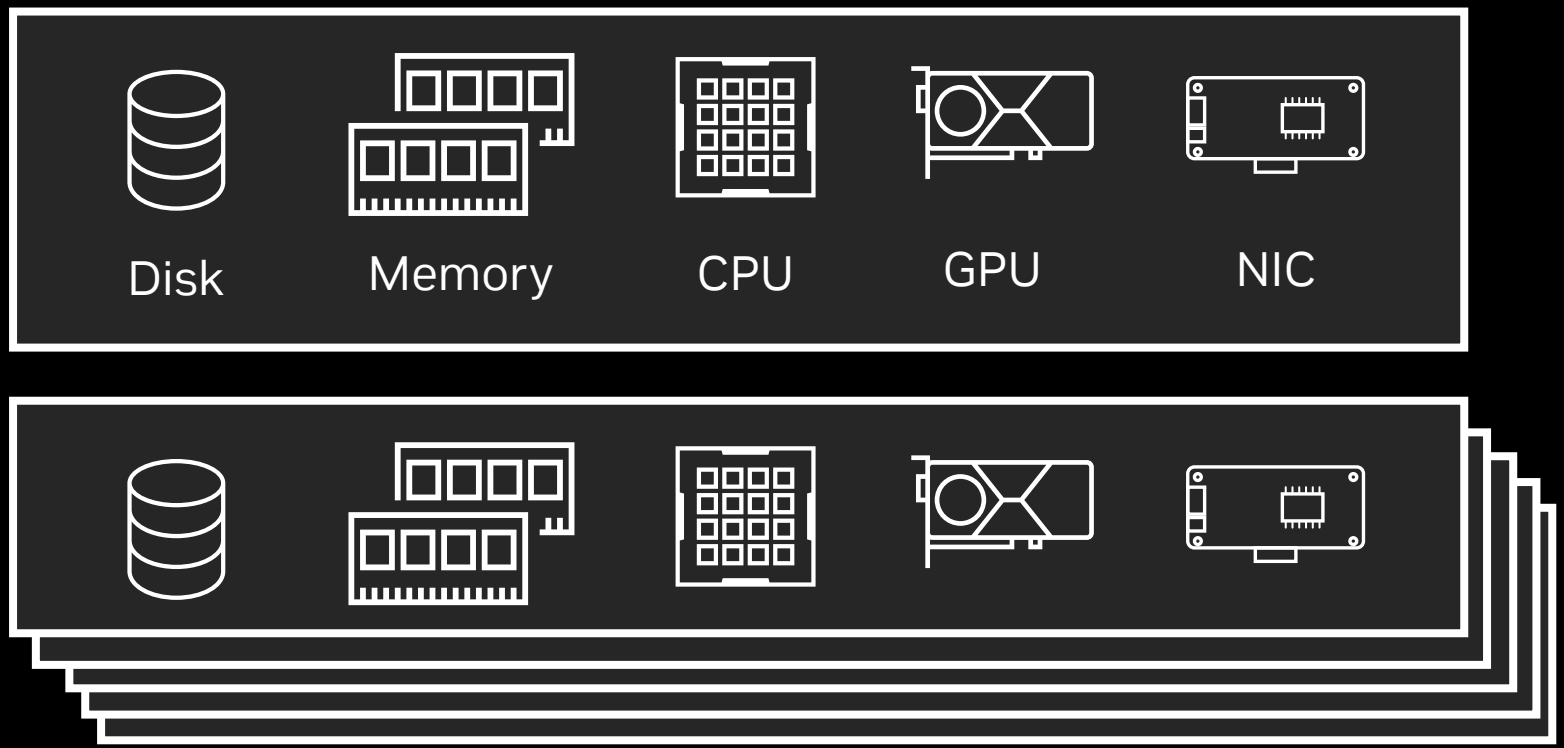
- **communicators** encapsulate communication spaces for library safety
- **datatypes** reduce copying costs and permit heterogeneity
- multiple **communication modes** allow more control of memory buffer management
- extensive **collective operations** for scalable global communication
- **process topologies** permit efficient process placement, user views of process layout
- **profiling interface** encourages portable tools

It Scales!

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

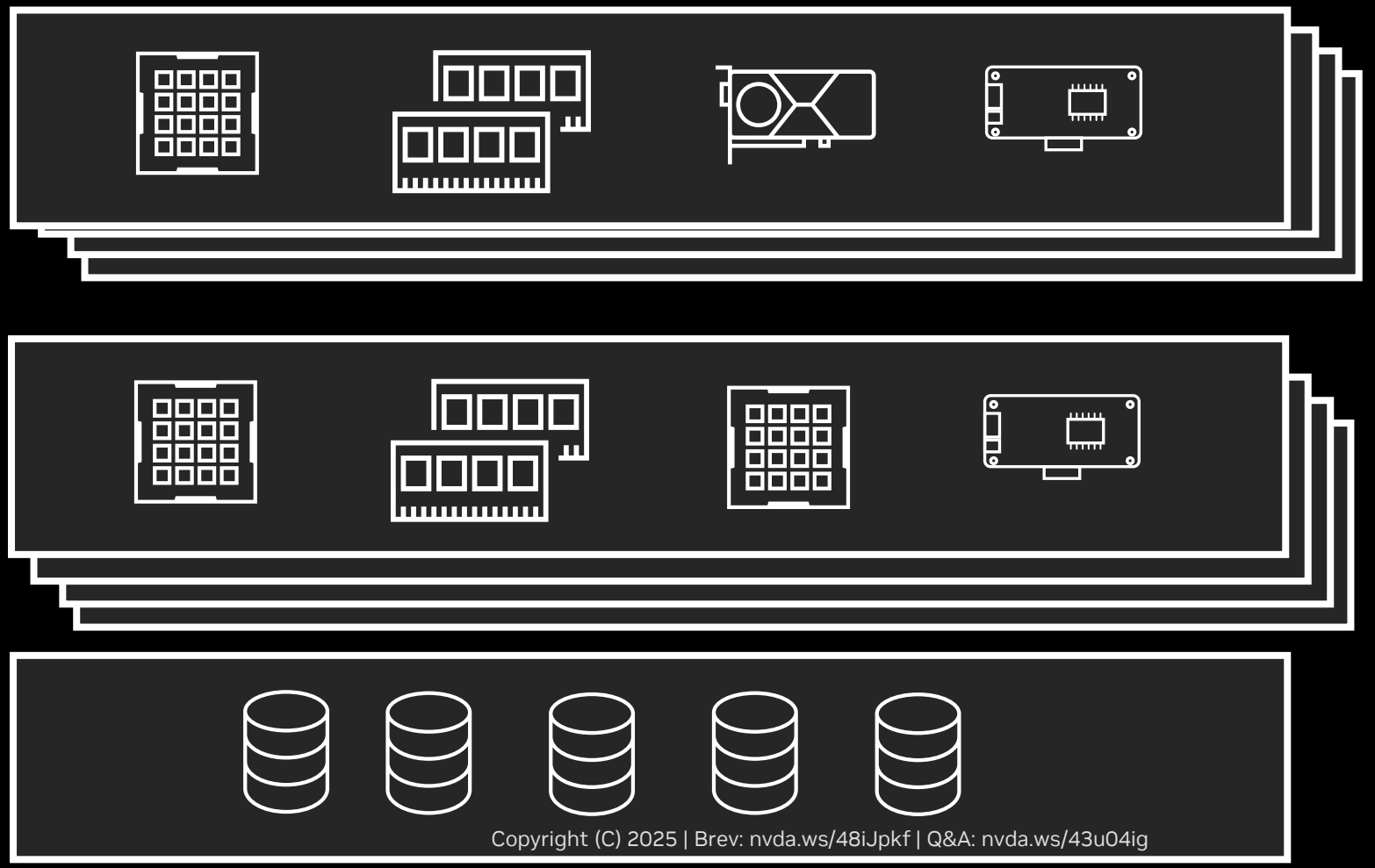
What is a Distributed System?

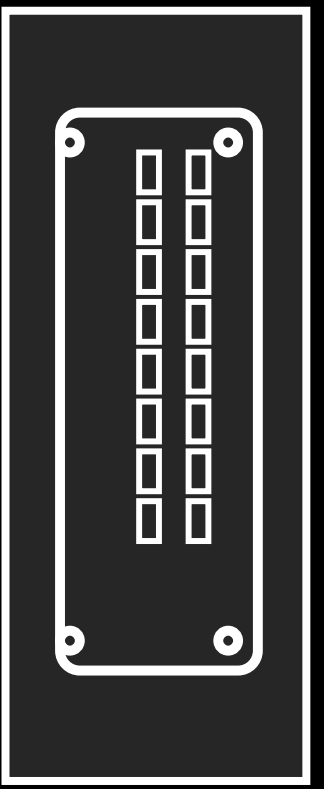
Node



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Distributed Systems are often different





Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

MPI Simplifies HPC



Most typical to have 1 program control the entire system

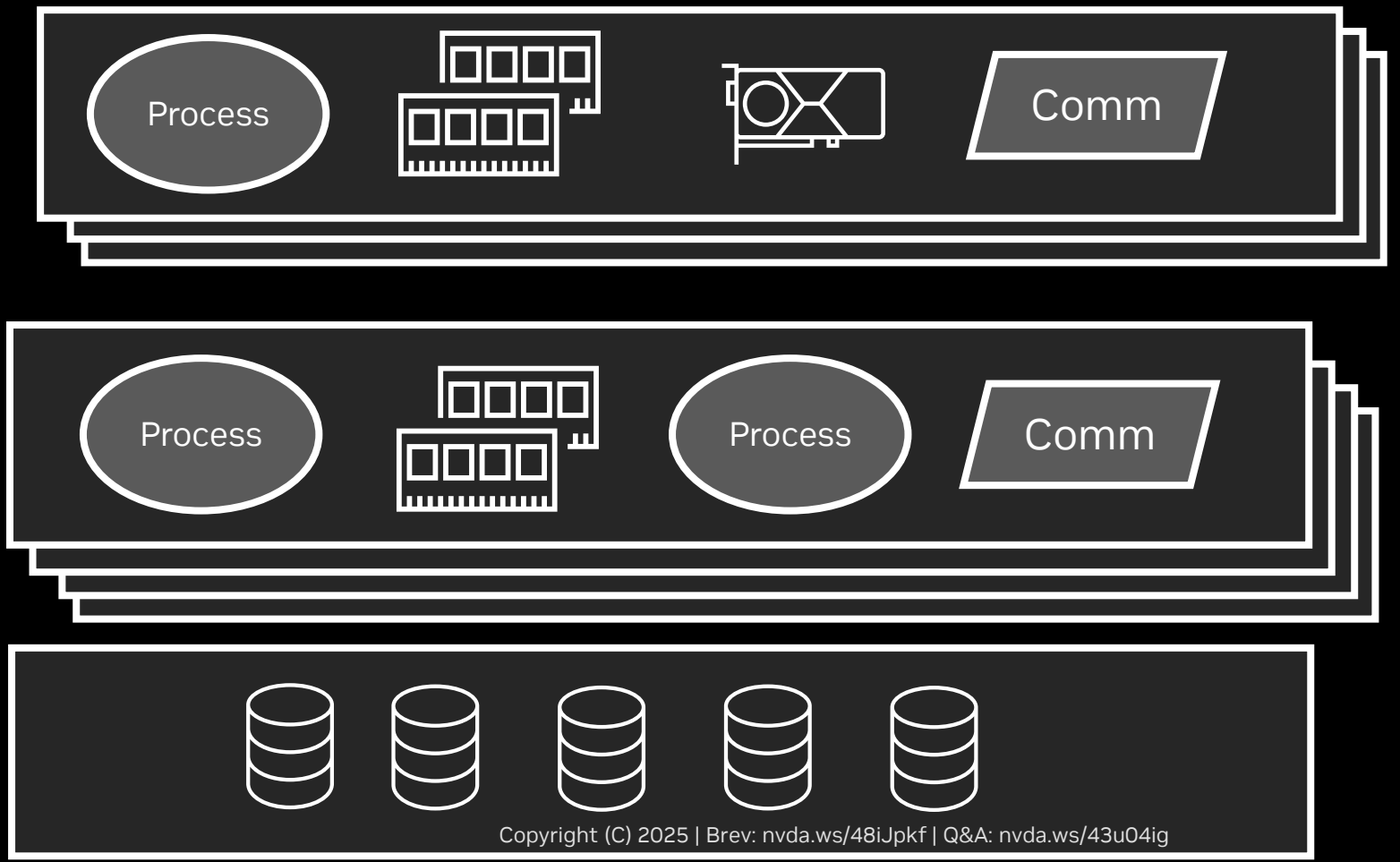
Typically, one per
- processor,
- socket, or
- gpu

Beyond our scope but could be managed by filesystem or a parallel IO job via MPI

Hardware independent ways to organize communications.
Could be:
- Local memory
- Ethernet network
- Infiniband network
- etc

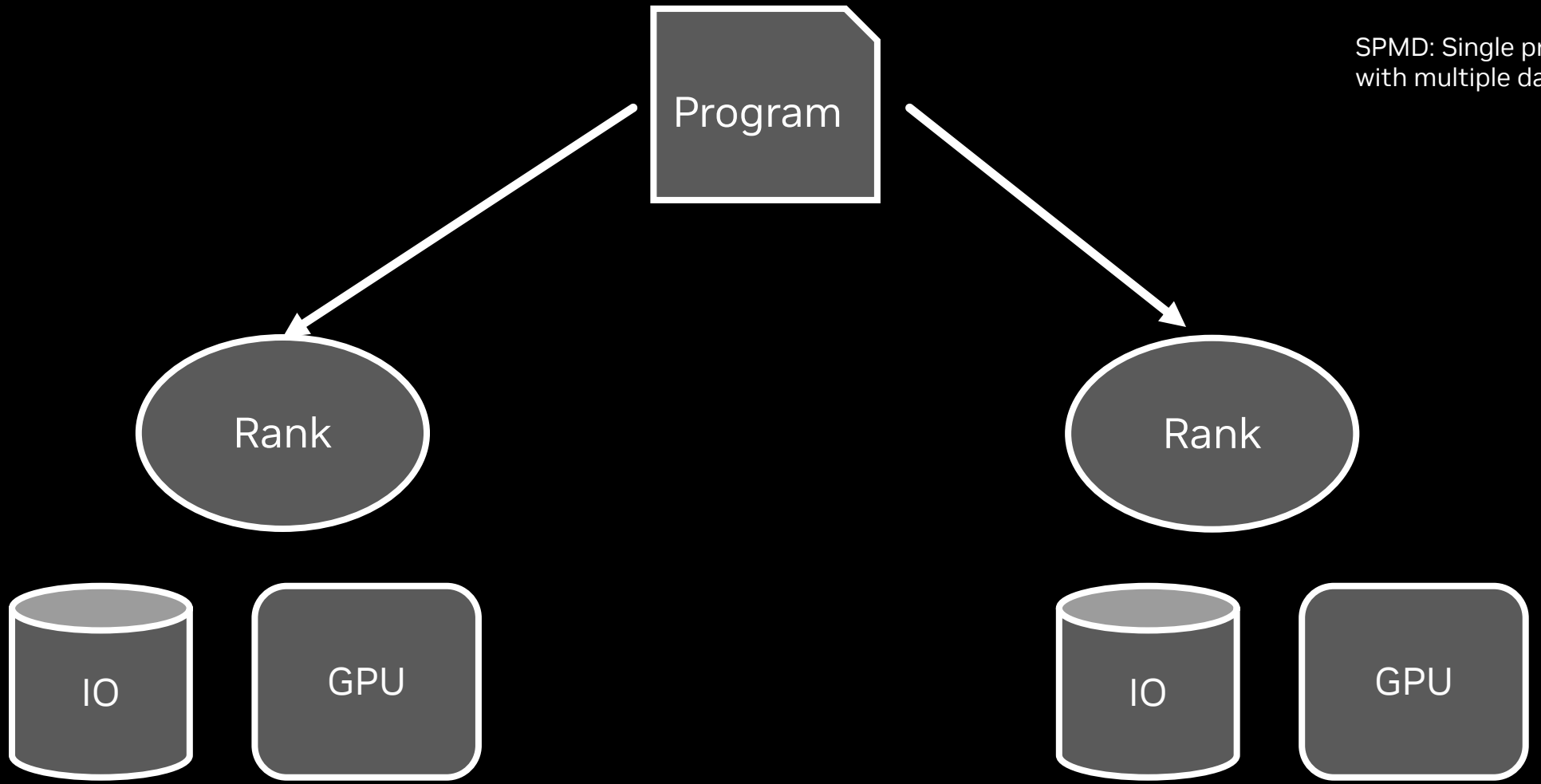
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

MPI Simplifies HPC



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Message Passing Paradigm



SPMD: Single program with multiple data

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Hello World

```
from mpi4py import MPI

print("Hello World!")
```

From a console in jupyter

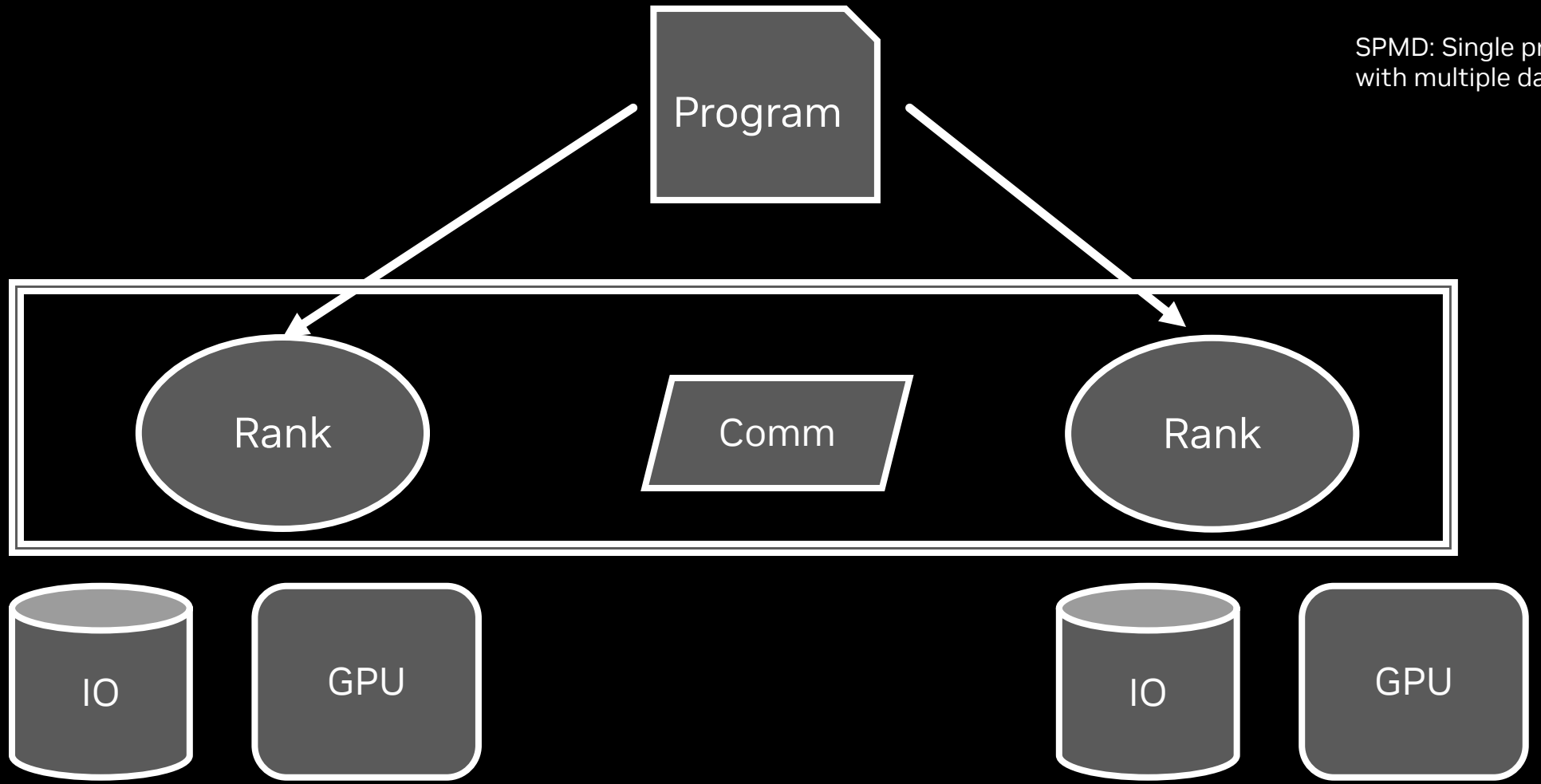
> source /global/common/software/trn018/init-training-pyhpc-2025.sh

> srun -n 8 -c 8 python hello_world.py

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Message Passing Paradigm

SPMD: Single program with multiple data



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Hello World With Rank Data

```
import os

from mpi4py import MPI

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

# set up basic variables
name = MPI.Get_processor_name()
pid = os.getpid()

print(f"Hello World! I am process {rank} of {size} on {name} "
       "with pid {pid}.\n")
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Communicators

- processes can be collected into **groups**
- each message is sent in a **context**, and must be received in the same context
- a **communicator** encapsulates a context for a specific group
- a given program may have many communicators with any level of overlap
- two initial communicators
  - MPI_COMM_WORLD (all processes)
  - MPI_COMM_SELF (current process)

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

mpi4py initialization

- mpi4py automatically calls MPI_Init() and MPI_Finalize()
- MPI_Init() is called when you import the MPI module from mpi4py
- MPI_Finalize() is called before the Python process ends
- If you need explicit control, use the mpi4py.rc module to configure before importing MPI

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

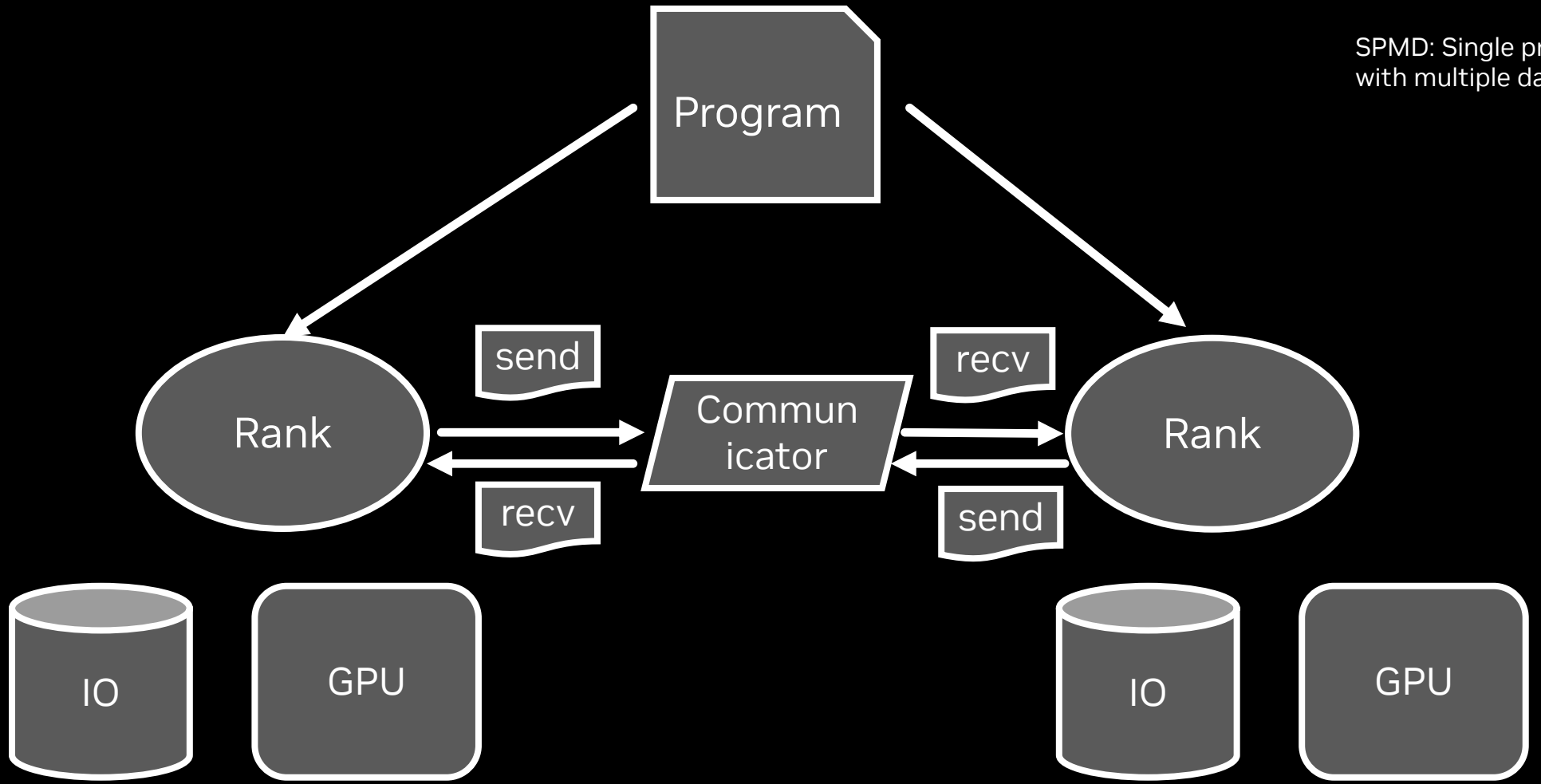
Exercise 1

Use the rank to print “fizz” if it divisible by 2, ”buzz” if divisible by 3, and “fizzbuzz” if both.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Message Passing Paradigm

SPMD: Single program with multiple data



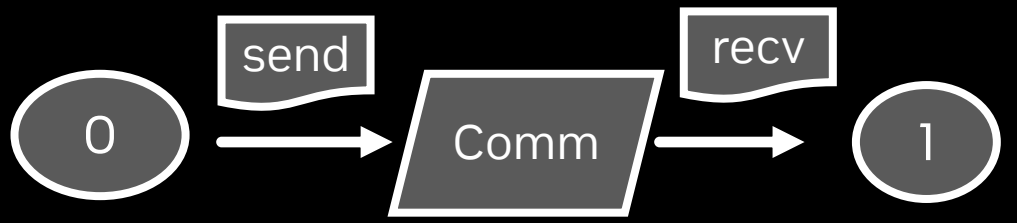
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

mpi4py Blocking Send / Recv

```python
from mpi4py import MPI

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

if rank == 0:
    data = {'a': 7, 'b': 3.14}
    comm.send(data, dest=1, tag=11)
elif rank == 1:
    data = comm.recv(source=0, tag=11)
    print(data)
```



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

mpi4py

Two interfaces for mpi commands:

- Generic API with lowercase function names, e.g. Comm.send
- Efficient API with titlecase function names, e.g. Comm.Send
  - The efficient API can still handle default arguments and type discovery for NumPy arrays and PEP-3118 buffers

Initialization

- Generic API with lowercase function names, e.g. Comm.send
- Efficient API with titlecase function names, e.g. Comm.Send
  - The efficient API can still handle default arguments and type discovery for NumPy arrays and PEP-3118 buffers

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

mpi4py Efficient API

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

if rank == 0:
    data = np.arange(10, dtype='i’)
    comm.Send([data, MPI.INT], dest=1, tag=77)
elif rank == 1:
    data = np.empty(10, dtype='i’)
    comm.Recv([data, MPI.INT], source=0, tag=77)
    print(data)
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Message Data Types

- the data in a message to send or receive is described by address, count and datatype

- a datatype is recursively defined as:
  - predefined, corresponding to a data type from the language (e.g., MPI_INT, MPI_DOUBLE)
  - a contiguous, strided block, or indexed array of blocks of MPI datatypes
  - an arbitrary structure of datatypes

- there are MPI functions to construct custom datatypes

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Message Tags

- messages are sent with an accompanying user-defined integer tag to assist the receiving process in identifying the message
- messages can be screened at the receiving end by specifying the expected tag, or not screened by using MPI_ANY_TAG

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

mpi4py numpy API

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

if rank == 0:
    data = np.arange(10, dtype=np.float64)
    comm.Send(data, dest=1, tag=13)
elif rank == 1:
    data = np.empty(10, dtype=np.float64)
    comm.Recv(data, source=0, tag=13)
    print(data)
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

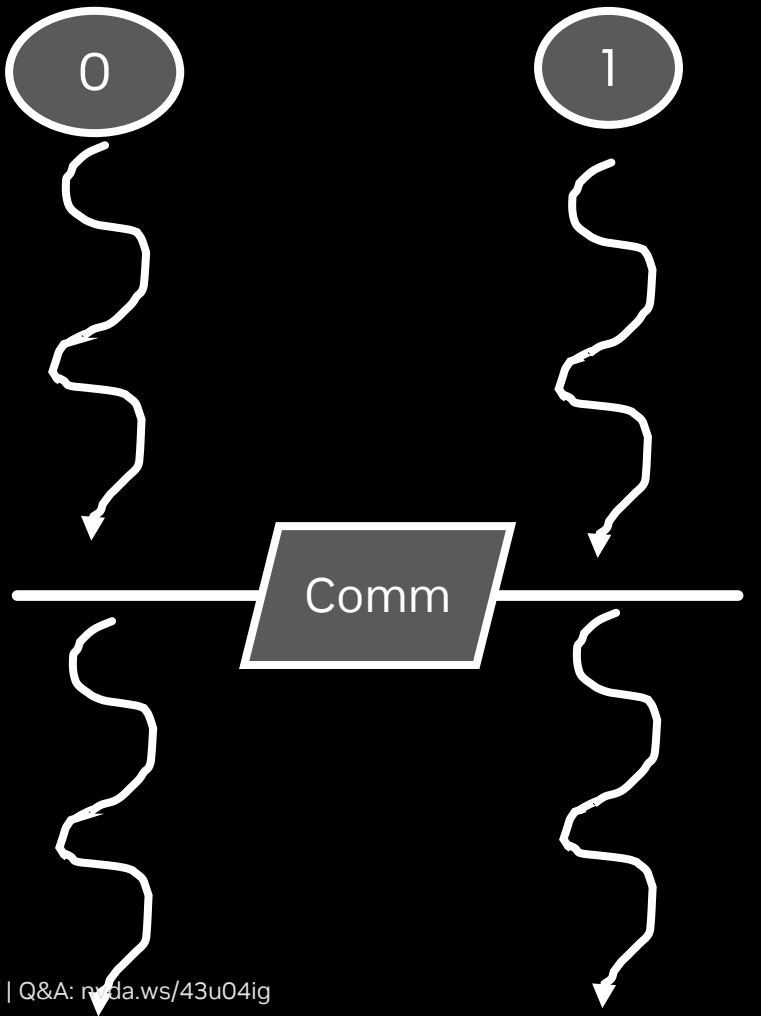
Barriers

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

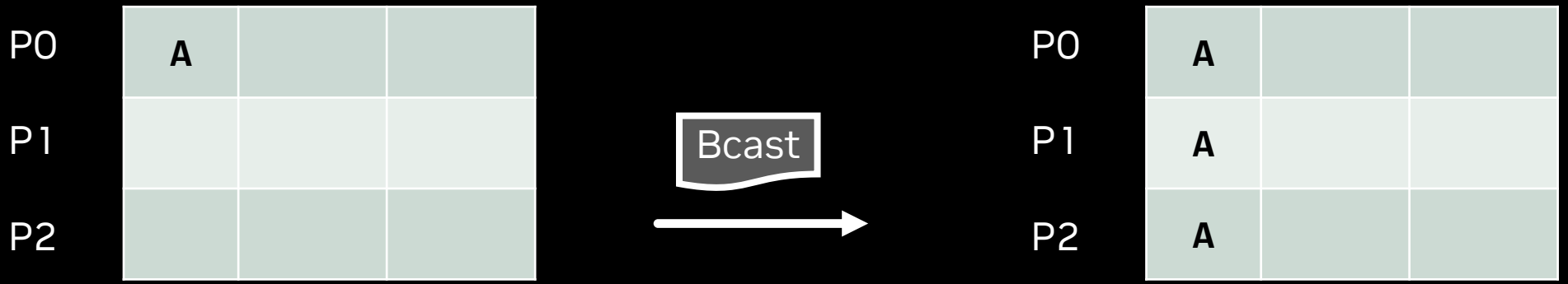
for r_id in range(comm.Get_size()):
    if rank == r_id:
        print(f"Hello from proc:")

comm.Barrier()
print(f"proc {rank} after barrier")
```



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Broadcast



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Bcast

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

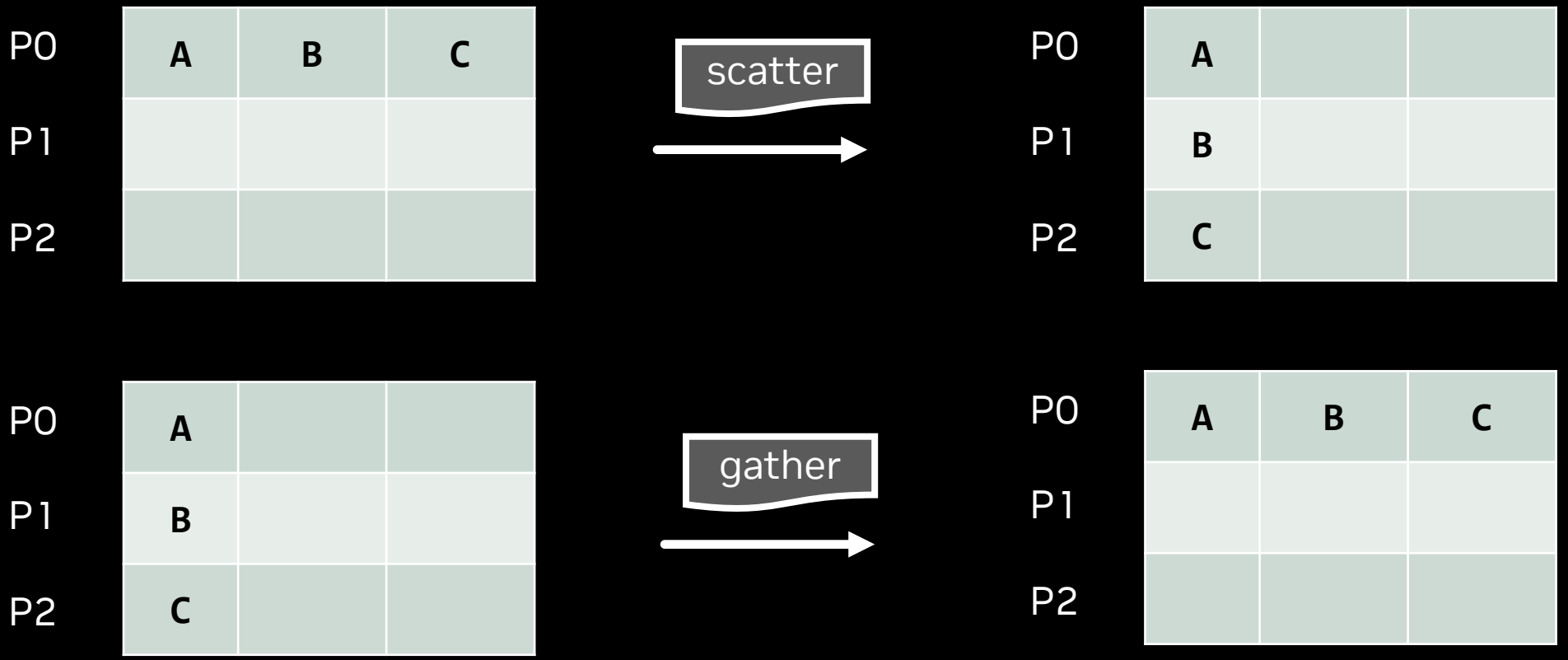
if rank == 0:
    data = {'key1' : [7, 2.72, 2+3j],
            'key2' : ( 'abc', 'xyz')}
else:
    data = None

data = comm.bcast(data, root=0)

print(f"bcast finished and data "
      "on rank %d is: {comm.rank}, {data}")
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Scatter / Gather



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

scatter

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    data = [(i+1)**2 for i in range(size)]
else:
    data = None

data = comm.scatter(data, root=0)

assert data == (rank+1)**2
print(f"data on rank %d is: {rank}, {data}")
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

gather

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

data = (rank+1)**2
print(f"before gather, data on rank %d is: {rank}, {data}")

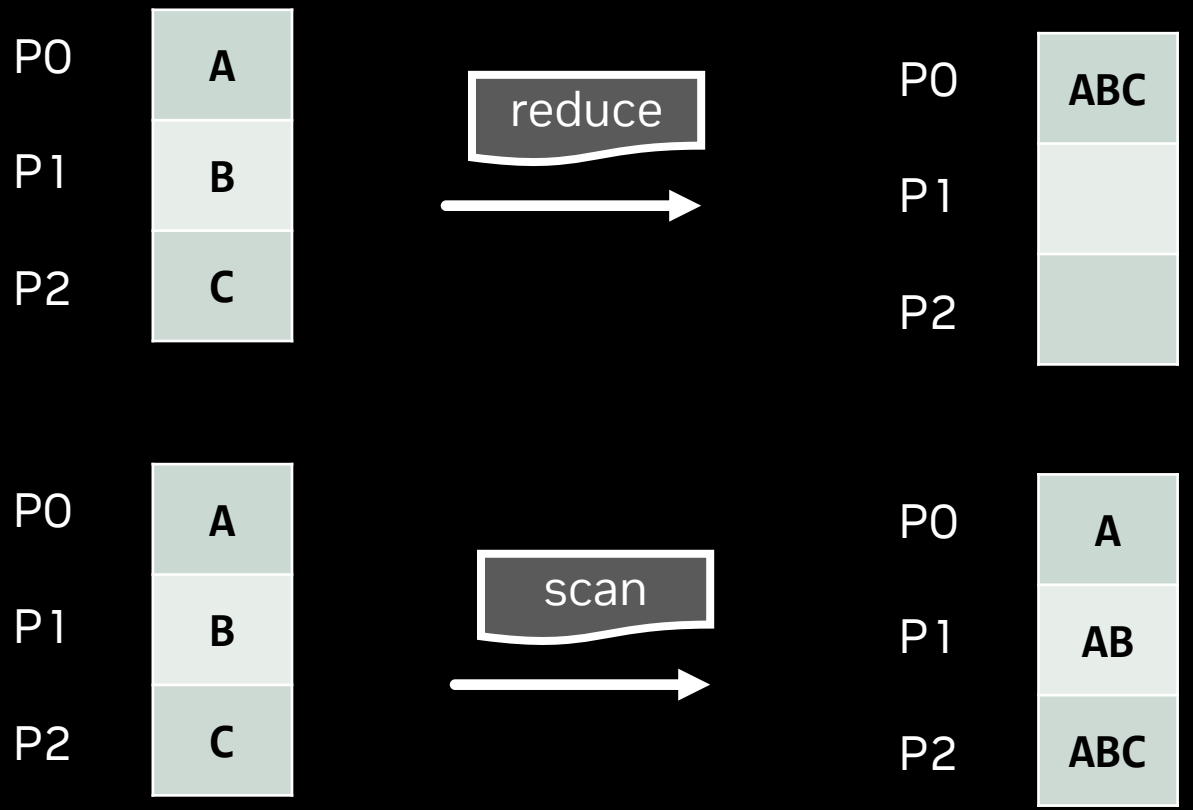
comm.Barrier()
data = comm.gather(data, root=0)

if rank == 0:
    for i in range(size):
        assert data[i] == (i+1)**2
else:
    assert data is None

print(f"data on rank: {rank} is: {data}")
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Reduce / Scan



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

reduce

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

sendmsg = comm.rank
recvmsg1 = comm.reduce(sendmsg, op=MPI.SUM, root=0)
recvmsg2 = comm.allreduce(sendmsg)
print recvmsg2
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

GPU Aware MPI

Several MPI implementations, including Open MPI and MVAPICH, support passing GPU pointers to MPI calls to avoid explicit data movement between host and device

This feature requires:

- mpi4py is built against a GPU-aware MPI library.
- The Python GPU arrays are compliant with either of the protocols.



CuPy







Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

CuPy Arrays

```
from mpi4py import MPI
import cupy as cp


comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()


sendbuf = cp.arange(10, dtype='i')
recvbuf = cp.empty_like(sendbuf)
cp.cuda.get_current_stream().synchronize()
comm.Allreduce(sendbuf, recvbuf)


assert cp.allclose(recvbuf, sendbuf*size)
```

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

Computing multiple computers

GO BIGGER!!!!

Weak Scaling

Data is growing and I want more to add more computers as it grows

Bigger Data, Same time to compute

Limited by Denard Scaling

GO FASTER!!!!

Strong Scaling

Data is fixed and I want more computers to go faster

Same size data, Faster time to compute

Limited by Amdahl's Law

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

What We Learned Today

- NumPy: Array-oriented programming in Python.
- CuPy: CUDA-accelerated NumPy, linear algebra, & scientific algos.
- mpi4py: Scaling out with distributed programming.
- Benchmarking and profiling.
- CUDA model and performance fundamentals.

Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

GTC CUDA Developer Track
nvda.ws/4dVjYXs

Accelerated Computing Hub
nvda.ws/43tdsU6

GENERAL CUDA

S72571 What's CUDA All About Anyways?
S72897 How To Write A CUDA Program
S72527 Debugging & Optimizing CUDA with Intelligent Developer Tools

CUDA PYTHON

S72450 Accelerated Python: Tour of Community & Ecosystem
S72448 The CUDA Python Developer's Toolbox
S72449 1001 Ways to Write CUDA Kernels in Python
S74639 Tensor Core Programming in Python with CUTLASS 4.0

CUDA C++

S72574 Building CUDA Software at the Speed-of-Light
S72572 The CUDA C++ Developer's Toolbox
S72575 How You Should Write a CUDA C++ Kernel

MULTI-GPU PROGRAMMING

S72576 Getting Started with Multi-GPU Scaling: Distributed Libraries
S72579 Going Deeper with Multi-GPU Scaling: Task-based Runtimes
S72578 Advanced Multi-GPU Scaling: Communication Libraries

PERFORMANCE OPTIMIZATION

S72683 CUDA Techniques to Maximize Memory Bandwidth & Hide Latency
S72685 CUDA Techniques to Maximize Compute & Instruction Throughput
S72686 CUDA Techniques to Maximize Concurrency & System Utilization
S72687 Get the Most Performance From Grace Hopper

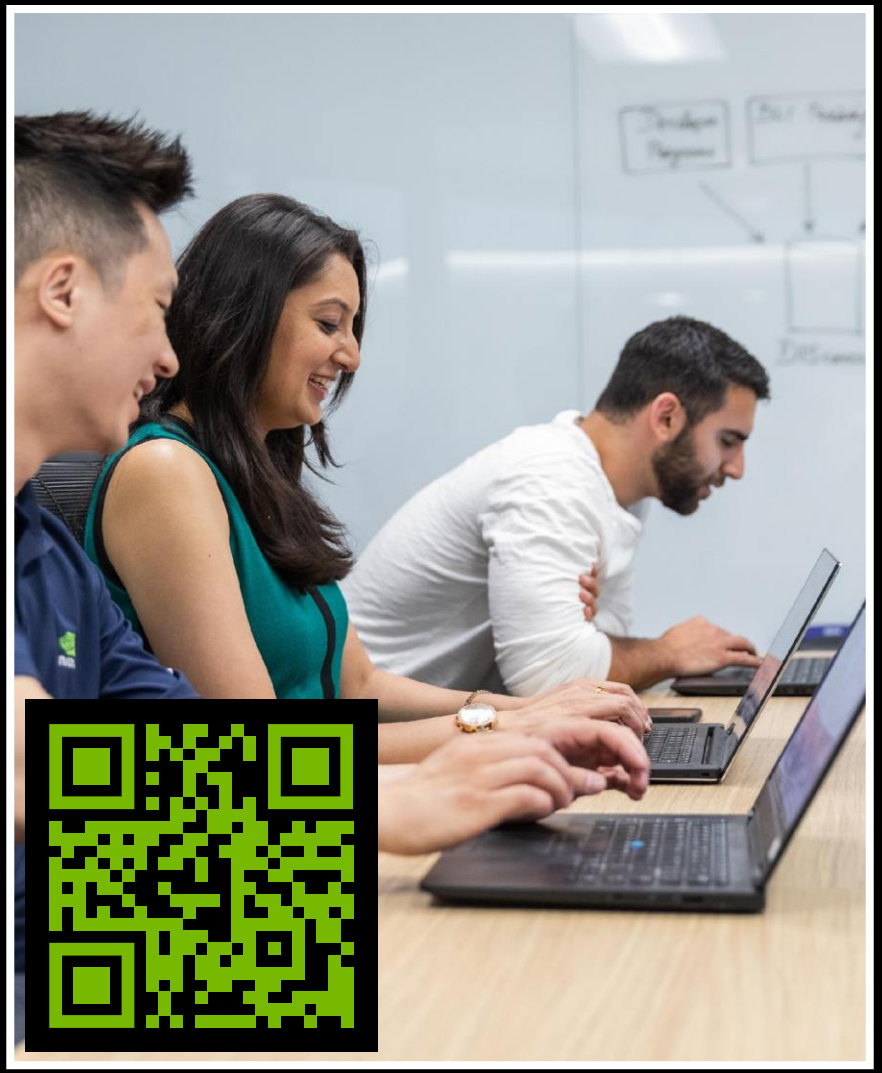
Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

NVIDIA DLI Courses

- Fundamentals of Accelerated Computing with CUDA Python
- Fundamental of Accelerated Computing with Modern CUDA C++
- Fundamentals of Accelerated Data Science
- Accelerate Data Science Workflows with Zero Code Changes
- Optimizing CUDA Machine Learning Codes With Nsight Profiling Tools

Scan the QR code to access the full course list and redeem one free training!

nvda.ws/3ZLPyAL



Copyright (C) 2025 | Brev: nvda.ws/48iJpkf | Q&A: nvda.ws/43u04ig

GTC CUDA
Developer Track
nvda.ws/4dVjYXs

Accelerated
Computing Hub
nvda.ws/43tdsU6

NVIDIA
DLI Courses
nvda.ws/3ZLPyAL

ADSP
Podcast
adspthepodcast.com

